



UNIVERSITÀ DEGLI STUDI ROMA TRE
Dipartimento di Ingegneria Civile, Informatica e
delle Tecnologie Aeronautiche
Corso di Laurea in Ingegneria Informatica

Tesi di Laurea

Sviluppo di un Simulatore High-Fidelity per Vele Solari in Unreal Engine 5

Laureando
Edoardo Conforti
Matricola 578163

Relatore
Prof. Franco Milicchio

Anno Accademico 2024/25

A [dedica]

Introduzione

Il problema della propulsione spaziale e i limiti chimici

Dagli inizi dei razzi suborbitali sino alle più recenti missioni interplanetarie, tutta la storia dell'esplorazione spaziale si è in sostanza giocata, e si gioca tuttora, sull'impiego della propulsione chimica, in base al principio della conservazione della quantità di moto, o della terza legge della dinamica di Newton, il quale afferma che per ottenere la spinta necessaria a far muovere in una direzione determinata un veicolo, esso deve accelerare ed espellere nella direzione opposta una determinata massa, propellente [1].

Seppur questa sia l'unica strategia con la quale si riesca a fornire una spinta sufficiente per vincere la forza di gravità terrestre durante le fasi di lancio, essa si rivela essere problematica per le missioni nello spazio profondo. Il paradosso alla base della propulsione classica è che per trasportare il propellente necessario a far accelerare il veicolo, è richiesto ulteriore propellente per far accelerare il propellente stesso, causando un circolo vizioso che limita drasticamente le capacità potenziali di una missione [2].

Per comprendere visivamente l'entità di questo paradosso, è sufficiente osservare le proporzioni di qualsiasi razzo storico o contemporaneo: gran parte del volume e della massa totale al momento del lancio è costituita dagli enormi serbatoi di propellente, lasciando alla navicella vera e propria — e al carico scientifico utile (*payload*) — una frazione di massa che spesso si attesta al di sotto del 2% del peso complessivo.



Figura 1: Rappresentazione visiva della sproporzione di massa in un lanciatore chimico: gran parte del volume è dedicata al propellente.

Con limiti di prestazioni così severi, missioni interplanetarie complesse o missioni che richiedono orbite più articolate di quelle kepleriane classiche diventano impossibili con la propulsione tradizionale [3]. Per raggiungere variazioni di velocità degne di nota, è necessario trasportare enormi quantità di propellente, che a sua volta richiede strutture più robuste e più costose, aumentando esponenzialmente i costi di lancio e limitando la fattibilità di missioni ambiziose [4].

Inoltre, una volta che il propellente si è esaurito, il veicolo perde totalmente la capacità di manovrare, diventando un oggetto controllato esclusivamente dalle leggi della meccanica orbitale. L'esigenza di liberarsi da questa catena logistica e di abilitare nuovi generi di missioni nello spazio profondo ha spinto la ricerca verso l'adozione di propulsori *propellantless*, dove la fonte di energia per la spinta non risiede nel veicolo stesso, ma viene attinta direttamente dall'ambiente spaziale [4].

Il principio della propulsione a vela solare

Nasce quindi l'esigenza di svincolarsi dalle limitazioni della propulsione chimica per favorire lo sviluppo di tecnologie propulsive alternative che possano fornire una spinta continua al veicolo senza richiedere il trasporto di una grande massa di propellente. Tra varie soluzioni proposte nel tempo, la propulsione a vela solare è senza dubbio una delle più promettenti e affascinanti, ma richiede un cambio radicale nello sviluppo di missioni spaziali.

L'intuizione storica e la definizione concettuale

L'idea che la luce, per sua stessa natura, eserciti una spinta sui corpi, non è una invenzione del secolo attuale, ma è stata tramandata dall'astronomia del secolo XVII. Nel 1619, osservando il passaggio di una cometa, l'astronomo Johannes Keplero notò che la coda di polveri e di gas si proiettava sempre nella direzione opposta al sole, senza tener conto della strada percorsa dal corpo celeste, ipotizzando l'esistenza di una vera e propria "brezza solare".

Oggi è stato chiarito come questa intuizione si traduca in una tecnologia concreta. Una vela solare è un dispositivo di propulsione spaziale che non utilizza alcun tipo di propellente, ma sfrutta la pressione di radiazione solare (SRP, *Solar Radiation Pressure*) per generare spinta. La caratteristica distintiva di questo sistema è che la forza propulsiva non deriva da una reazione chimica interna, ma dall'interazione dinamica tra i fotoni emessi dal Sole e una superficie riflettente estremamente leggera e sottile, chiamata appunto "vela".

Tale fenomeno fu predetto teoricamente per la prima volta da James Clerk Maxwell nella seconda metà dell'Ottocento, attraverso le sue equazioni dell'elettromagnetismo che dimostrano come i campi elettromagnetici trasportino non solo energia ma anche quantità di moto, esercitando così una forza su superfici riflettenti [1] [4]. Sebbene i fotoni siano particelle prive di massa, essi possiedono una quantità di moto direttamente proporzionale alla loro energia. Quando un flusso di fotoni colpisce la superficie riflettiva della vela, si genera una variazione della quantità di moto che, per il principio di conservazione, imprime una forza sulla superficie della vela [2].



Figura 2: Effetto della pressione di radiazione solare e del vento solare sulla coda di una cometa, fenomeno osservato originariamente da Keplero.

Vantaggi e sfide ingegneristiche dei materiali

Il vantaggio principale e ineguagliabile della vela solare risiede in un impulso specifico teoricamente infinito. Non espellendo alcun propellente, infatti, il veicolo è limitato unicamente dalla degradazione dei suoi materiali nel tempo e non dalla capacità dei suoi serbatoi. Questa caratteristica si traduce in una spinta continua che, accumulata su mesi o anni di missione, permette di raggiungere variazioni di velocità totali sbalorditive, inarrivabili per qualsiasi propulsore chimico [2].

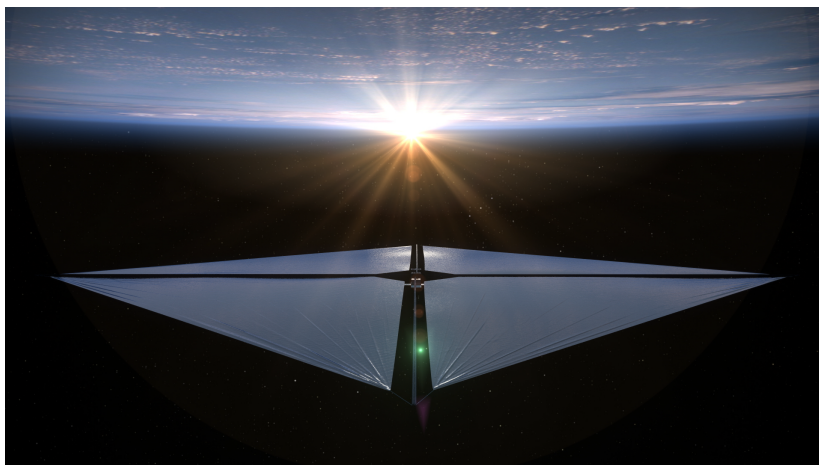


Figura 3: Rappresentazione concettuale di una vela solare dispiegata nello spazio profondo, evidenziando le ampie superfici riflettenti in materiale polimerico.

Di contro, la spinta fotonica presenta svantaggi significativi che derivano dalla sua intensità. La pressione di radiazione solare a 1 AU (Unità Astronomica) è estremamente debole [2]. Di conseguenza, per generare accelerazioni utili, sono necessarie vele di dimensioni enormi accoppiate a masse di payload minime.

Questo solleva enormi problemi di resistenza dei materiali, di dispiegamento della vela e di controllo del veicolo [4]. Per ottenere masse contenute, la membrana della vela deve possedere spessori dell'ordine di pochissimi micrometri (una frazione dello spessore di un capello umano), impiegando avanzati film polimerici come il Kapton o il Mylar. Inoltre, la spinta decade rapidamente spingendosi verso il sistema solare esterno, seguendo la rigorosa legge dell'inverso del quadrato della distanza, limitando l'impiego primario di queste sonde al sistema solare interno o a traiettorie di fuga solari ravvicinate.

Vele solari nella storia dell'esplorazione spaziale

Nonostante le immense sfide riguardanti la progettazione e la scienza dei materiali, la propulsione fotonica ha riscosso successo per quanto riguarda la validazione dei modelli teorici e la dimostrazione pratica. Tra le prime missioni, la più celebre è senz'altro IKAROS (Interplanetary Kite-craft Accelerated by Radiation Of the Sun), lanciata dall'agenzia spaziale giapponese JAXA nel 2010.

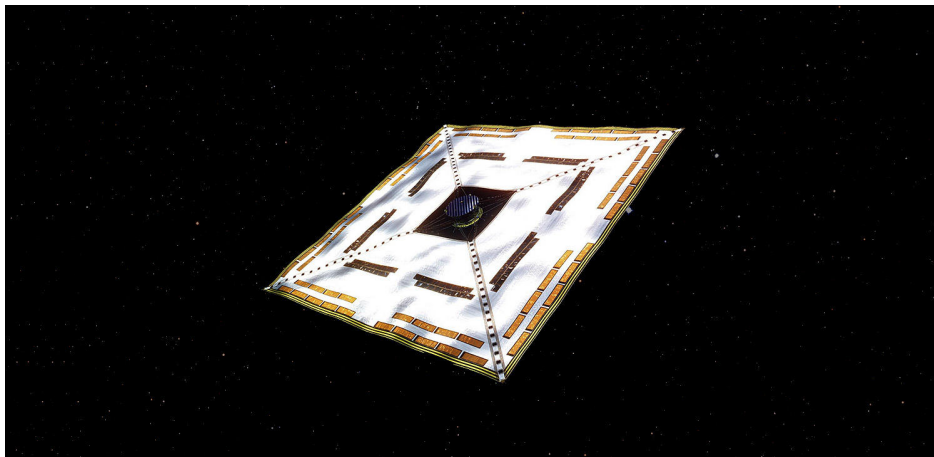


Figura 4: La sonda IKAROS della JAXA, la prima a dimostrare con successo la propulsione fotonica interplanetaria.

IKAROS è stata la prima sonda a dimostrare con successo la propulsione a vela solare in un ambiente interplanetario, dispiegando tramite forza centrifuga una vela quadrata di 14 metri di lato [1] [4]. Il veicolo ha persino dimostrato la capacità di alterare il proprio assetto variando la riflettività di specifici pannelli LCD disposti agli estremi della vela.

Nello stesso periodo, nel 2010 la NASA ha lanciato NanoSail-D2, progettata per testare la tecnologia di dispiegamento su piattaforme miniaturizzate CubeSat, osservando effetti pratici significativi [4]. Seconda per importanza mediatica e ingegneristica è la missione LightSail 2 della *Planetary Society*, lanciata nel 2019 e finanziata interamente dal pubblico. La piccola sonda ha dimostrato con successo la fattibilità del controllo

INTRODUZIONE

dell'orbita terrestre, innalzando il proprio apogeo utilizzando esclusivamente la spinta fotonica generata dalla vela solare [5].

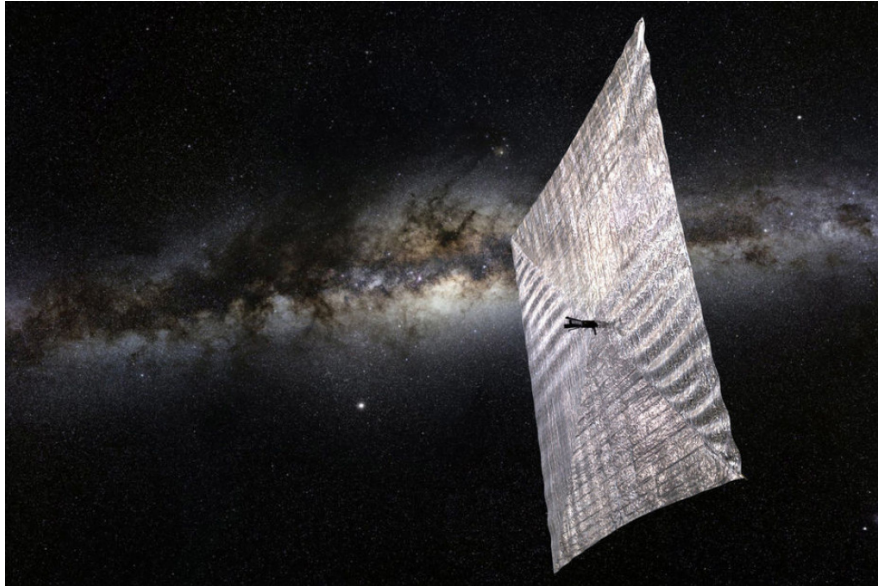


Figura 5: La missione LightSail 2 in orbita terrestre, progettata dalla Planetary Society per dimostrare il controllo e l'innalzamento orbitale.

Queste missioni confermano inequivocabilmente che la vela solare non è più una speculazione teorica, ma una piattaforma tecnologica pronta per l'uso operativo. Tuttavia, la progettazione di tali veicoli richiede sistemi di *testing* e simulazione sempre più accurati, in grado di gestire le complesse interazioni vettoriali in tempo reale prima della fase di fabbricazione delle componenti.

Motivazioni e scopi della simulazione software

Come già evidenziato, la tecnologia a vela solare ha sicuramente bisogno di un lungo processo di sviluppo e di prototipazione. La natura stessa della spinta fotonica si scontra con le limitazioni fisiche e ambientali del nostro pianeta, rendendo la fase di test hardware delle componenti molto complessa e costosa.

Il limite fondamentale nella sperimentazione fisica risiede nella gigantesca sproporzione tra le forze in gioco. Sulla superficie terrestre, l'infinitesimale spinta fotonica è completamente impercettibile: infatti essa viene istantaneamente annullata da forze ambientali molto più intense come il vento, l'attrito atmosferico e la gravità. Di conseguenza, testare la dinamica di volo o il dispiegamento dei bracci telescopici di una vela solare all'aria aperta è fisicamente impossibile: la gravità farebbe collassare su se stesse le delicatissime strutture.

D'altro canto, le camere a vuoto termico (TVAC) utilizzate per i test satellitari non possiedono quasi mai le dimensioni sufficienti per testare una vela completamente dispiegata (che può raggiungere l'estensione di un campo da tennis). L'alternativa definitiva, ovvero il lancio diretto in orbita di dimostratori tecnologici in scala reale, comporta costi esorbitanti e un rischio di fallimento non indifferente.

In questo connubio di fenomeni complessi e di vasti interessi, la simulazione software costituisce un'indispensabile chiave per superare tale ostacolo, permettendo agli ingegneri di progettare, di sperimentare e di ottimizzare i veicoli in un ambiente virtuale, che riproduce fedelmente le condizioni spaziali. Attraverso la simulazione, è possibile condurre un'analisi parametrica completa, variando la massa del veicolo, l'estensione e i coefficienti di riflettività per osservare come influenzino le traiettorie risultanti senza alcun costo di fabbricazione.

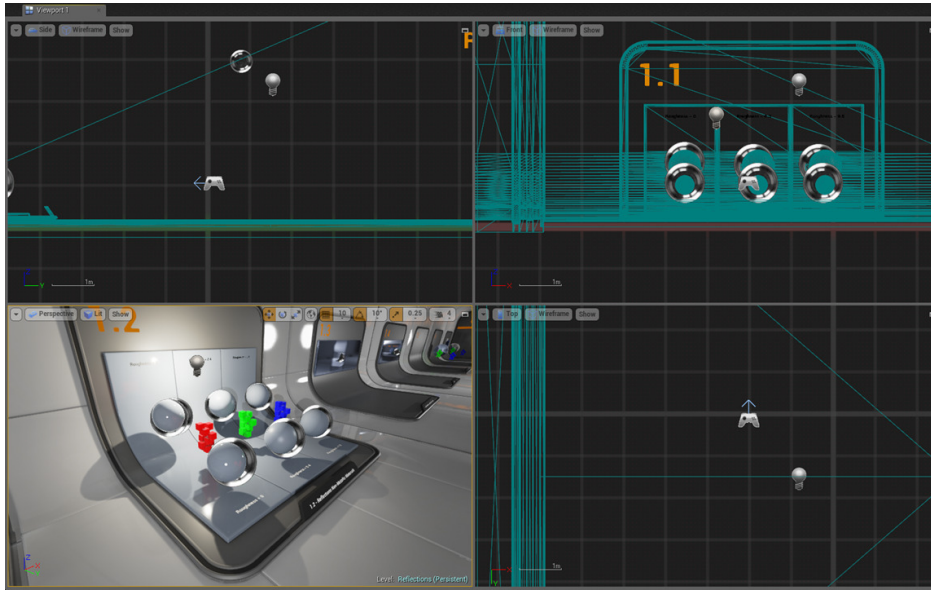


Figura 6: Esempio di ambiente di simulazione virtuale: i motori grafici permettono di superare i limiti fisici dei test di laboratorio a terra.

L'obiettivo primario di questo elaborato si sposa con tale necessità: sviluppare un ambiente di simulazione *High-Fidelity* per vele solari che unisca il rigore dell'integrazione fisica numerica con le potenzialità visive e interattive di un motore grafico moderno (Unreal Engine 5). La scelta di adottare un *game engine* non è di natura puramente estetica, bensì offre precisi vantaggi ingegneristici:

- **Esecuzione in tempo reale (Real-time):** Permette all'utente di alterare i parametri fisici a runtime e osservare istantaneamente le variazioni cinematiche e di moto.
- **Gestione spaziale e Ray Casting:** I motori grafici sono nativamente ottimizzati per algoritmi di intersezioni spaziali ad altissime prestazioni. Sfruttando le tecniche di *ray casting* [6], è possibile simulare il flusso fotonico come un fascio di raggi proiettati sul modello 3D della vela, calcolando con precisione vettoriale l'angolo di incidenza e l'ombreggiamento in tempo reale.
- **Telemetria visiva:** La capacità di renderizzare a schermo i vettori tridimensionali della forza, della velocità e dell'assetto fornisce all'ingegnere uno strumento di *Visual Telemetry* insostituibile per la comprensione immediata di fenomeni astrodinamici complessi.

Obiettivi specifici del progetto di tesi

L'applicativo software sviluppato nel contesto di questa ricerca è concepito per modellare il comportamento cinematico e dinamico di una o più vele solari sottoposte all'influenza del campo gravitazionale e della pressione solare. Lo sviluppo ha avuto tre obiettivi ingegneristici primari:

1. **Validazione del modello fisico:** Traduzione rigorosa delle equazioni dell'ottica e della meccanica orbitale in algoritmi C++ efficienti, calcolando la forza gravitazionale e la complessa pressione fotonica vettoriale in tempo reale per aggiornare la posizione del veicolo nello spazio tridimensionale.
2. **Sviluppo di un motore a doppia modalità:** Implementazione di un'architettura che consenta all'utente di operare in *Real-Time Mode* (per l'analisi dettagliata e visiva del controllo d'assetto e delle micro-manovre) e in *Accelerated Time Mode* (per la proiezione e l'analisi di scenari orbitali a lungo termine, della durata di mesi o anni simulati in pochi secondi).
3. **Telemetria visiva e tracciamento dati persistente:** Sviluppo di un sistema di *visual debugging* direttamente posto nel viewport 3D dell'editor e di una procedura di esportazione dati (in formato CSV) per campionare e analizzare esternamente l'integrità della telemetria orbitale prodotta dal motore fisico.

Struttura dell'elaborato

Al fine di esporre in modo esaustivo il lavoro svolto, il presente elaborato è strutturato come segue:

Capitolo 1 - Fondamenti Teorici: Fornisce la sintesi matematica del progetto, analizzando l'equazione del razzo e le leggi vettoriali della meccanica orbitale e della pressione fotonica necessarie al calcolo delle forze.

Capitolo 2 - Architettura Software: Inquadra la progettazione orientata agli oggetti (OOP) all'interno dell'ecosistema di Unreal Engine, descrivendo la gerarchia delle classi C++ sviluppate e l'attore principale [ASolarSail](#).

Capitolo 3 - Implementazione Fisica: Mostra dettagliatamente come le equazioni ottiche non ideali siano state tradotte in logiche operative tramite l'impiego di algoritmi di *Ray Casting* in tempo reale.

Capitolo 4 - Dinamica a Tempo Discreto: Analizza i metodi di integrazione differenziale scelti per l'aggiornamento della cinematica spaziale e l'architettura per la gestione della scala temporale (*TimeScale*) accelerata.

Capitolo 5 - Telemetria e Tracciamento Dati: Descrive l'infrastruttura di *output* implementata, dal disegno vettoriale di debug su schermo fino al sistema di salvataggio a intervalli regolari dei dati di volo su file *CSV*.

Capitolo 6 - Analisi dei Risultati : Espone e analizza criticamente i grafici dei dati estratti per validare l'effettivo sollevamento orbitale e il comportamento dinamico del veicolo.

Capitolo 7 - Conclusioni e Sviluppi Futuri: Riassume i risultati ottenuti, riflette sulle limitazioni del modello attuale e propone potenziali miglioramenti e direzioni di ricerca future.

Indice

Introduzione

1	Fondamenti Teorici	1
1.1	I limiti fisici e termodinamici della propulsione chimica	1
1.1.1	Conservazione della quantità di moto ed equazione del razzo . .	1
1.1.2	Il rapporto di massa e la "tirannia" dell'equazione	2
1.1.3	Il vincolo termodinamico insuperabile	3
1.2	Fisica della radiazione elettromagnetica e pressione solare	4
1.2.1	Il dualismo onda-particella e la quantità di moto del fotone . . .	4
1.2.2	Costante solare e pressione in orbita terrestre	5
1.2.3	Decadimento spaziale e legge dell'inverso del quadrato	5
1.2.4	Luminosità solare	6
1.3	Modellazione ottica vettoriale e interazione fotonica	7
1.3.1	Area effettiva e angolo di incidenza	7
1.3.2	Fenomeni di ombreggiamento e auto-occlusione	8
1.3.3	Occlusione ambientale e asimmetria di spinta	9
1.3.4	Il modello ottico ideale (Riflessione speculare perfetta)	10
1.3.5	Il modello ottico realistico (Superfici non ideali)	11
1.3.6	L'equazione vettoriale dell'accelerazione SRP	12
1.4	Parametri di design e sintesi della dinamica orbitale	13
1.4.1	Il Carico Alare (Sail Loading)	13
1.4.2	Accelerazione Caratteristica	13
1.4.3	Numero di Leggerezza	14
1.4.4	Il campo gravitazionale primario e la meccanica orbitale	14
1.4.5	Sintesi finale: l'Equazione del Moto	16
2	Architettura Software	19
2.1	Unreal Engine come piattaforma di simulazione scientifica	19
2.1.1	Lo schema Actor-Component	19
2.1.2	Il motore fisico: BeginPlay e il ciclo di Tick	20
2.2	Programmazione Orientata agli Oggetti (OOP)	22
2.2.1	Gerarchia delle classi e diagramma UML	23
2.2.2	Protezione dei dati e interfaccia utente in Editor	24
2.3	Il gestore della simulazione (Simulation Manager)	25
2.3.1	Inizializzazione e controllo dello stato globale	25
2.3.2	Ispezione della scena e gestione degli input	26

2.4	Modellazione del veicolo spaziale (Solar Sail)	27
2.4.1	Rappresentazione geometrica e mesh poligonale	27
2.4.2	Configurazione del profilo di riflettanza	27
2.4.3	Sistema fisico ibrido e calcolo in doppia precisione	28
3	Implementazione Fisica	31
3.1	Dimensioni di riferimento	31
3.1.1	Il sistema di riferimento del simulatore	31
3.1.2	Scale astronomiche e necessità della doppia precisione	32
3.2	Inizializzazione delle condizioni di partenza	33
3.2.1	Collegamento al gestore globale	33
3.2.2	Parametrizzazione strutturale del veicolo	33
3.2.3	Collocamento geometrico e configurazione dell'orbita	34
3.2.4	Correzione dell'assetto e allineamento orbitale	35
3.2.5	Calcolo e attribuzione della velocità orbitale iniziale	36
3.2.6	Inizializzazione della telemetria e delle condizioni di partenza . .	38
3.3	Modellazione della gravità terrestre	38
3.3.1	Calcolo vettoriale dell'accelerazione di gravità	38
3.3.2	Applicazione della forza e aggiornamento dinamico	39
3.4	Il Campo di Pressione Solare (SRP)	40
3.4.1	Calcolo della distanza Vela-Sole	40
3.4.2	La Legge del Quadrato Inverso applicata ai fotoni	41
3.4.3	Ottimizzazione del calcolo della pressione solare	42
3.4.4	Alterazione controllata delle forze con <i>Force Multiplier</i>	42
3.5	Lo schema ottico della Membrana	42
3.5.1	Scomposizione dell'interazione fotone-superficie	42
3.5.2	Parametrizzazione dei Coefficienti Ottici	42
3.5.3	Analisi dei Preset di Riflettività	43
3.5.4	Geometria dell'illuminazione e angolo di incidenza	44
3.6	Il Calcolo della spinta a raggi (Ray Casting)	44
3.6.1	Logica di campionamento della superficie	45
3.6.2	Generazione e orientamento dei raggi di controllo	45
3.6.3	Ottimizzazioni prestazionali con <i>RaycastTimer</i>	46
3.7	Gestione delle ombre e degli ostacoli	47
3.7.1	Preparazione del raggio e auto-collisione	47
3.7.2	L'esecuzione del raggio (<i>Line Trace</i>)	47
3.7.3	Conversione spaziale e debug visivo	48
3.7.4	Il filtro della luce e il conteggio dei fotoni	48
3.8	Integrazione Vettoriale e Forza Risultante	49
3.8.1	Calcolo della forza propulsiva per singola cella	49
3.8.2	Integrazione numerica della spinta totale	51
3.8.3	Applicazione al motore fisico e controlli di sicurezza	52
4	Dinamica Discreta e Time Warp	55
4.1	Il ciclo di simulazione e il limite del calcolo in tempo reale	55
4.1.1	Criticità del componente Rigid Body in fase di Time Warp . . .	55
4.2	Lo Switch Architetturale	56

INDICE

4.3	Integrazione numerica della traiettoria tramite il Metodo di Eulero . . .	57
4.4	Aggiornamento indipendente dell'assetto	58
5	Telemetria e Strumenti di Diagnostica	61
5.1	Telemetria Live e Debug Visivo	61
5.1.1	Rappresentazione Vettoriale e Scia Orbitale	61
5.1.2	Debug Ottico del Ray Casting	63
5.1.3	L'Interfaccia Telemetrica a Schermo (HUD)	65
5.2	Il Sistema di Logging Persistente (CSV)	66
5.2.1	Ottimizzazione I/O e gestione del Buffer temporale	67
5.3	Post-Processing e Visualizzazione (Script Python)	68
5.3.1	Automazione e Librerie Scientifiche	68

—

Capitolo 1

Fondamenti Teorici

La simulazione accurata del volo spaziale a vela solare richiede una modellazione rigorosa dell'ambiente fisico in cui il veicolo opera. Prima di procedere con l'implementazione degli algoritmi informatici all'interno del motore grafico, è indispensabile definire il *background* matematico che descrive i limiti dei propulsori tradizionali, le leggi della meccanica orbitale e i modelli vettoriali che intervengono nella spinta fotonica. L'obiettivo di questo capitolo è fornire tutte le conoscenze che costituiranno il nucleo del simulatore numerico in Unreal Engine.

1.1 I limiti fisici e termodinamici della propulsione chimica

Per comprendere la necessità ingegneristica di sviluppare tecnologie a vela solare per lo spazio profondo, è opportuno analizzare matematicamente i problemi della propulsione spaziale tradizionale. Il limite insuperabile dei razzi chimici risiede nella dipendenza diretta tra la massa del propellente trasportato e la variazione di velocità ottenibile dal veicolo, un problema che la meccanica classica modella con estrema rigidità.

1.1.1 Conservazione della quantità di moto ed equazione del razzo

Il principio di funzionamento di un qualsiasi propulsore chimico si basa sulla terza legge della dinamica di Newton e sul principio di conservazione della quantità di moto. In un sistema chiuso, in assenza di forze esterne, la variazione totale della quantità di moto deve essere nulla.

Considerando un veicolo spaziale che espelle massa (propellente) a una velocità di scarico v_e , la quantità di moto totale del sistema (veicolo + propellente) deve rimanere costante. Se m è la massa del veicolo in un dato istante e dv è la variazione infinitesimale della sua velocità, mentre dm è la variazione infinitesimale della massa (negativa, poiché il propellente viene espulso), si può scrivere:

$$m \cdot dv = -v_e \cdot dm \tag{1.1}$$

Riorganizzando i termini per separare le variabili, si ottiene l'equazione differenziale fondamentale del moto a propulsione chimica:

$$dv = -v_e \frac{dm}{m} \quad (1.2)$$

Integrando questa equazione tra lo stato iniziale (velocità v_0 e massa m_0) e lo stato finale (velocità v_f e massa m_f), supponendo che la velocità di espulsione dei gas v_e rimanga costante per tutta l'accensione, si ottiene:

$$\int_{v_0}^{v_f} dv = -v_e \int_{m_0}^{m_f} \frac{1}{m} dm \quad (1.3)$$

Il risultato di questa integrazione è la celebre equazione del razzo di Tsiolkovsky, formulata alla fine del XIX secolo dal fisico russo Konstantin Tsiolkovsky, che quantifica la variazione di velocità totale Δv ottenibile dal razzo in funzione della massa iniziale, finale e della velocità di scarico:

$$\Delta v = v_e \ln \left(\frac{m_0}{m_f} \right) = I_{sp} \cdot g_0 \cdot \ln \left(\frac{m_0}{m_f} \right) \quad (1.4)$$

dove i parametri fondamentali sono così definiti:

- v_e è la velocità effettiva di scarico dei gas.
- $I_{sp} = v_e/g_0$ è l'impulso specifico del propulsore, misurato in secondi, che rappresenta l'efficienza con cui la massa propellente viene convertita in spinta.
- g_0 è l'accelerazione di gravità standard al livello del mare (9.80665 m/s^2).
- m_0 è la massa iniziale al lancio, data dalla somma della massa a vuoto (struttura e *payload*) e dalla massa del propellente.
- m_f è la massa finale, ovvero la sola massa a vuoto una volta esaurito il propellente.

1.1.2 Il rapporto di massa e la "tirannia" dell'equazione

L'equazione 1.4 mette in luce un problema fondamentale per l'ingegneria aerospaziale. Invertendo la formula per isolare il rapporto di massa (definito come $\Lambda = m_0/m_f$), si ottiene:

$$\Lambda = \frac{m_0}{m_f} = e^{\frac{\Delta v}{I_{sp} \cdot g_0}} \quad (1.5)$$

Questa relazione esponenziale è spesso definita come "tirannia dell'equazione del razzo". Essa dimostra che la massa iniziale richiesta cresce in modo esponenziale rispetto alla variazione di velocità desiderata [2].

Attraverso un esempio pratico è possibile chiarire questo concetto. Se si desidera raddoppiare il Δv di una missione mantenendo costante il carico utile e il tipo di motore, non è sufficiente raddoppiare il propellente. Sarà necessario trasportare una quantità di carburante esponenzialmente maggiore, poiché il propulsore dovrà spendere energia per accelerare anche il carburante stesso che non è stato ancora espulso. Per le

missioni interplanetarie dirette verso destinazioni distanti dal sistema solare, i requisiti di Δv rendono il rapporto Λ talmente elevato da lasciare una frazione di massa per il carico scientifico (*payload*) inferiore all'1% / 2% della massa totale al momento del lancio [4].

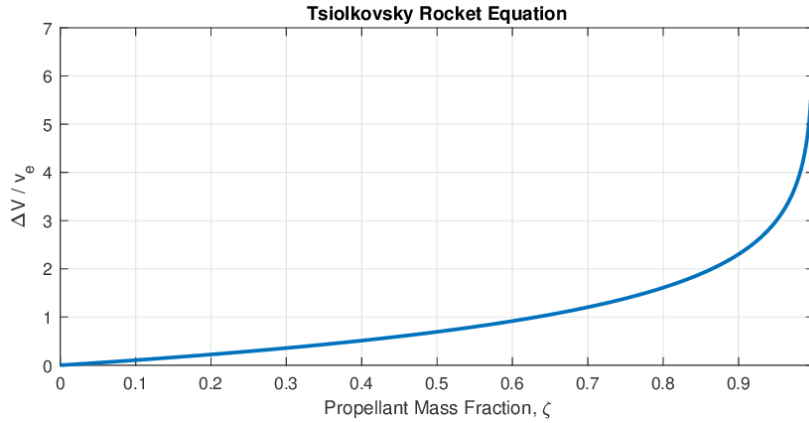


Figura 1.1: Rappresentazione grafica della "tirannia dell'equazione del razzo": la massa iniziale necessaria cresce in modo esponenziale all'aumentare della variazione di velocità richiesta (Δv).

1.1.3 Il vincolo termodinamico insuperabile

Di fronte alla crescita esponenziale della massa, l'unica soluzione matematica per ridurre il carico di propellente è aumentare l'impulso specifico I_{sp} , o la velocità di scarico v_e . Tuttavia, nei motori chimici questo parametro non può essere aumentato arbitrariamente.

La velocità di espulsione dei gas è strettamente vincolata dal calore massimo che la reazione chimica riesce a sprigionare e dalla leggerezza dei prodotti che generano la spinta. Pur ricorrendo alla miscela propulsiva più efficiente a disposizione dell'ingegneria aerospaziale (idrogeno e ossigeno allo stato liquido, LOX/LH2), il limite termodinamico teorico dell'impulso specifico si scontra con una soglia invalicabile di circa 450-460 secondi. Semplicemente, nessuna reazione chimica conosciuta possiede un'intensità energetica sufficiente per superare questa barriera naturale.

Con dei limiti prestazionali così severi, l'esplorazione umana nello spazio profondo incontra una barriera fisica invalicabile [3]. È in questo panorama di difficoltà termodinamiche che si inserisce la necessità di sviluppare propulsori di natura non-chimica. Se un veicolo potesse attingere la propria quantità di moto dall'esterno, eliminando del tutto la massa di propellente ($m_0 \approx m_f$), l'equazione di Tsiolkovsky decadrebbe, offrendo un impulso specifico potenzialmente infinito. Questa è l'esatta premessa teorica su cui si fonda la propulsione a vela solare.

1.2 Fisica della radiazione elettromagnetica e pressione solare

L'esplorazione sostenibile dello spazio profondo, quindi, richiede un cambio di strategia propulsiva che svincoli il veicolo dall'equazione di Tsiolkovsky. La propulsione a vela solare risponde a questa esigenza attingendo a una fonte di energia esterna e inesauribile: il Sole. Tuttavia, per comprendere matematicamente in che modo un raggio di luce possa generare una spinta meccanica tangibile su un oggetto macroscopico, è necessario esplorare la natura quantistica della radiazione elettromagnetica e definire le grandezze fisiche che governano l'ambiente interplanetario.

1.2.1 Il dualismo onda-particella e la quantità di moto del fotone

Dal punto di vista della fisica classica, la luce è descritta dalle equazioni dell'elettromagnetismo formulate da James Clerk Maxwell nel 1873. Secondo questo modello, la radiazione solare è un'onda elettromagnetica che si propaga nello spazio e che, incidendo su una superficie, esercita una pressione derivante dalla forza di Lorentz applicata alle cariche elettriche del materiale investito.

Sebbene il modello ondulatorio confermi l'esistenza della pressione di radiazione, la sua classificazione moderna si appoggia in modo più completo alla meccanica quantistica, in particolar modo riguardo alla propulsione. Nel 1905, Albert Einstein, spiegando l'effetto fotoelettrico, teorizzò che la luce non fosse unicamente un'onda continua, ma fosse quantizzata in pacchetti di energia, successivamente chiamati "fotoni".

Questa scoperta diede origine al concetto di *dualismo onda-particella*, successivamente formalizzato e analizzato da Louis de Broglie. Un fotone è una particella elementare priva di massa ($m_0 = 0$). Tuttavia, secondo l'equazione dell'energia relativistica di Einstein:

$$E^2 = (pc)^2 + (m_0c^2)^2 \quad (1.6)$$

- E è l'energia totale della particella
- p è la quantità di moto
- c è la velocità della luce nel vuoto (299 792 458 m/s)

Ponendo la massa a riposo m_0 pari a zero, l'equazione si riduce a un'uguaglianza fondamentale per l'astrodinamica delle vele solari:

$$E = p \cdot c \implies p = \frac{E}{c} \quad (1.7)$$

Questa relazione quantistica elementare è il cuore della propulsione fotonica [2]. Essa dimostra che ogni singolo fotone che colpisce la membrana di una vela solare trasporta una quantità di moto infinitesimale. Quando il fotone viene assorbito o riflesso dalla superficie, la sua quantità di moto subisce una variazione (Δp). Per il principio di conservazione della quantità di moto, questa variazione deve essere trasferita alla vela sotto forma di impulso meccanico, generando una forza continua nel tempo.

1.2.2 Costante solare e pressione in orbita terrestre

Un singolo fotone possiede un'energia estremamente piccola. Affinché il trasferimento di quantità di moto diventi sufficiente per muovere un veicolo spaziale di svariati chilogrammi, è necessario calcolare l'effetto del flusso luminoso emesso da una stella.

Il Sole irradia energia in tutte le direzioni e la quantità di energia solare che attraversa perpendicolarmente una superficie unitaria in un secondo è definita come *irradianza*. Alla distanza media tra la Terra e il Sole, definita come Unità Astronomica (1 AU $\approx 149.6 \times 10^6$ km), questo flusso prende il nome di **Costante Solare** (W_E).

I dati moderni stabiliscono che il valore medio della costante solare all'esterno dell'atmosfera terrestre è:

$$W_E \approx 1361 \text{ W/m}^2 \quad \left(\text{oppure } \frac{\text{J}}{\text{s} \cdot \text{m}^2} \right) \quad (1.8)$$

Sfruttando la relazione di Einstein ricavata in precedenza (Equazione 1.7), è possibile convertire questo flusso di energia in una pressione meccanica. Dividendo l'irradianza per la velocità della luce, si ottiene la pressione nominale della radiazione solare (P_E) misurata a un'Unità Astronomica di distanza su una superficie teorica perfettamente assorbente:

$$P_E = \frac{W_E}{c} = \frac{1361}{299\,792\,458} \approx 4.56 \times 10^{-6} \text{ N/m}^2 \quad (1.9)$$

Questo valore estremamente piccolo (pari a circa 4.56 micronewton per metro quadrato) evidenzia il primo problema della propulsione a vela solare: la forza generata da un singolo metro quadrato di membrana è infinitesimale. Per ottenere una spinta significativa, è necessario dispiegare vele di dimensioni enormi (centinaia o migliaia di metri quadrati) e operare in un ambiente privo di attriti aerodinamici, come lo spazio interplanetario [1].

1.2.3 Decadimento spaziale e legge dell'inverso del quadrato

La costante P_E calcolata è valida esclusivamente nelle vicinanze dell'orbita terrestre (orbite LEO, MEO, GEO). Tuttavia, un simulatore astrodinamico progettato per missioni interplanetarie deve modellare dinamicamente la variazione della spinta in funzione della posizione del veicolo nel sistema solare.

L'energia emessa dal Sole si propaga radialmente e, poiché l'area di una sfera cresce proporzionalmente al quadrato del suo raggio ($A = 4\pi r^2$), la densità dei fotoni (e di conseguenza la pressione di radiazione) subisce una diminuzione quadratica.

Definendo r_E come la distanza di riferimento (1 AU) e r come l'effettiva distanza tra il Sole e la vela solare nell'istante di tempo considerato, l'intensità della pressione locale $P(r)$ è governata dalla legge dell'inverso del quadrato:

$$P(r) = P_E \left(\frac{r_E}{r} \right)^2 \quad (1.10)$$

Questa equazione impone un vincolo molto preciso, che grazie al simulatore software sarà possibile verificare in tempo reale: la spinta di una vela solare non è costante, ma decresce rapidamente man mano che il veicolo si allontana dal Sole. Avvicinandosi al

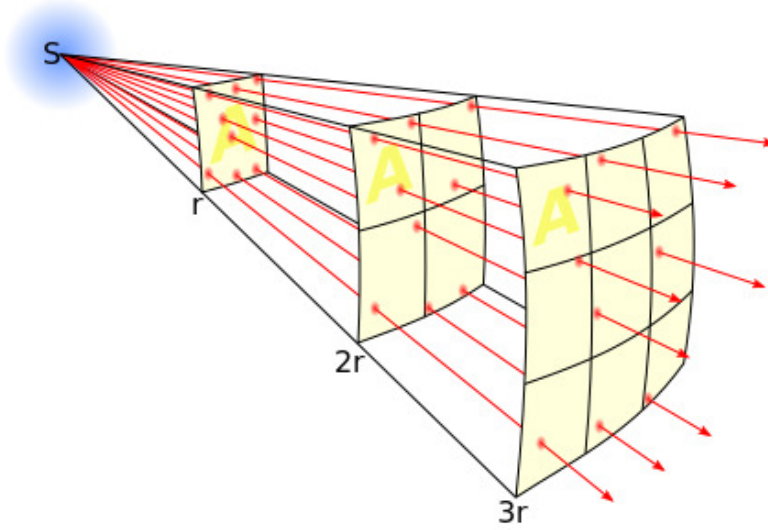


Figura 1.2: Propagazione sferica della radiazione elettromagnetica: la densità dei fotoni e la relativa pressione diminuiscono con il quadrato della distanza dalla sorgente solare.

Sole (ad esempio verso Venere o Mercurio, come avvenuto per la sonda IKAROS), il denominatore r diminuisce, causando un incremento quadratico della spinta propulsiva e del carico termico. Al contrario, dirigendosi verso i pianeti esterni (Giove, Saturno), la pressione solare crolla drasticamente [4]. Ad esempio, a 5 AU (l'orbita di Giove), la spinta fotonica si riduce a solo $1/25$ del valore misurato sulla Terra, rendendo le manovre a vela solare estremamente lente e complesse nello spazio profondo [1].

1.2.4 Luminosità solare

Sebbene l'utilizzo della Costante Solare (P_E) sia comune per le missioni riferite unicamente all'orbita terrestre, lo sviluppo di un simulatore astrodinamico complesso richiede un approccio più generalizzato. Ecco perché un approccio più universale consiste nel calcolare la pressione di radiazione a partire dalla potenza totale emessa dal Sole, ovvero la sua luminosità (L), e adattare dinamicamente il calcolo in base alla posizione del veicolo.

Difatti il Sole si comporta come un'enorme lampada sferica che irradia energia in tutte le direzioni. La potenza totale emessa dal Sole è una costante fisica, espressa in W che può essere calcolata a partire dalla Costante Solare e dall'area della sfera di raggio 1 AU:

$$L \approx 3.828 \times 10^{26} \text{ W} \quad (1.11)$$

Poiché questa immensa energia si propaga nello spazio vuoto distribuendosi sulla superficie di una sfera in continua espansione (la cui area è $4\pi r^2$), il flusso energetico locale a una qualsiasi distanza r dal centro del Sole è dato da:

$$W(r) = \frac{L}{4\pi r^2} \left[\frac{\text{W}}{\text{m}^2} \right] \quad (1.12)$$

Figura 1.3: Rappresentazione geometrica della Legge dell'Inverso del Quadrato: la Potenza totale emessa dalla stella (Luminosità L) si diluisce sulla superficie di sfere concentriche in espansione, riducendo l'intensità del flusso locale in proporzione a $1/r^2$.

Dividendo questo flusso per la velocità della luce c , si ottiene l'equazione universale della pressione fotonica locale con cui è possibile calcolare la forza istantanea che agisce su una vela solare in qualsiasi punto del sistema solare:

$$P(r) = \frac{L}{4\pi r^2 c} \quad (1.13)$$

Questa formulazione è più flessibile e adatta a un simulatore astrodinamico, poiché consente di modellare con precisione la spinta fotonica in qualsiasi punto del sistema solare, tenendo conto della posizione dinamica del veicolo rispetto al Sole. È proprio questa equazione che è stata implementata nel simulatore sviluppato in Unreal Engine, permettendo il calcolo istantaneo della pressione di radiazione locale e dell'accelerazione risultante durante l'intero arco della missione.

1.3 Modellazione ottica vettoriale e interazione fotonica

Avendo definito la pressione di radiazione solare locale $P(r)$, è possibile calcolare la forza totale che agisce su una vela solare. Tuttavia, questa forza non è semplicemente il prodotto scalare tra $P(r)$ e l'area geometrica della vela. La spinta risultante dipende da una complessa interazione tra i fotoni incidenti e la superficie della membrana, che coinvolge fenomeni di riflessione, assorbimento e diffusione.

1.3.1 Area effettiva e angolo di incidenza

Il primo parametro fondamentale per calcolare l'interazione fotonica è la determinazione della parte della vela effettivamente esposta al flusso di fotoni. La forza generata dalla pressione solare non dipende dall'area totale della vela, ma dall'area proiettata perpendicolarmente alla direzione dei raggi solari.

Per prima cosa è necessario definire un sistema di riferimento solidale alla vela solare in cui $\mathbf{n}$ rappresenta il versore normale (perpendicolare) alla superficie della vela. Inoltre, è utile dichiarare $\mathbf{u}$ come il versore direzione dell'illuminazione, ovvero il vettore che punta dal centro di massa del veicolo verso il Sole.

L'angolo compreso tra il versore normale $\mathbf{n}$ e la direzione dei raggi solari $\mathbf{u}$ prende il nome di *angolo di incidenza* (α). La superficie della vela non sempre è completamente esposta al flusso solare. L'area effettiva che riceve i fotoni dipende dall'orientamento della vela rispetto al Sole ed è calcolata così:

$$A_{\text{eff}} = A \cos(\alpha) \quad (1.14)$$

Questa relazione geometrica implica che la vela cattura il massimo della quantità di moto quando è orientata frontalmente al Sole ($\alpha = 0^\circ$, dove $\cos(0) = 1$). Man mano

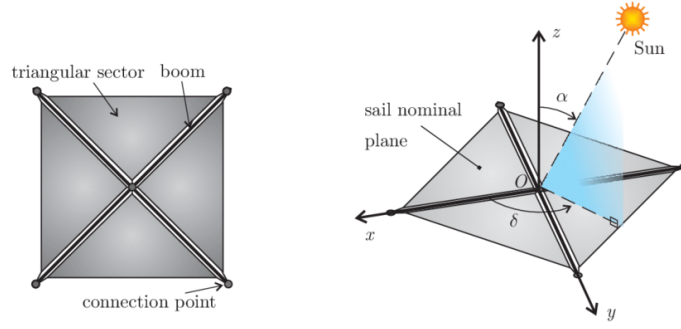


Figura 1.4: Schema vettoriale dell'interazione fotonica: il versore normale $\mathbf{n}$, la direzione dei raggi solari $\mathbf{u}$ e l'angolo di incidenza α che determina l'area effettivamente esposta.

che la vela viene inclinata, l'area esposta diminuisce, fino ad annullarsi completamente quando la vela è posta "di taglio" rispetto ai raggi solari ($\alpha = 90^\circ$, dove $\cos(90) = 0$).

1.3.2 Fenomeni di ombreggiamento e auto-occlusione

Il modello matematico finora descritto assume che la vela solare sia una superficie piana e perfettamente esposta ai raggi solari. Nella realtà operativa, tuttavia, la sonda è una struttura tridimensionale complessa. Elementi come il modulo centrale (*bus*), i bracci telescopici per il dispiegamento (*booms*) e la strumentazione di bordo sono posti tra il Sole e la vela riflettente, proiettando delle ombre.

Questo fenomeno, noto come **auto-occlusione**, azzerla la spinta fotonica nelle zone oscurate. Le conseguenze sulla dinamica del veicolo sono due:

- Una riduzione netta della spinta propulsiva totale.
- Uno spostamento asimmetrico del centro di pressione; questo può generare un momento torcente che induce rotazioni non pianificate del veicolo, complicando le manovre di controllo d'assetto.

Oltre a ciò, la vela può presentare deformazioni strutturali, pieghe o irregolarità che causano ulteriori ombre.

Per calcolare con precisione l'area effettivamente illuminata, l'equazione algebrica $A_{\text{eff}} = A \cos(\alpha)$ non basta. Il calcolo deve essere impostato come un integrale di superficie che includa una funzione di visibilità $v(x, y)$, la quale restituisce il valore 1 se il punto è direttamente illuminato e il valore 0 se è in ombra:

$$A_{\text{eff}} = \int_A v(x, y) \cos(\alpha) dA \quad (1.15)$$

Purtroppo questo calcolo risulta essere impossibile da risolvere analiticamente, poiché la funzione di visibilità $v(x, y)$ dipende dalla geometria complessa del veicolo e dalla posizione relativa del Sole. Questo ostacolo giustifica la necessità di utilizzare un motore grafico: algoritmi di calcolo spaziale discretizzato, come il *Ray Casting*, permettono di campionare dinamicamente la funzione di visibilità $v(x, y)$ sulla geometria della vela, risolvendo il problema delle ombre in tempo reale.

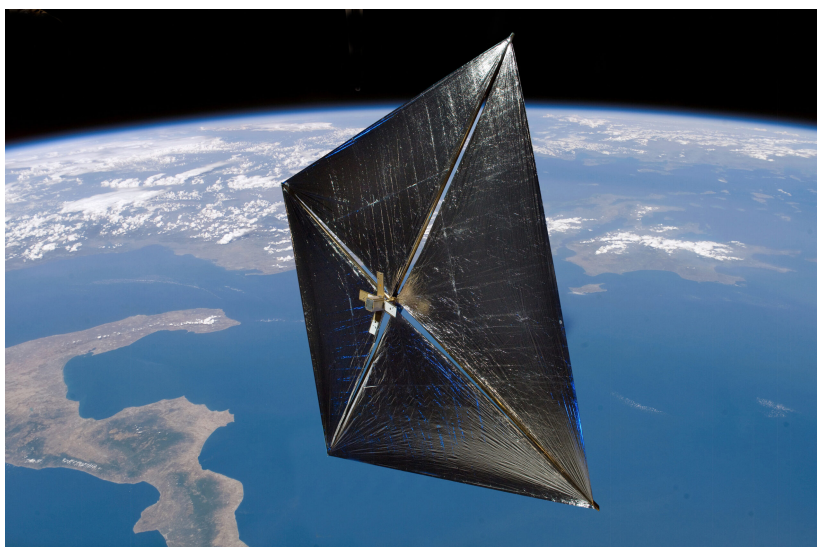


Figura 1.5: Rappresentazione concettuale dell'auto-occlusione: i componenti strutturali del veicolo proiettano ombre sulla membrana riflettiva, rendendo necessario il calcolo locale dell'area effettivamente illuminata.

1.3.3 Occlusione ambientale e asimmetria di spinta

Oltre all'auto-occlusione causata dai componenti specifici della vela solare, un simulatore astrodinamico ad alta fedeltà deve modellare accuratamente l'**occlusione ambientale**. Questo fenomeno si verifica quando un corpo celeste (ad esempio un pianeta, una luna o un asteroide) si interpone tra il Sole e la vela, causando un'occlusione totale o parziale.

Se un'eclissi totale comporta semplicemente l'azzeramento globale della spinta fotonica, le fasi di transizione o di occlusione parziale introducono problemi molto più complessi. Quando un corpo oscura solo una porzione della vela solare, si instaura una forte asimmetria nella distribuzione delle forze.

Dal punto di vista della meccanica dei corpi rigidi, la differenza di pressione causata dalla radiazione solare, tra la zona in ombra e la superficie illuminata, dà luogo ad una discordanza tra i due centri di pressione fotonica e di massa del veicolo, generando un momento torcente che provoca la rotazione del veicolo.

Il simulatore sviluppato in questo elaborato è capace di analizzare questa complessa casistica. In particolar modo, grazie alla modellazione tridimensionale della vela solare e alla simulazione dinamica del sistema solare, è possibile calcolare in tempo reale l'effetto di un'eclissi parziale, prevedendo con precisione la forza risultante e il momento torcente che agisce sul veicolo durante le fasi di transizione.

Figura 1.6: Effetto dell'occlusione ambientale parziale: l'asimmetria di illuminazione sposta il centro di pressione rispetto al centro di massa, generando un momento torcente (*torque*) che induce il rotolamento del veicolo.

L'effetto di questo momento torcente è di fondamentale importanza per la dinamica del veicolo, poiché può indurre un "rotolamento" incontrollato che a sua volta può compromettere l'orientamento dei pannelli solari e interrompere i collegamenti telemetrici

con la Terra. Inoltre costringe il sistema di controllo d'assetto a consumare preziose risorse propulsive per contrastare queste rotazioni, riducendo ulteriormente l'efficienza complessiva della missione [1].

1.3.4 Il modello ottico ideale (Riflessione speculare perfetta)

Per comprendere l'origine delle componenti di forza, è utile partire da un modello ideale in cui la superficie della vela solare è perfettamente liscia e riflettente, senza alcuna perdita di energia. In questo scenario ideale, si assume che il 100% dei fotoni incidenti venga riflesso in modo perfettamente elastico (riflessione speculare), senza alcuna perdita di energia per assorbimento o diffusione.

Quando un fotone con quantità di moto p colpisce la superficie con un angolo α , esso viene riflesso con il medesimo angolo rispetto alla normale. La variazione della quantità di moto del fotone lungo l'asse tangenziale alla vela è nulla, mentre lungo l'asse normale alla vela è pari a $2p \cos(\alpha)$.

Integrando questo scambio di quantità di moto sull'intera area esposta $A \cos(\alpha)$ e moltiplicandolo per il flusso di fotoni, si ottiene la forza della Pressione di Radiazione Solare nel caso ideale ($\mathbf{F}_{\text{ideale}}$). Poiché in un urto elastico la componente trasversale si annulla, l'intero vettore forza è diretto lungo la normale $\mathbf{n}$ della vela:

$$\mathbf{F}_{\text{ideale}} = 2P(r)A \cos^2(\alpha) \mathbf{n} \quad (1.16)$$

Per comprendere l'origine del termine $\cos^2(\alpha)$, è necessario analizzare in dettaglio l'interazione fotonica per ogni singolo fotone incidente. Considerando un fotone incidente che trasporta quantità di moto $p = E/c$.

Quando il fotone colpisce la superficie della vela con angolo di incidenza α (misurato rispetto alla normale $\mathbf{n}$), la sua quantità di moto può essere ripartita in due componenti perpendicolari tra loro:

- Componente normale: $p_n = p \cos(\alpha)$
- Componente tangenziale: $p_t = p \sin(\alpha)$

Nel caso di riflessione speculare perfetta, il fotone rimbalza elasticamente mantenendo l'angolo di incidenza. La componente tangenziale rimane invariata ($p'_t = p_t$), mentre la componente normale si inverte ($p'_n = -p_n$). La variazione totale di quantità di moto del fotone lungo la direzione normale è:

$$\Delta p_n = p'_n - p_n = -p \cos(\alpha) - p \cos(\alpha) = -2p \cos(\alpha) \quad (1.17)$$

Per il principio di conservazione della quantità di moto, la vela riceve un impulso diretto lungo la propria normale pari a:

$$\Delta p_{\text{vela}} = 2p \cos(\alpha) = 2 \frac{E}{c} \cos(\alpha) \quad (1.18)$$

Tuttavia, il flusso luminoso non interagisce con l'intera area geometrica A della membrana, bensì soltanto con la porzione effettivamente esposta alla radiazione, la quale si riduce all'aumentare dell'inclinazione del veicolo:

$$A_{\text{eff}} = A \cos(\alpha) \quad (1.19)$$

La forza totale esercitata sulla vela si ricava integrando la pressione locale $P(r)$ sull'area effettiva, tenendo conto del fattore di riflessione elastica lungo la normale. Moltiplicando questi termini si ottiene:

$$F = P(r) \cdot A_{\text{eff}} \cdot (2 \cos \alpha) = 2P(r)A \cos^2(\alpha) \quad (1.20)$$

Il termine quadratico $\cos^2(\alpha)$ è di fondamentale importanza nell'astrodinamica fotonica e ha una duplice origine fisica: un primo fattore $\cos(\alpha)$ deriva dalla riduzione puramente geometrica dell'area intercettata (A_{eff}), mentre il secondo $\cos(\alpha)$ deriva dalla proiezione dell'impulso trasferito lungo l'asse normale alla superficie.

Questa dipendenza quadratica dimostra che la spinta di una vela solare diminuisce molto rapidamente rispetto alla semplice inclinazione della vela: a 30° di incidenza, la forza propulsiva si riduce al 75% del suo valore ($\cos^2(30^\circ) \approx 0.75$), a 45° di inclinazione invece la forza scende al 50% [2].

1.3.5 Il modello ottico realistico (Superfici non ideali)

Il modello ideale descritto dall'Equazione 1.16 è un'ottima approssimazione per i calcoli preliminari, ma risulta inadeguato per un simulatore informatico *High-Fidelity*. Nella realtà, nessun materiale polimerico metallizzato (come il Kapton alluminato) possiede una riflettività del 100%.

L'interazione della luce con un materiale reale è un fenomeno complesso in cui i fotoni incidenti subiscono rimbalzi differenti. Per modellare matematicamente questo comportamento, la letteratura aerospaziale definisce tre coefficienti ottici, la cui somma conserva l'energia totale (deve essere pari a 1):

$$\rho_s + \rho_d + \alpha_a = 1 \quad (1.21)$$

Questi parametri descrivono le tre reazioni ottiche della superficie:

- **Riflessione speculare (ρ_s):** La parte di fotoni che rimbalza elasticamente mantenendo l'angolo di incidenza. Genera una forza diretta unicamente lungo la normale $\mathbf{n}$.
- **Riflessione diffusa (ρ_d):** La parte di fotoni che, a causa delle irregolarità del materiale, viene riflessa in direzioni casuali. Questa componente introduce una forza di taglio parallela alla superficie della vela, deviando la forza totale rispetto alla normale $\mathbf{n}$.
- **Assorbimento (α_a):** La frazione di fotoni che non viene riflessa, ma assorbita dal materiale. Questa componente non genera "rimbalzo", trasferendo la propria quantità di moto esclusivamente nella direzione di incidenza originale dei raggi solari ($\mathbf{u}$). L'energia assorbita si trasforma poi in calore termico.

La coesistenza di questi tre fenomeni fa sì che la forza risultante su una vela reale non sia mai perfettamente allineata al versore normale $\mathbf{n}$ (tranne quando $\alpha = 0^\circ$).

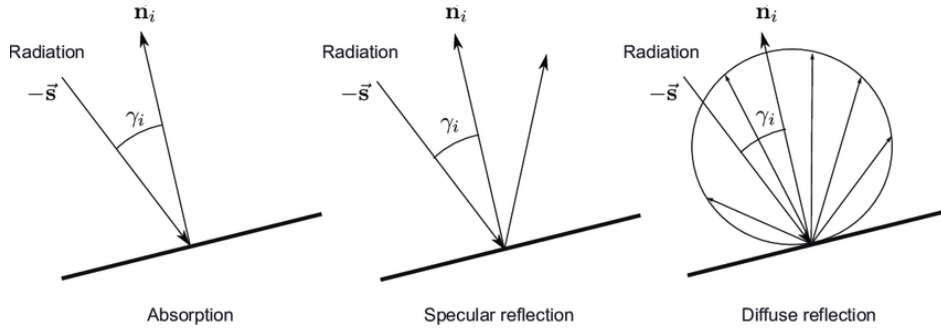


Figura 1.7: Confronto visivo tra i fenomeni ottici su una superficie non ideale: riflessione speculare (ρ_s), riflessione diffusa (ρ_d) e assorbimento (α_a).

L'assorbimento e la diffusione introducono un'asimmetria che genera una forza di taglio (*shear force*) parallela alla superficie della vela, un elemento critico che può deviare la traiettoria orbitale.

1.3.6 L'equazione vettoriale dell'accelerazione SRP

Volendo quindi determinare l'effettiva spinta generata sul veicolo, i valori derivanti dai diversi fenomeni ottici descritti devono essere raggruppati in un'unica formula. Per la buona riuscita di questa operazione, si introduce il versore trasversale $\mathbf{t}$, definito come parallelo al piano della vela solare e complanare al piano formato dalla normale $\mathbf{n}$ e dalla direzione di incidenza dei raggi solari $\mathbf{u}$.

L'accelerazione totale generata dalla Pressione di Radiazione Solare ($\mathbf{a}_{\text{SRP}}$) su un veicolo di massa m e area A , inclinato di un angolo α lungo la direzione del versore $\mathbf{t}$, è espressa dalla seguente equazione vettoriale [7]:

$$\mathbf{a}_{\text{SRP}} = \frac{P(r)A}{m} \cos(\alpha) \left[\left(1 + \rho_s \cos(2\alpha) + \frac{2}{3} \rho_d \cos(\alpha) \right) \mathbf{n} + (\rho_s \sin(2\alpha) + \rho_d \sin(\alpha)) \mathbf{t} \right] \quad (1.22)$$

Questa formulazione mostra l'intera fisica propulsiva del modello non-ideale. Al suo interno è possibile distinguere chiaramente due valori ortogonali:

- La componente associata al versore $\mathbf{n}$ rappresenta la **spinta normale**, ovvero l'accelerazione generata lungo la direzione normale alla superficie della vela.
- La componente associata al versore $\mathbf{t}$ rappresenta la **forza di taglio** (*shear force*), elemento critico per la navigazione solare poiché in grado di generare spostamenti laterali della traiettoria orbitale.

Calcolare questa espressione vettoriale istante per istante, ricavando ogni volta le normali geometriche $\mathbf{n}$ e osservando come i raggi incidono sulla vela, rappresenta la prima sfida che il simulatore software in C++, sviluppata in Unreal Engine e presentata nel capitolo successivo, deve svolgere.

1.4 Parametri di design e sintesi della dinamica orbitale

Attraverso le equazioni esposte nei paragrafi precedenti, è possibile calcolare con precisione la forza di pressione solare che agisce su una vela in qualsiasi punto del sistema solare, tenendo conto dell'orientamento della vela e delle proprietà ottiche del materiale. Tuttavia, il reale comportamento cinematico del veicolo spaziale sottoposto alla perturbazione solare non dipende esclusivamente dall'efficienza ottica del materiale o dalla distanza dal Sole, ma è rigidamente vincolata dalla geometria strutturale e dalla distribuzione delle masse del veicolo. Ecco perché diventa utile definire alcuni parametri di design che sintetizzano le caratteristiche macroscopiche del veicolo e che permettono di valutare le prestazioni propulsive in modo più immediato, indipendentemente dall'orbita specifica o dalle condizioni operative.

1.4.1 Il Carico Alare (Sail Loading)

Il parametro di design più critico nella progettazione di un veicolo a propulsione fotonica è il Carico Alare (*Sail Loading*), solitamente indicato con la lettera greca σ . Questo è definito come il rapporto tra la massa totale del veicolo spaziale e l'area totale della membrana dispiegata per la "cattura" dei fotoni:

$$\sigma = \frac{m}{A} \quad \left[\frac{\text{kg}}{\text{m}^2} \right] \quad (1.23)$$

La massa m al numeratore comprende tutti i componenti del veicolo: la massa del carico utile (*payload* scientifico), la massa riguardante i sistemi satellitari (sistemi di comunicazione, computer di bordo, pannelli solari, giroscopi per la rotazione della vela), la massa dei bracci telescopici di dispiegamento (*booms*) e, infine, la massa della membrana stessa.

L'obiettivo primario durante il dimensionamento di una missione è minimizzare σ . Un valore di *Sail Loading* basso indica infatti che il veicolo è molto leggero rispetto alla superficie che espone alla radiazione solare, massimizzando così l'accelerazione generata dalla pressione di radiazione. Le vele solari di prima generazione (come IKAROS) presentavano valori di σ relativamente alti a causa dell'utilizzo di membrane più spesse e meccanismi di dispiegamento delle stesse più pesanti. Attraverso gli sviluppi moderni e futuri si può mirare a spingere questo valore al di sotto dei 10 g/m^2 , impiegando pellicole di spessore sub-micrometrico e strutture in fibra di carbonio, che sono in grado di resistere alle sollecitazioni meccaniche e termiche dello spazio profondo pur mantenendo una massa estremamente ridotta [1] [4].

1.4.2 Accelerazione Caratteristica

Un altro parametro di design fondamentale è l'Accelerazione Caratteristica (a_c). Esso rappresenta il valore massimo di accelerazione che una vela solare può generare quando è esposta alla luce solare nelle condizioni più favorevoli: posizionata a 1 Unità Astronomica dal Sole e orientata perpendicolarmente rispetto ai raggi solari ($\alpha = 0^\circ$).

Sostituendo l'equazione del *Sail Loading* (σ) all'interno del modello ottico realistico discusso in precedenza (Equazione 1.22), e considerando che per $\alpha = 0^\circ$ la componente trasversale si annulla, si ricava la formula analitica dell'Accelerazione Caratteristica:

$$a_c = \frac{P_E}{\sigma} \left(1 + \rho_s + \frac{2}{3}\rho_d \right) \quad \left[\frac{\text{m}}{\text{s}^2} \right] \quad (1.24)$$

Questo valore rappresenta un fondamentale parametro di riferimento per la validazione dell'ambiente virtuale. Al fine di certificare l'accuratezza del simulatore, durante le fasi di collaudo i vettori tridimensionali restituiti in tempo reale da Unreal Engine dovranno essere confrontati con il valore a_c teorico, a patto che la vela simulata venga esposta alle condizioni operative ideali.

1.4.3 Numero di Leggerezza

Un ulteriore parametro di design di grande interesse astrodinamico è il Numero di Leggerezza (*Lightness Number*, β). Esso esprime il rapporto tra la massima accelerazione repulsiva della pressione solare a_c e l'attrazione gravitazionale del Sole g_{sole} alla stessa distanza:

$$\beta = \frac{a_c}{g_{\text{sole}}} \quad (1.25)$$

Il Numero di Leggerezza è un valore che mostra rapidamente la capacità propulsiva di una vela solare rispetto alla forza gravitazionale che agisce su di essa. Un valore di $\beta < 1$ indica che la forza gravitazionale del Sole vince su quella fotonica, di conseguenza la vela non potrà mai "sfuggire" dal sistema solare e per acquisire velocità dovrà rimanere in orbita per molto. Al contrario, un valore di $\beta > 1$ indica che la spinta fotonica supera la gravità solare, permettendo al veicolo di "sfuggire" direttamente dal sistema solare seguendo una traiettoria iperbolica, senza necessità di lunghi periodi di accelerazione orbitale [2].

1.4.4 Il campo gravitazionale primario e la meccanica orbitale

Seppur fino ad ora si sia parlato esclusivamente delle forze riguardanti la dinamica fotonica, è importante evidenziare che la vela solare è comunque sempre soggetta alla forza gravitazionale di uno o più corpi celesti.

Il veicolo fotonico è infatti governato dalle leggi della meccanica orbitale, ed in particolar modo dalla legge di gravitazione universale formulata da Isaac Newton. Questa afferma che ogni corpo dotato di massa esercita un'attrazione gravitazionale su ogni altro corpo, con una forza $\mathbf{F}_g$ che è direttamente proporzionale al prodotto delle loro masse e inversamente proporzionale al quadrato della distanza che unisce i due centri di massa:

$$\mathbf{F}_g = -G \frac{M \cdot m}{r^2} \hat{\mathbf{r}} \quad (1.26)$$

dove:

- G è la Costante di Gravitazione Universale ($6.674 \times 10^{-11} \text{ m}^3 \text{kg}^{-1} \text{s}^{-2}$), fondamentale che quantifica l'intensità gravitazionale;

- M è la massa del corpo centrale, espressa in kg, che genera il campo gravitazionale;
- m è la massa del veicolo spaziale, che subisce l'attrazione gravitazionale generata dal corpo centrale;
- r è la distanza tra i centri di massa del corpo centrale e del veicolo spaziale, misurata in metri;
- $\hat{\mathbf{r}}$ è il versore che indica la direzione dal centro del pianeta verso il veicolo spaziale, con il segno negativo che indica che la forza è attrattiva (verso il corpo centrale).

Affinché il veicolo spaziale non venga attratto verso il corpo centrale, è necessario che la forza di gravità sia controbilanciata da una forza centrifuga generata dal moto orbitale del veicolo. Questa velocità orbitale, che dipende dalla distanza r e dalla massa del corpo centrale M , è quella necessaria per mantenere un'orbita stabile.

Tale velocità si ricava uguagliando la forza gravitazionale alla forza centrifuga (mv^2/r) e risolvendo per v , ottenendo così la formula della velocità orbitale circolare:

$$v = \sqrt{\frac{G \cdot M}{r}} \quad (1.27)$$

La moltiplicazione tra G e M è un prodotto così ricorrente nella meccanica orbitale che viene spesso accorpato in un unico parametro, noto come *parametro gravitazionale standard* (μ) definito come:

$$\mu = G \cdot M \quad (1.28)$$

È dunque possibile esprimere anche la forza gravitazionale in termini di accelerazione, dividendo $\mathbf{F}_g$ per la massa del veicolo m :

$$\mathbf{a}_g = \frac{\mathbf{F}_g}{m} = -G \frac{M}{r^2} \hat{\mathbf{r}} = -\frac{\mu}{r^2} \hat{\mathbf{r}} \quad (1.29)$$

Inoltre, tenendo conto che il versore $\hat{\mathbf{r}}$ è definito come $\mathbf{r}/r$, dove $\mathbf{r}$ è il vettore posizione del veicolo spaziale rispetto al corpo centrale è possibile esprimere tale accelerazione in forma vettoriale. Infine quindi, l'equazione vettoriale dell'accelerazione gravitazionale centrale che agisce su un veicolo spaziale in orbita attorno a un corpo centrale è:

$$\mathbf{a}_g(\mathbf{r}) = -\frac{\mu}{r^3} \mathbf{r} \quad (1.30)$$

Figura 1.8: Diagramma vettoriale dell'accelerazione gravitazionale centrale. Il vettore posizione $\mathbf{r}$ è diretto dal fuoco all'oggetto; il vettore accelerazione $\mathbf{a}_g$ è sempre diretto in verso opposto (verso il corpo attrattore), giustificando il segno negativo nell'equazione vettoriale.

1.4.5 Sintesi finale: l'Equazione del Moto

Ora che sono state definite sia gravità ($\mathbf{a}_g$) che spinta fotonica ($\mathbf{a}_{\text{SRP}}$), è possibile unirle per ottenere l'equazione finale che descrive il movimento della vela solare.

Per mantenere il calcolo gestibile, si ricorre solitamente a un'approssimazione pratica: vengono ignorate le forze secondarie marginali, come l'attrito dell'alta atmosfera, l'attrazione della Luna o la forma non perfettamente sferica della Terra. Fatto ciò, l'accelerazione totale della navicella è semplicemente la somma di chi la tira verso il basso (il pianeta) e di chi la spinge via (il Sole):

$$\ddot{\mathbf{r}} = \mathbf{a}_g(\mathbf{r}) + \mathbf{a}_{\text{SRP}}(\mathbf{r}, \alpha, \rho_s, \rho_d) \quad (1.31)$$

Espandendo questa formula e inserendo tutte le equazioni che sono state dimostrate fino ad ora, compreso l'uso della Luminosità solare (L) per calcolare la pressione, è possibile ottenere l'equazione completa del sistema:

$$\frac{d^2\mathbf{r}}{dt^2} = -\frac{\mu}{r^3}\mathbf{r} + \frac{L \cdot A}{4\pi r^2 c \cdot m} \cos(\alpha) \left[\left(1 + \rho_s \cos(2\alpha) + \frac{2}{3}\rho_d \cos(\alpha) \right) \mathbf{n} + (\rho_s \sin(2\alpha) + \rho_d \sin(\alpha)) \mathbf{t} \right] \quad (1.32)$$

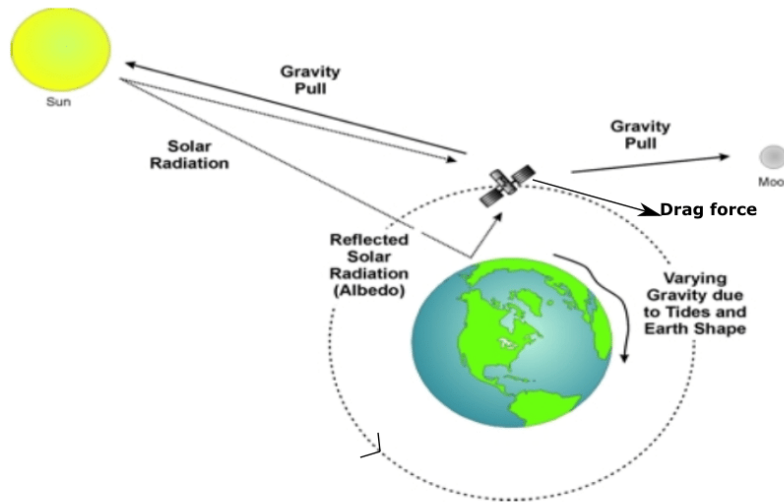


Figura 1.9: Diagramma delle forze in gioco: l'accelerazione totale della vela solare è data dalla somma vettoriale tra la gravità ($\mathbf{a}_g$) e la perturbazione fotonica ($\mathbf{a}_{\text{SRP}}$).

Come è facile intuire, questa equazione è estremamente complessa. Tale complessità rende notevolmente difficile la possibilità di prevedere con precisione la traiettoria del veicolo spaziale, poiché ogni variabile è in continua evoluzione durante il viaggio. Man mano che il veicolo viaggia, la distanza (r) varia, modificando contemporaneamente sia l'attrazione gravitazionale che l'intensità della luce. Inoltre, ogni minima rotazione della vela cambia gli angoli di incidenza e i vettori di direzione ($\mathbf{n}$ e $\mathbf{t}$).

A causa di queste continue variazioni, è matematicamente impossibile risolvere questa equazione "a mano" con carta e penna per prevedere dove si troverà la vela tra un mese. L'unico modo per tracciare la rotta è usare un approccio virtuale: far calcolare al

computer questa enorme formula ogni istante. Il prossimo capitolo spiegherà proprio come la programmazione a oggetti e il motore grafico Unreal Engine 5 siano stati impiegati per compiere questo complesso lavoro.

Capitolo 2

Architettura Software

L'implementazione delle meccaniche fisiche viste nel capitolo precedente in un ecosistema virtuale interattivo richiede una struttura software solida e ben progettata. Questo capitolo mira ad analizzare l'architettura software del progetto, sviluppato tramite il celebre motore grafico Unreal Engine 5.

2.1 Unreal Engine come piattaforma di simulazione scientifica

Anche se Unreal Engine è stato originariamente sviluppato per la creazione di prodotti videoludici, la potenza di calcolo di Unreal Engine è ideale per simulazioni in ambito scientifico. Il simulatore sviluppato infatti ne sfrutta il *rendering* in tempo reale e l'efficiente allocazione delle risorse per condurre simulazioni astrodinamiche complesse, offrendo anche un'interfaccia visiva molto più immersiva e interattiva rispetto alle soluzioni tradizionali [6].

Grazie alla grande efficienza garantita dal linguaggio C++, il motore grafico fornisce un'infrastruttura robusta per l'esecuzione di calcoli vettoriali complessi e l'aggiornamento continuo delle variabili fisiche. Per comprendere la transizione delle leggi fisiche della vela solare in questo ambiente virtuale, è necessario prima esaminare due concetti fondamentali di Unreal Engine: lo schema *Actor-Component* e la gestione ciclica del tempo.

2.1.1 Lo schema Actor-Component

L'architettura di Unreal Engine favorisce in modo marcato l'uso della composizione. Tale strategia si concretizza nel modello noto come *Actor-Component*.

L'elemento fondamentale che "esiste" nel mondo tridimensionale è l'Attore (derivato dalla classe propria di Unreal Engine `AActor`) [8]. Un Attore può essere progettato inizialmente come un contenitore vuoto: esso possiede proprietà che lo identificano nel mondo virtuale (posizione, rotazione e scala), ma non possiede alcuna logica complessa o di espressione grafica. I comportamenti e l'estetica vengono aggiunti in modo modulare attribuendogli una o più componenti (derivati dalla classe `UActorComponent`) [8].

Per costruire il veicolo principale della simulazione (la classe `ASolarSail`), sono stati assemblati i seguenti componenti:

- **Modello 3D e Root Component:** Un `UStaticMeshComponent` viene impiegato per inserire la vela nell'ambiente virtuale ed è impostato come Componente "Radice" (*Root Component*) per l'intero Attore: questo indica che tutte le modifiche di posizione o rotazione dell'Attore si riflettono direttamente anche sul *Root Component* stesso. Oltre a renderlo visibile, ottiene la funzione di *collider*: è la superficie contro cui "sbattono" i raggi virtuali (*ray casting*), permettendo al sistema di calcolare con esattezza l'area illuminata e le zone in ombra.
- **Assi di orientamento spaziale:** Per semplificare la complessa matematica vettoriale, il codice sfrutta i riferimenti direzionali nativi dell'Attore (spesso visualizzati in editor tramite `UArrowComponent`). Attraverso funzioni specifiche (come `GetActorUpVector()`), il programma estrae in modo immediato l'asse perpendicolare della superficie (il vettore normale $\mathbf{n}$). Questo dato è indispensabile per calcolare l'inclinazione della vela rispetto ai raggi solari (l'angolo α) e stabilire la direzione della spinta.
- **Vettori a doppia precisione:** Per gestire le enormi distanze spaziali, la vela solare non può affidarsi alle variabili a singola precisione (`FVector` composto da `float`) fornite nativamente da Unreal Engine [8]. Per le sue operazioni di fisica infatti, la classe `ASolarSail` sfrutta una struttura dati basata su variabili a doppia precisione (`FVector3d`). Questo consente di mantenere l'accuratezza dei calcoli anche a scale astronomiche, evitando errori di arrotondamento che potrebbero compromettere la stabilità della simulazione.
- **Sistema di telemetria e tracciamento:** La classe include anche un sistema di registrazione dei dati telemetrici. Ad ogni ciclo, raccoglie i dati di volo vitali (come le forze applicate, l'efficienza dei fotoni e la distanza dalla Terra) per stamparli a schermo e, contemporaneamente, salvarli in modo persistente su file *CSV* per l'analisi post-simulazione.

Tenere separata la matematica dell'orbita dal rendering 3D (applicando il noto principio della *Separation of Concerns*) rende il codice molto più pulito e scattante. In questo modo, l'Attore principale opera come centro logico: si concentra solo sulle formule fisiche e calcola come far muovere la vela. Parallelamente, i calcoli geometrici più pesanti per la CPU, come tracciare i raggi di luce per trovare le zone in ombra (*ray casting*), vengono delegati ai singoli componenti. In questo modo le prestazioni del motore grafico vengono sfruttate in modo ottimale.

2.1.2 Il motore fisico: `BeginPlay` e il ciclo di Tick

Il secondo elemento fondamentale per il calcolo orbitale è la gestione del tempo. In Unreal Engine, il ciclo di vita di un oggetto virtuale è governato da ben note funzioni native in C++ che è comunque possibile sovrascrivere tramite *override*. Per implementare la dinamica del veicolo spaziale, il codice si appoggia interamente su due metodi chiave: `BeginPlay()` e `Tick()` [8].

La fase di avvio del veicolo è interamente affidata al metodo `BeginPlay()`. Questo metodo viene eseguito una sola volta, quando l'oggetto viene creato nel mondo virtuale. Il suo compito è "preparare il terreno": alloca le risorse in memoria, imposta le

Actor-Component Paradigm in Unreal Engine

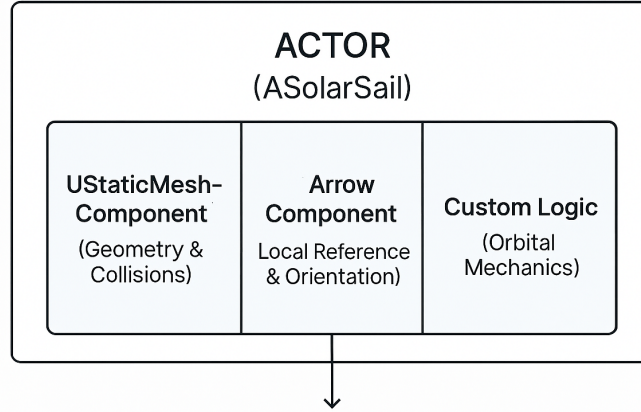


Figura 2.1: Architettura del paradigma *Actor-Component*: l'Attore funge da contenitore unificatore per le logiche visive, fisiche e di collisione.

costanti strutturali (come la massa e l'area) posiziona il modello 3D alla giusta distanza orbitale (es. LEO o GEO), allinea l'orientamento e calcola la velocità tangenziale iniziale necessaria per immetterlo in un'orbita stabile. Infine, prepara i file *CSV* per la registrazione della telemetria.

Conclusa questa fase di avvio, l'esecuzione entra in un ciclo continuo gestito dalla funzione `Tick(float DeltaTime)`. Questo ciclo permette al software sviluppato di rielaborare l'intera fisica del veicolo ad ogni singolo *frame* renderizzato sullo schermo. Il parametro in ingresso `DeltaTime` (Δt) indica i millisecondi o secondi trascorsi esattamente dal fotogramma precedente.

Ad ogni iterazione del `Tick`, l'oggetto derivato dalla classe Attore `ASolarSail` esegue i seguenti passaggi:

1. **Gestione del Tempo:** Verifica il moltiplicatore del tempo (`TimeScale`). Per garantire stabilità numerica a velocità estreme, il codice passa in automatico dal motore fisico nativo di Unreal (per `TimeScale` ≤ 1.1) a un motore di calcolo cinematico manuale indipendente (per accelerazioni temporali più elevate).
2. **Aggiornamento dell'Assetto:** Ruota dinamicamente la vela nello spazio tridimensionale (`UpdateSailRotation`), assicurandosi che il suo asse principale segua fedelmente la direzione del vettore velocità.
3. **Calcolo delle Forze:** Controlla il `SimulationManager` per ricalcolare l'attrazione gravitazionale ($\mathbf{F}_g$) e proietta la griglia di raggi virtuali (*ray casting*) per misurare l'area effettivamente esposta alla luce, ricavando la spinta fotonica ($\mathbf{F}_{\text{SRP}}$). I due vettori vengono sommati per ottenere la Forza Totale risultante.
4. **Integrazione del Moto (Metodo di Eulero):** Nel motore manuale, il codice traduce le forze in movimento vero e proprio. Prima calcola l'accelerazione

($\mathbf{a} = \mathbf{F}/m$). Poi applica le regole base della cinematica usando un tempo "accelerato" (ovvero il `DeltaTime` moltiplicato per il `TimeScale`): partendo da questo intervallo, aggiorna prima la velocità attuale e la usa per spostare materialmente le coordinate 3D della navicella lungo la sua orbita.

5. **Output e Telemetria:** Conclude il ciclo immagazzinando le metriche di volo nel buffer per il salvataggio su file CSV e aggiornando l'interfaccia di *debug* a schermo.

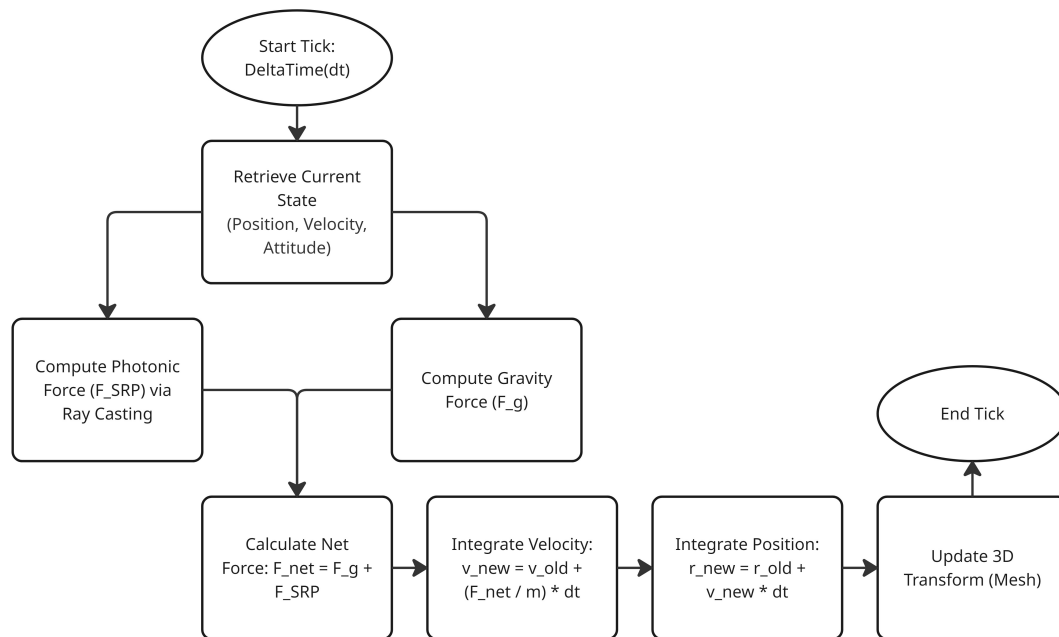


Figura 2.2: Diagramma di flusso dell'integrazione numerica eseguita a ogni ciclo di *Tick*.

Legare i calcoli fisici al normale ciclo di *rendering* attribuisce al simulatore una reattività immediata. L'ingegnere può modificare l'inclinazione della vela a *runtime* e osservare istantaneamente la variazione di spinta e di traiettoria, senza alcun bisogno di interrompere l'esecuzione o ricalcolare il percorso orbitale.

2.2 Programmazione Orientata agli Oggetti (OOP)

Un buon simulatore non si basa solo su formule matematiche corrette, ma richiede anche un codice ben strutturato. Per gestire in modo fluido i calcoli complessi e lo scorrere del tempo, il sistema è stato costruito usando la Programmazione Orientata agli Oggetti (OOP).

Adottare questo schema permette di suddividere il programma in blocchi logici distinti, evitando che le varie sezioni del codice si intreccino in modo rigido tra loro. Dividere il codice in componenti indipendenti offre due vantaggi pratici fondamentali:

- **Manutenzione e Debugging:** Mantenere le componenti isolate semplifica enormemente la ricerca e la correzione degli errori. Se, ad esempio, si riscontrano errori nel calcolo della gravità, è sufficiente intervenire sul singolo blocco responsabile di quella formula, senza dover analizzare o rischiare di compromettere il resto dell'applicativo.
- **Scalabilità futura:** Un sistema modulare rende il simulatore facilmente espandibile. Se in futuro si volessero aggiungere nuove perturbazioni fisiche basterà aggiungere e collegare una nuova componente di calcolo senza riscrivere o modificare le classi esistenti, lasciando invariata l'integrità del codice già testato e funzionante.

2.2.1 Gerarchia delle classi e diagramma UML

Il progetto si basa sull'estensione delle classi native di Unreal Engine tramite le classiche macro (`UCLASS()`, `GENERATED_BODY()`) [8]. Come mostrato nel diagramma UML (Figura 2.3), l'architettura poggia sulla classe base `AActor` e si divide in due blocchi logici principali:

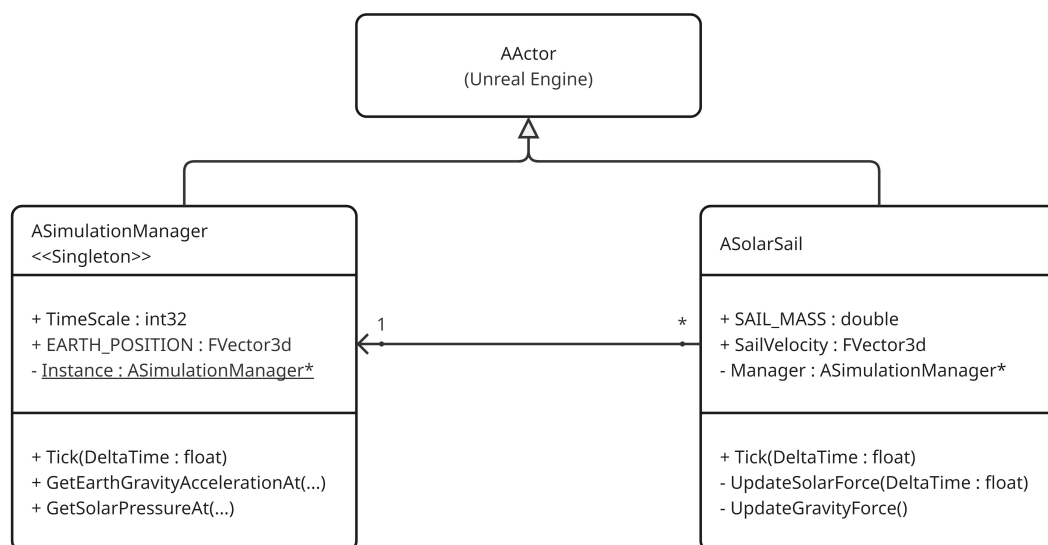


Figura 2.3: Diagramma UML semplificato che mostra l'ereditarietà dalla classe `AActor` di Unreal Engine e le relazioni tra le classi principali del simulatore.

- **ASimulationManager:** È il "regista" della simulazione. Sviluppato seguendo il *Singleton pattern* (un'unica istanza globale), si occupa di gestire tutto l'ambiente spaziale: si occupa della gestione del tempo manipolando il `TimeScale`, calcola le costanti di gravità e pressione solare nei vari punti dello spazio, e prepara il sistema di log. Avere un coordinatore centrale evita che ogni singola vela debba ricalcolarsi la posizione della Terra o del Sole a ogni frame, risparmiando risorse di calcolo.

- **ASolarSail**: Rappresenta il veicolo fotonico vero e proprio. Sfrutta il suo modello 3D per intercettare i raggi solari (*ray casting*) e ricalcola le spinte a ogni ciclo, ma soprattutto mantiene i parametri unici e vitali della singola sonda: posizione orbitale, vettore velocità, massa, area esposta e coefficienti ottici.

Avendo diviso in maniera netta l'ambiente globale (**ASimulationManager**) dall'attore fisico (**ASolarSail**) si ottiene un enorme vantaggio pratico: in questo modo infatti si rende la simulazione nativamente "multi-vela". È possibile inserire nella stessa scena decine di sonde diverse per confrontarne simultaneamente le prestazioni (ad esempio variando l'area o la riflettività), avendo la certezza che tutte quante stiano operando sotto le stesse condizioni ambientali imposte dal **ASimulationManager**.

2.2.2 Protezione dei dati e interfaccia utente in Editor

Proteggere i parametri del simulatore è fondamentale per evitare calcoli impossibili o crash di sistema. Se l'utente potesse inserire valori casuali (come un'area della vela negativa o una massa pari a zero), il motore fisico andrebbe in errore, violando le leggi della fisica e interrompendo l'esecuzione.

Nel contesto di Unreal Engine, il vero metodo per proteggere le variabili esposte all'utente non avviene solo tramite i classici modificatori C++ (**private** o **protected**), ma attraverso l'uso strategico della macro **UPROPERTY** [8]. Questo costrutto permette di far comunicare il codice sorgente con l'interfaccia grafica dell'Editor, impostando dei vincoli numerici di sicurezza direttamente a livello di codice:

Listing 2.1: Esempio reale tratto da **SolarSail.h**: l'uso dei metadati **ClampMin** e **ClampMax** per vincolare matematicamente i parametri della vela nell'interfaccia dell'editor.

```

1 UPROPERTY(EditAnywhere,
2     Category = "Sail Parameters | Sail",
3     meta = (DisplayName = "Area della Vela (m^2)",
4     ClampMin = "9.3",
5     ClampMax = "1672.0"))
6 double SAIL_AREA = 100.0;
7
8 UPROPERTY(EditAnywhere,
9     Category = "Sail Parameters | Sail",
10    meta = (DisplayName = "Massa della Vela (Kg)",
11    ClampMin = "3.2",
12    ClampMax = "307"))
13 double SAIL_MASS = 90.0;

```

L'inclusione dei metadati restrittivi **ClampMin** e **ClampMax** agisce come scudo: impedisce, ad esempio, l'inserimento di una massa strutturale nulla (0.0), che causerebbe una divisione per zero fatale nel momento in cui il codice calcola l'accelerazione ($\mathbf{a} = \mathbf{F}/m$).

Inoltre, per garantire un codice pulito e isolato, le componenti che calcolano la gravità newtoniana (**UpdateGravityForce**) e la spinta fotonica (**UpdateSolarForce**) operano in modo totalmente autonomo. Le due funzioni calcolano i rispettivi vettori senza mai interferire tra loro, per poi confluire solo nell'ultimo passaggio e formare il vettore della Forza Totale netta.

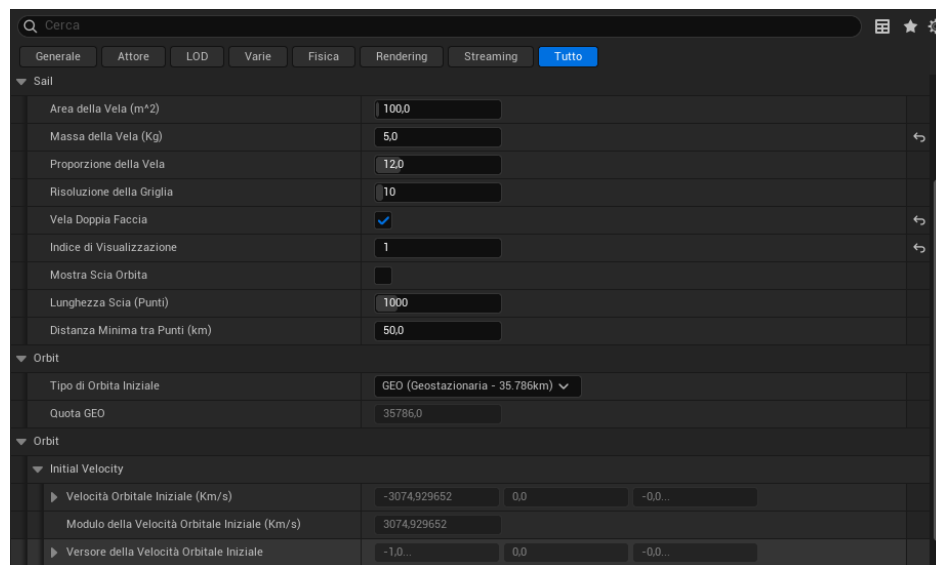


Figura 2.4: L'esposizione sicura delle variabili di stato tramite **UPROPERTY**. L'interfaccia generata automaticamente dall'editor rispetta i vincoli imposti dal codice C++, impedendo l'inserimento di valori fisicamente invalidi.

2.3 Il gestore della simulazione (Simulation Manager)

In un simulatore con più elementi in gioco, è fondamentale avere un "registra" che si faccia carico delle regole dell'ambiente, evitando che ogni singolo veicolo debba ricalcolare continuamente gli stessi dati globali condivisi, come la posizione del Sole o l'accelerazione del tempo. Questo è l'esatto compito della classe **ASimulationManager**. Estendendo la classe base **AActor**, questo modulo centrale funziona come gestore di sistema: detta lo scorrere del tempo per tutti e assicura che ogni elemento sulla scena (che sia la Terra, il Sole o un'intera flotta di vele solari) sia perfettamente sincronizzato all'interno dello stesso spazio virtuale tridimensionale.

2.3.1 Inizializzazione e controllo dello stato globale

Tutto il lavoro del gestore inizia non appena viene fatta partire la simulazione (più precisamente quando viene richiamata la funzione **BeginPlay()**). In questa fase, il Manager si occupa di preparare l'ambiente spaziale: posiziona la Terra e il Sole alle corrette distanze e orienta la sorgente luminosa. Grazie all'implementazione del *Singleton pattern*, il Manager non ha bisogno di cercare attivamente i veicoli nella scena; al contrario, sono le singole istanze di **ASolarSail** che, al momento dello *spawn*, si collegano autonomamente al gestore centrale. Questa architettura decentralizzata garantisce un'elevata modularità e permette alle vele di leggere in tempo reale due parametri chiave condivisi:

- **Direzione del Sole:** Invece di far cercare il Sole a ogni singola vela, il Manager interroga una volta sola la luce del livello (**Directional Light**) per capire da dove arrivano i raggi (**u**) [8]. Questo dato viene poi letto da tutte le vele, assicurando che la spinta solare sia calcolata con la stessa angolazione per tutti i veicoli in volo, senza sprecare potenza di calcolo.

È importante sottolineare che, sebbene Unreal Engine gestisca nativamente l'illuminazione per scopi grafici, esso non prevede calcoli per la pressione di radiazione solare. Per questo motivo, il sistema utilizza la *Directional Light* di Unreal esclusivamente come riferimento geometrico per ottenere la direzione dei raggi, lasciando poi al codice C++ personalizzato il compito di calcolare l'impatto fisico effettivo sulla vela

- **Gestione del tempo (TimeScale):** Il Manager è responsabile anche della gestione del tempo per tutta la durata della simulazione tramite la variabile `TimeScale`. Poiché muoversi nello spazio in tempo reale sarebbe lento e poco utile ai fini di calcolare traiettorie complete, il gestore fornisce alle vele il moltiplicatore necessario per definire la velocità della progressione fisica. Questo permette di simulare mesi di navigazione in pochi minuti, scalando il tempo matematico (`DeltaTime`) senza rovinare la fluidità visiva del programma.

Infine, il `ASimulationManager` si occupa anche della gestione dei dati persistenti. Esso infatti prepara e pulisce i file CSV per la telemetria. Questo approccio centralizzato evita conflitti nelle operazioni di scrittura e garantisce la raccolta di tutte le informazioni di volo all'interno di un unico archivio strutturato.

2.3.2 Ispezione della scena e gestione degli input

Per l'analisi visiva della simulazione, il software si affida al sistema di navigazione libera nativo di Unreal Engine. Non essendo stata impostata una telecamera fissa o vincolata alla struttura della vela, l'operatore può muoversi nello spazio virtuale con totale libertà, in modo analogo a quanto avviene all'interno del *viewport* dell'editor.

Questa scelta garantisce flessibilità nell'ispezione dei vari componenti del sistema:

- **Navigazione libera:** L'assenza di un vincolo rigido tra la visuale e il veicolo permette di esaminare la vela da qualsiasi angolazione e distanza. Tale libertà è fondamentale per osservare il comportamento dei raggi luminosi durante le fasi di *ray casting* o per monitorare la posizione della sonda rispetto ai riferimenti planetari.
- **Verifica delle routine di debug:** la libertà di movimento permette di ispezionare da ogni angolazione i vettori di forza e i punti di impatto generati dal sistema di *ray casting*. Tale osservazione risulta fondamentale per confermare visivamente che le normali della superficie e la direzione dei fotoni siano calcolate correttamente rispetto alla posizione della sorgente luminosa.
- **Gestione delle scale astronomiche:** data la vastità delle distanze coinvolte, una visuale svincolata permette di passare rapidamente da un'analisi ravvicinata della struttura della vela a una prospettiva globale del sistema. Ciò facilita il monitoraggio della traiettoria e della posizione relativa del veicolo rispetto ai riferimenti planetari.
- **Monitoraggio multi-veicolo:** l'assenza di una telecamera fissa consente di spostarsi agevolmente tra diverse istanze di `ASolarSail` eventualmente presenti nella

scena. In questo modo è possibile confrontare in tempo reale il comportamento di vele con diverse caratteristiche fisiche (come massa o riflettività) operanti sotto le medesime condizioni ambientali.

Questa distinzione tra l'osservatore e il veicolo risponde a una logica di pulizia del software: permette di analizzare l'intero scenario di test con la massima libertà, lasciando che il codice della classe `ASolarSail` si occupi esclusivamente dell'esecuzione dei calcoli fisici senza l'aggiunta di componenti non necessari.

2.4 Modellazione del veicolo spaziale (Solar Sail)

Mentre il `ASimulationManager` gestisce l'ambiente e il tempo, la classe `ASolarSail` costituisce il nucleo fisico della simulazione. Questa entità agisce come il centro logico e fisico del veicolo: utilizza il modello geometrico per calcolare l'impatto dei raggi solari e conserva tutti i dati essenziali della vela, come la velocità, la posizione e la massa.

2.4.1 Rappresentazione geometrica e mesh poligonale

Nel simulatore, la vela solare non viene considerata come un semplice punto matematico, ma come un oggetto fisico con una superficie estesa. Per ottenere questa rappresentazione, la classe `ASolarSail` utilizza un componente `UStaticMeshComponent`, il quale ospita il modello tridimensionale importato da un software CAD.

L'uso di un modello 3D a poligoni è essenziale per calcolare correttamente la pressione solare. La superficie della vela è infatti suddivisa in migliaia di piccoli triangoli, una scelta costruttiva che offre due vantaggi tecnici immediati:

- **Estrazione dei vettori normali:** Il motore grafico ricava in automatico il vettore normale (ovvero la direzione perpendicolare) di ogni singolo triangolo colpito dalla luce. Questo dato è fondamentale per stabilire l'angolo di incidenza dei raggi solari e, di conseguenza, calcolare in quale direzione esatta verrà spinta la sonda.
- **Risoluzione delle intersezioni:** Quando i raggi solari intersecano i poligoni, il sistema di collisione genera un pacchetto di dati (`FHitResult`) che contiene le coordinate precise dell'impatto [8]. Questo meccanismo permette all'algoritmo di capire quali zone della vela siano effettivamente illuminate e quali siano in ombra, evitando di calcolare spinte fotoniche sulle parti non direttamente esposte al Sole.

2.4.2 Configurazione del profilo di riflettenza

Un aspetto importante nella modellazione del veicolo riguarda il comportamento ottico delle due facce della vela. Nelle missioni reali, le sonde solari sono asimmetriche: presentano un lato frontale altamente riflettente e un retro, del tutto incapace di generare spinta fotonica.

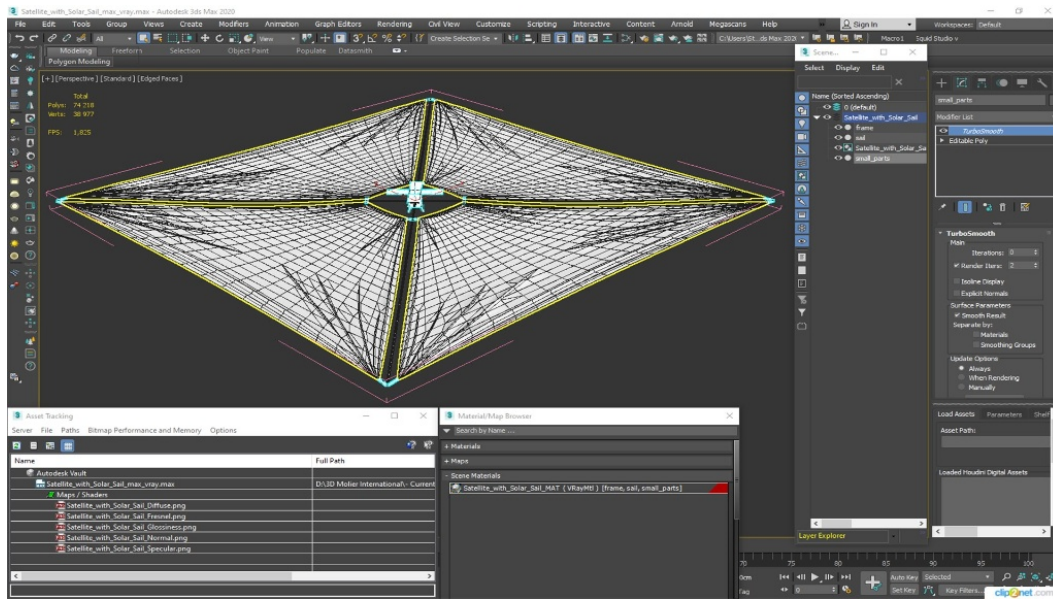


Figura 2.5: Visualizzazione *wireframe* (a fil di ferro) della membrana solare simulata. La suddivisione in triangoli permette di calcolare la spinta fotonica in modo dettagliato su tutta la superficie.

Il simulatore replica questa dinamica offrendo un parametro per la gestione della doppia faccia (*Two-Sided*), modificabile direttamente dall'interfaccia. Questa opzione permette di testare due scenari distinti:

- **Modalità realistica:** il sistema distingue logicamente il fronte dal retro calcolando il prodotto scalare tra la normale della vela e la direzione della luce. Se la matematica indica che i fotoni stanno colpendo la faccia posteriore, l'algoritmo azzerava in automatico la spinta.
- **Modalità simmetrica:** la vela viene trattata come un corpo capace di generare propulsione su entrambi i lati. Pur essendo fisicamente meno realistica, questa opzione è utile per testare manovre teoriche o materiali sperimentali senza rischiare la perdita improvvisa di spinta.

Questa flessibilità permette di osservare direttamente come un errato orientamento del veicolo, con la conseguente esposizione del lato sbagliato al Sole, comporti l'immediato azzeramento della propulsione e modifichi drasticamente la traiettoria orbitale.

2.4.3 Sistema fisico ibrido e calcolo in doppia precisione

Per assicurare che il veicolo rispetti le leggi fisiche indipendentemente dall'accelerazione temporale impostata, la classe `ASolarSail` adotta un sistema di calcolo ibrido.

I motori fisici come *Chaos Physics* di Unreal Engine sono ottimizzati per il tempo reale, ma operando con numeri a singola precisione (`float`) possono generare problemi di stabilità se sottoposti a forti accelerazioni temporali su scale interplanetarie [8]. Per

ovviare al problema, il codice valuta ad ogni ciclo di `Tick` il moltiplicatore del tempo (`TimeScale`):

- **Modalità nativa ($\text{TimeScale} \leq 1.1$):** a velocità normale, il simulatore si affida al motore fisico di Unreal, applicando vettori di forza continui alla mesh per sfruttarne le ottimizzazioni predefinite.
- **Modalità cinematica manuale ($\text{TimeScale} > 1.1$):** quando la progressione temporale viene accelerata, il software disattiva automaticamente la simulazione fisica nativa. Il calcolo passa a un solutore matematico personalizzato che aggiorna la posizione dell'attore (`SetActorLocation`) aggirando i limiti del motore grafico.

Questo solutore manuale si basa su un'architettura dati a doppia precisione (64 bit), obbligatoria per evitare errori di arrotondamento. Il sistema gestisce l'area e la massa della vela (parametri strutturali), i coefficienti ottici (per modulare l'efficienza riflettente della membrana) e tre vettori fondamentali: posizione ($\mathbf{r}$), velocità ($\mathbf{v}$) e accelerazione ($\mathbf{a}$).

Ad ogni ciclo di calcolo, il solutore somma le forze in gioco (fotonica e gravitazionale) per ricavare l'accelerazione netta del veicolo; successivamente, integra questo dato nel tempo per calcolare la nuova velocità istantanea e, infine, aggiornare la posizione esatta della sonda nello spazio tridimensionale.

Capitolo 3

Implementazione Fisica

Dopo aver descritto nel capitolo precedente l'architettura del software, in questa sezione viene analizzata la logica fisica che permette al simulatore di funzionare. Se il capitolo precedente si è occupato della "struttura" del programma, questo si concentra su come le leggi fisiche vengano tradotte in algoritmi veri e propri. L'obiettivo principale è trasformare i modelli teorici presentati nel Capitolo 2 in istruzioni che il computer possa eseguire in tempo reale.

In questo contesto, Unreal Engine non viene utilizzato solo come motore grafico, ma come un vero e proprio solutore numerico. Il sistema deve infatti calcolare ogni forza agente sulla vela solare, dalla gravità terrestre alla spinta dei fotoni solari, per ottenere la forza totale che ne determina il movimento. Nelle sezioni che seguono verrà descritto l'intero schema di calcolo: dalla gestione delle unità di misura e delle condizioni iniziali, fino all'uso del *ray casting* per simulare l'impatto della luce sulla superficie della vela.

3.1 Dimensioni di riferimento

Sviluppare un simulatore spaziale all'interno di Unreal Engine 5 richiede di superare un ostacolo tecnico importante: adattare un motore nato per i videogiochi a calcoli scientifici di alta precisione. Nativamente, infatti, la struttura di Unreal è pensata per gestire distanze e movimenti su scala "umana", dove tutto si misura in metri e centimetri.

Nello spazio profondo, al contrario, le distanze sono enormi. Il movimento di una sonda spaziale non può essere calcolato come quello di un normale personaggio 3D: a distanze astronomiche, gli errori di arrotondamento del computer si accumulerebbero a ogni singolo fotogramma (*Tick*), compromettendo l'orbita in pochissimo tempo e portando all'alterazione della simulazione o al suo diretto fallimento. Per risolvere questo problema, il simulatore esegue i calcoli fisici "dietro le quinte", in cui le unità di misura vengono ricalibrate appositamente per le distanze interplanetarie.

3.1.1 Il sistema di riferimento del simulatore

In astrodinamica, le traiettorie vengono sempre calcolate prendendo come ancoraggio il corpo celeste principale attorno a cui si orbita. Poiché il simulatore è progettato per ancorare le vele solari alla Terra, l'architettura software adotta nativamente un sistema

di riferimento geocentrico. Per tradurre questo concetto all'interno di Unreal Engine, il centro di massa della Terra viene fatto coincidere con l'origine tridimensionale del livello virtuale. Come definito all'interno della classe `ASimulationManager`, la variabile `EARTH_POSITION` è impostata alle coordinate (0.0, 0.0, 0.0):

Listing 3.1: Definizione delle coordinate centrali in `SimulationManager.h`.

```
1 UPROPERTY(VisibleAnywhere,
2     Category = "Simulation Parameters | Bodies | Earth")
3 FVector3d EARTH_POSITION = FVector3d(0.0, 0.0, 0.0);
4
5 UPROPERTY(VisibleAnywhere,
6     Category = "Simulation Parameters | Bodies | Sun")
7 FVector3d SUN_POSITION = FVector3d(149597870.7, 0.0, 0.0);
```

Questa configurazione crea un sistema di riferimento inerziale geocentrico. Il Sole viene invece posizionato lungo l'asse X alla sua distanza astronomica reale (circa 149 milioni di chilometri).

È importante precisare che posizionare la Terra esattamente al centro dell'universo virtuale è una scelta dettata da pura comodità logica e visiva. Il codice non è affatto vincolato in modo rigido all'origine del mondo: poiché l'intera architettura matematica è basata sul calcolo di vettori relativi (come la distanza tra la vela e `EARTH_POSITION`), se al variare delle necessità di progetto si decidesse di spostare la Terra in un altro punto dello spazio, basterebbe aggiornarne le coordinate. Di conseguenza, tutti gli altri corpi coinvolti slitterebbero in modo coerente, mantenendo intatte le proporzioni e le leggi fisiche della simulazione.

3.1.2 Scale astronomiche e necessità della doppia precisione

Un motore grafico ragiona nativamente tramite unità di misura generiche definite "Unreal Units" (UU). Mantenere questa metrica anche per lo spazio profondo significherebbe costringere il calcolatore a gestire valori estremamente grandi, portando in breve tempo a errori di arrotondamento e alla perdita di stabilità dell'ambiente virtuale.

Per ovviare a questo ostacolo, all'interno del `SimulationManager` è stata adottata una specifica convenzione di scala spaziale. Il sistema ricalibra le grandezze stabilendo che un chilometro di distanza nello spazio simulato corrisponda esattamente a 1000 unità del motore grafico:

Listing 3.2: Fattori di scala per le distanze astronomiche in `SimulationManager.h`.

```
1 UPROPERTY(VisibleAnywhere,
2     Category = "Simulation Parameters | Unit Conversions")
3 float KM_TO_UU = 1000.0f;
4
5 UPROPERTY(VisibleAnywhere,
6     Category = "Simulation Parameters | Unit Conversions")
7 float UU_TO_KM = 0.001f;
```

Attraverso questa proporzione, posizionare un satellite in orbita LEO a 400 km di quota si traduce in uno spostamento di 400.000 UU dal centro della terra, un valore numerico che il sistema è in grado di gestire in modo ottimale.

Tuttavia, la sola gestione della scala spaziale non è sufficiente a garantire la stabilità orbitale. I vettori a singola precisione (`float` a 32 bit) nativi del motore non sono in grado di identificare accuratamente variazioni millimetriche quando un corpo si trova a distanze di scala astronomica. Per far in modo di evitare eventuali deviazioni di traiettoria o errori di arrotondamento, l'intero sistema matematico del simulatore è basato su `FVector3d`. L'implementazione della doppia precisione (`double` a 64 bit) per le variabili di posizione rappresenta il requisito fondamentale e obbligatorio per operare in modo affidabile su distanze così grandi.

3.2 Inizializzazione delle condizioni di partenza

Affinché il solutore numerico possa calcolare correttamente l'evoluzione della traiettoria, è necessario definire con precisione lo stato fisico iniziale del veicolo al tempo zero ($t = 0$). Questa fase preparatoria viene gestita dalla classe `ASolarSail` attraverso una sequenza di funzioni di inizializzazione che si avviano automaticamente all'avvio della simulazione, all'interno dell'evento `BeginPlay()`.

3.2.1 Collegamento al gestore globale

La prima operazione è la funzione `InitializeManager()`. Prima ancora di leggere la propria massa o posizione, la vela deve necessariamente agganciarsi all'istanza *Singleton* del `SimulationManager`; senza questo collegamento, la vela non avrebbe accesso alle costanti universali (come la massa della Terra o il fattore di scala `KM_TO_UU`) e qualsiasi calcolo successivo genererebbe un errore. Una volta stabilito questo ponte comunicativo, è possibile procedere con il caricamento dei parametri fisici.

3.2.2 Parametrizzazione strutturale del veicolo

Prima di poter elaborare qualsiasi forza attiva, il sistema deve caricare in memoria le proprietà fisiche e geometriche della vela. Questa operazione è delegata alla funzione `InitializePhysicsProperties()`. I parametri fondamentali estratti dal pannello di controllo dell'utente sono due:

- **Massa del veicolo (m):** Valore espresso in chilogrammi, essenziale per derivare l'accelerazione a partire dalla forza totale applicata, secondo il secondo principio della dinamica ($a = F/m$).
- **Area della vela (A):** Valore espresso in metri quadrati, che determina l'ampiezza della superficie esposta al flusso di fotoni. Maggiore è l'area, superiore sarà la spinta fotonica raccolta dal sistema.

Questi parametri, prelevati dall'interfaccia utente, vengono applicati direttamente al componente `UStaticMeshComponent` (`SAIL_MESH`), il quale rappresenta fisicamente la vela. In questo modo, l'algoritmo sovrascrive le impostazioni di default del motore per imporre i valori esatti richiesti dal modello matematico desiderato.

3.2.3 Collocamento geometrico e configurazione dell'orbita

Una volta definita la struttura della vela, l'algoritmo deve posizionarla nello spazio. Questa operazione non si limita ad assegnare un semplice raggio orbitale predefinito, ma richiede di collocare il veicolo in un punto esatto lungo un'orbita tridimensionale. La funzione `SetInitialPositions()` gestisce questa necessità unendo i parametri numerici della missione con il posizionamento spaziale stabilito visivamente nell'editor.

Per agevolare la configurazione rapida, il sistema usa un `enum` (`EOrbitStartType`) contenente i preset delle altitudini orbitali più comuni (come l'orbita LEO a 400 km o quella geostazionaria).

Listing 3.3: Enumerazione dei preset orbitali nel file `SolarSail.h`.

```
1 UENUM(BlueprintType)
2 enum class EOrbitStartType : uint8 {
3     LEO_ISS          UMETA(DisplayName = "LEO (Stazione Spaziale - 400km)",
4     ME0_GPS          UMETA(DisplayName = "ME0 (GPS - 20.200km)",
5     Geostationary    UMETA(DisplayName = "GEO (Geostazionaria - 35.786km)",
6     LunarDistance    UMETA(DisplayName = "Distanza Lunare (384.400km)",
7     CustomAltitude  UMETA(DisplayName = "Altitudine Personalizzata")
8 };
```

Il sistema, tuttavia, non forza la comparsa della vela su un asse cartesiano predefinito. Al contrario, il codice rileva le coordinate in cui l'utente ha posizionato manualmente la vela all'interno della scena di Unreal (`EditorLocation`) e calcola un versore (`RadialDir`) che parte dal centro della Terra e punta esattamente in quella direzione.

Listing 3.4: Calcolo della posizione e della distanza radiale in `SolarSail.cpp`.

```
1 FVector3d EditorLocation = FVector3d(GetActorLocation());
2 FVector3d EarthPosUU = Manager->EARTH_POSITION * Manager->KM_TO_UU;
3
4 // Calcolo della direzione in base al posizionamento nell'editor
5 FVector3d RadialDir = (EditorLocation - EarthPosUU).GetSafeNormal();
6
7 // Controllo di sicurezza per evitare vettori nulli
8 if (RadialDir.IsNearlyZero()) {
9     RadialDir = FVector3d(1, 0, 0);
10 }
```

Una volta stabilita la direzione, l'algoritmo estrae l'altitudine richiesta dal valore scelto dall'utente nell'editor e vi somma il raggio equatoriale terrestre, ottenendo la distanza radiale reale dal centro di massa del pianeta. Questo valore viene convertito nelle unità di misura del motore grafico (`TargetOrbitRadiusUU`) e moltiplicato per il versore direzionale.

Listing 3.5: Calcolo del raggio orbitale in `SolarSail.cpp`.

```
1 double TargetOrbitRadiusKm = SelectedAltitudeKm + Manager->EARTH_RADIUS;
2 double TargetOrbitRadiusUU = TargetOrbitRadiusKm * Manager->KM_TO_UU;
3 FVector3d NewLocationUU = EarthPosUU + (RadialDir * TargetOrbitRadiusUU);
```

A questo punto, il calcolo matematico deve essere tradotto in un'azione all'interno dell'ambiente virtuale di Unreal. Tramite l'istruzione `SetActorLocation()`, il software sposta fisicamente il modello 3D della vela alle nuove coordinate appena ottenute.

(`NewLocationUU`). Contemporaneamente, le variabili di stato che rappresentano la posizione della vela e la distanza dal centro della Terra vengono aggiornate con i nuovi valori:

Listing 3.6: Assegnazione delle coordinate e delle variabili di stato in `SolarSail.cpp`.

```
1 // Spostamento fisico dell'attore nella scena
2 SetActorLocation(FVector(NewLocationUU));
3
4 // Inizializzazione delle variabili per il solutore fisico
5 SailPosition = NewLocationUU;
6 OrbitRadius = TargetOrbitRadiusKm;
7 SailDistanceFromEarth = OrbitRadius;
```

Conservare in memoria questi valori (in particolar modo `SailDistanceFromEarth`) è fondamentale: senza di essi, infatti, il motore fisico non avrebbe i dati di partenza per poter calcolare la forza gravitazionale e la spinta solare nel momento di inizio della simulazione.

Più semplicemente, l'algoritmo non stravolge la scena costruita dall'utente, ma la regolarizza. Il sistema guarda in che direzione è stata piazzata la vela nell'editor e si limita a "spingerla" lungo quella linea immaginaria, allontanandola, o avvicinandola, finché non raggiunge l'altitudine impostata.

È stato inoltre inserito un meccanismo di emergenza: se per errore la vela venisse posizionata al centro della Terra (coordinate 0.0,0.0,0.0), il software non saprebbe in quale direzione allontanarla, poiché la distanza risulterebbe zero. Per evitare che questo errore matematico faccia andare in *crash* il programma, l'algoritmo sposta la vela lungo l'asse X, garantendo un avvio sicuro della simulazione.

Listing 3.7: Meccanismo di sicurezza per posizionamento errato in `SolarSail.cpp`.

```
1 if (RadialDir.IsNearlyZero()) {
2     RadialDir = FVector3d(1, 0, 0);
3 }
```

3.2.4 Correzione dell'assetto e allineamento orbitale

Posizionare la vela alla giusta distanza dalla Terra non basta. Affinché un'orbita circolare sia stabile sin dal primo istante, la spinta iniziale deve essere rigorosamente perpendicolare all'attrazione gravitazionale (ovvero a 90 gradi rispetto al raggio che unisce la vela al centro del pianeta).

Poiché il simulatore permette all'utente di orientare liberamente la vela nell'editor grafico prima dell'avvio, è altamente probabile che la direzione scelta manualmente non sia una tangente perfetta. Anche un errore umano di frazioni di grado nell'inclinazione del modello 3D comporterebbe una componente di velocità radiale indesiderata, trasformando l'orbita circolare in un'ellisse o, nel peggiore dei casi, portando la vela a schiantarsi.

Per prevenire questo problema, prima di applicare qualsiasi forza, il sistema esegue la funzione `SetInitialRotations()`, che si occupa di "raddrizzare" l'assetto del veicolo. L'algoritmo confronta la direzione verticale della vela (l'asse Z, scelto dall'utente) con la direzione radiale esatta della gravità terrestre (`RadialDir`). Tramite un prodotto scalare

(`DotProduct`), il sistema calcola la porzione del vettore utente che sta accidentalmente puntando verso o contro la Terra, e la sottrae dal vettore originale:

Listing 3.8: Correzione vettoriale della rotazione iniziale in `SolarSail.cpp`.

```
1 // Estrazione asse Z impostato dall'utente
2 FVector3d UserForward = FVector3d(GetActorUpVector());
3
4 // Calcolo della componente errata lungo il raggio e sua sottrazione
5 double RadialComponent = FVector3d::DotProduct(UserForward, RadialDir);
6 FVector3d ValidTangentDir = UserForward - (RadialDir * RadialComponent);
```

Il risultato, `ValidTangentDir`, è un vettore puramente tangenziale all'orbita. A questo punto, il codice ricostruisce la matrice di rotazione dell'oggetto (`FRotationMatrix::MakeFromZX`) e costringe il modello 3D della vela a ruotare (`SetActorRotation`), allineando in modo impeccabile il suo asse Z alla nuova tangente calcolata.

Listing 3.9: Allineamento dell'assetto alla tangente orbitale in `SolarSail.cpp`.

```
1 FRotator NewRotation = FRotationMatrix::MakeFromZX((FVector)
   ValidTangentDir, (FVector)UserUpRef).Rotator();
2
3 SetActorRotation(NewRotation);
```

Anche in questo caso è previsto un meccanismo di sicurezza: se la vela fosse stata orientata dall'utente esattamente verso il centro della Terra, la sottrazione genererebbe un vettore nullo. In tal caso il sistema sceglie arbitrariamente di allineare la vela lungo l'asse X, garantendo comunque un assetto tangenziale corretto e un avvio stabile della simulazione.

Listing 3.10: Meccanismo di sicurezza per assetto errato in `SolarSail.cpp`.

```
1 if (ValidTangentDir.IsNearlyZero()) {
2     ValidTangentDir = FVector3d::CrossProduct(RadialDir,
3                                               FVector3d(1, 0, 0));
4     if (ValidTangentDir.IsNearlyZero()) {
5         ValidTangentDir = FVector3d(0, 1, 0);
6     }
7 }
```

3.2.5 Calcolo e attribuzione della velocità orbitale iniziale

Ancora una volta le modifiche strutturali applicate alla vela non sono sufficienti. Se la vela venisse semplicemente "rilasciata" in orbita con velocità nulla, l'attrazione gravitazionale terrestre la farebbe precipitare verso il centro del pianeta al primo istante di simulazione. Per mantenere la vela in un'orbita stabile, è indispensabile conferirle una velocità di lancio iniziale, in modo che la forza centrifuga bilanci l'attrazione gravitazionale.

Questa operazione viene eseguita dalla funzione `SetInitialVelocity()`. L'algoritmo calcola innanzitutto il modulo (l'intensità) della velocità necessaria, sfruttando la formula classica della meccanica orbitale:

$$v = \sqrt{\frac{G \cdot M}{r}} \quad (3.1)$$

Dove:

- G è la costante di gravitazione universale, definita come `GRAVITATIONAL_CONSTANT`.
- M è la massa della Terra, definita come `EARTH_MASS`.
- r è la distanza radiale della vela dal centro della Terra, convertita in metri e ottenuta dalla variabile `SailDistanceFromEarth`, precedentemente calcolata durante l'inizializzazione della posizione.

Tuttavia, il motore fisico di Unreal Engine richiede un vettore direzionale tridimensionale, non un semplice numero scalare. Essendo l'assetto della vela appena stato corretto e forzato sulla tangente orbitale (come descritto nel paragrafo precedente tramite la funzione `SetInitialRotation()`), l'algoritmo non ha bisogno di eseguire ulteriori calcoli per stabilire la direzione in cui deve essere applicata la velocità orbitale iniziale. La direzione di lancio viene semplicemente estratta dall'asse verticale (l'asse Z locale, o "Up Vector") dell'attore tramite l'istruzione `GetActorUpVector()`.

Una volta ottenuto questo versore direzionale il sistema lo moltiplica per il modulo `InitialOrbitVelocityModule` calcolato in precedenza per ottenere così il vettore della velocità iniziale.

Infine, tramite la funzione `SetPhysicsLinearVelocity()`, la velocità calcolata viene passata direttamente al motore fisico di Unreal, imprimendo alla sonda la spinta meccanica reale per farla partire. In questo modo, l'oggetto entra a far parte a tutti gli effetti del sistema dinamico gestito dal motore.

Listing 3.11: Calcolo e assegnazione della velocità orbitale iniziale in `SolarSail.cpp`.

```
1 // Calcolo della distanza in metri con soglia di sicurezza
2 double Dist = FVector3d::Distance(CurrentPos, EarthPosUU) * Manager->
  UU_TO_METERS;
3 if (Dist < 1000.0) {
4     Dist = 1000.0;
5 }
6
7 // Calcolo del modulo della velocita' orbitale
8 InitialOrbitVelocityModule = FMath::Sqrt((Manager->GRAVITATIONAL_CONSTANT
  * Manager->EARTH_MASS) / Dist);
9
10 // Estrazione della direzione di lancio
11 InitialOrbitVelocityVersor = FVector3d(GetActorUpVector());
12
13 // Calcolo del vettore finale
14 InitialOrbitVelocity = InitialOrbitVelocityVersor *
  InitialOrbitVelocityModule;
15
16 // Assegnazione della velocita' al motore fisico di Unreal
17 if(SAIL_MESH->IsSimulatingPhysics()) {
18     SAIL_MESH->SetPhysicsLinearVelocity(FVector(InitialOrbitVelocity));
19 }
```

3.2.6 Inizializzazione della telemetria e delle condizioni di partenza

Dopo aver posizionato la vela, averne corretto l'assetto e averle conferito la giusta velocità iniziale, la vela è fisicamente pronta per iniziare a orbitare. L'ultima istruzione eseguita dal `BeginPlay()` non riguarda la dinamica della vela, ma la preparazione al salvataggio dei dati persistenti.

Qualora l'utente abbia abilitato la registrazione dei dati, l'esecuzione della funzione `InitializeCSVReporting()` prepara il terreno per l'analisi della missione. In questa fase il sistema non elabora ancora alcuna dinamica, ma prepara l'ambiente di lavoro sul disco, creando le cartelle di destinazione e generando un file univoco per la vela (ad esempio `SolarSailData_Sail_1.csv`). Il vero pregio di questa impostazione è la sua scalabilità nativa: se la simulazione prevede il lancio simultaneo di un'intera flotta di vele, il codice lo rileva e inizializza in automatico un file di telemetria dedicato per ogni singolo veicolo.

In questa fase il sistema si limita a preparare i file e a resettare i flag di scrittura (`bCsvHeaderWritten`); la vera registrazione dei parametri fisici iniziali e la scrittura dell'intestazione (*header*) avverranno in automatico durante la prima chiamata della funzione `AppendDataToCSV` al primo frame utile della simulazione. Lo schema di salvataggio dei dati persistenti e l'analisi della telemetria di debug verranno approfonditi nel Capitolo 5.

3.3 Modellazione della gravità terrestre

Conclusa la fase di inizializzazione, il simulatore entra nel suo ciclo vitale: l'aggiornamento continuo a ogni fotogramma (*Tick*). Da questo momento in poi, il moto della vela non è più un semplice parametro preimpostato, ma è governato dall'interazione dinamica delle forze in gioco. La più importante e costante di queste è senza dubbio l'attrazione gravitazionale terrestre.

Per mantenere l'architettura del software pulita ed efficiente, l'elaborazione di questa dinamica è stata suddivisa in due passaggi logici: il calcolo matematico (affidato al sistema centrale `SimulationManager`) e l'applicazione meccanica (gestita dal singolo veicolo `SolarSail`).

3.3.1 Calcolo vettoriale dell'accelerazione di gravità

La funzione `GetEarthGravityAccelerationAt()`, contenuta nel `SimulationManager`, funge da calcolatrice universale per l'intero sistema. Il suo scopo è determinare l'esatta accelerazione di gravità in un qualsiasi punto dello spazio virtuale.

L'algoritmo parte dalla formulazione classica della legge di gravitazione universale di Newton, riadattata per trovare l'accelerazione (g):

$$g = \frac{G \cdot M}{r^2} \quad (3.2)$$

Dove:

- G è la costante di gravitazione universale, definita come `GRAVITATIONAL_CONSTANT` e convertita in metri cubi al secondo quadrato per chilogrammo.
- M è la massa della Terra, definita come `EARTH_MASS` e anch'essa espressa in chilogrammi.
- r è la distanza radiale della vela dal centro della Terra, espressa in metri e calcolata a partire dalla distanza in chilometri (`DistanceFromEarthKm`) fornita come parametro di ingresso alla funzione.

Prima di eseguire il calcolo, il codice converte la distanza dal centro della Terra (r) in metri, un passaggio obbligatorio poiché la costante gravitazionale G opera su questa specifica unità di misura. Ottenuto quindi il modulo dell'accelerazione (`GravityAccelerationMagnitude`), il sistema necessita di un versore per orientare la forza nel mondo tridimensionale di Unreal Engine.

L'algoritmo genera quindi un vettore direzionale normalizzato (`DirectionToEarth`) che parte dalla posizione della sonda e punta dritto verso le coordinate terrestri. Moltiplicando questo versore per il modulo g , si ottiene il risultato finale: un vettore tridimensionale che indica al motore grafico non solo l'intensità dell'attrazione gravitazionale, ma anche la direzione verso la quale la vela viene attratta.

Listing 3.12: Calcolo dell'accelerazione gravitazionale in `SimulationManager.cpp`.

```

1 double DistanceFromEarthM = DistanceFromEarthKm * 1000.0;
2
3 // Calcolo del modulo dell'accelerazione (g = GM/r^2)
4 double GravityAccelerationMagnitude = (GRAVITATIONAL_CONSTANT *
5     EARTH_MASS) / (DistanceFromEarthM * DistanceFromEarthM);
6
7 // Vettore direzionale verso la Terra
8 FVector3d DirectionToEarth = (EARTH_POSITION - (SailLocationUU * UU_TO_KM
9     )).GetSafeNormal();
10
11 return DirectionToEarth * GravityAccelerationMagnitude;

```

3.3.2 Applicazione della forza e aggiornamento dinamico

Mentre il Manager si occupa della teoria matematica, la classe `ASolarSail` deve tradurre quei calcoli in vero movimento. Ad ogni singolo *tick*, la vela richiama la funzione `UpdateGravityForce()`.

La specifica vela consulta il Manager (`SimulationManager`) passandogli la propria posizione e ottiene in risposta il vettore dell'accelerazione appena calcolato. Applicando il secondo principio della dinamica ($F = m \cdot a$), l'algoritmo lo moltiplica per la massa del veicolo (`SAIL_MASS`), ricavando la forza gravitazionale vera e propria, espressa in Newton.

A questo punto, la forza viene inserita nel motore fisico di Unreal tramite la funzione nativa `AddForce()`. Mentre la velocità di lancio è stata assegnata una sola volta come valore iniziale, `AddForce` esercita un'attrazione ininterrotta. Ad ogni *tick*, questa istruzione attira la sonda verso il centro del pianeta, agendo come forza centripeta per bilanciare l'inerzia del veicolo e mantenerlo costantemente vincolato alla sua orbitale.

Listing 3.13: Applicazione dinamica della forza gravitazionale in `SolarSail.cpp`.

```

1 // Richiesta dell'accelerazione al Manager
2 FVector3d GravityAccel = Manager->GetEarthGravityAccelerationAt(
    SailPosition, SailDistanceFromEarth);
3
4 // F = m * a
5 GravityForce = GravityAccel * SAIL_MASS;
6
7 // Applicazione continua della forza al motore fisico (se nei limiti di
    tempo)
8 if(Manager->TimeScale <= 1.1f) {
9     if(SAIL_MESH->IsSimulatingPhysics()) {
10         SAIL_MESH->AddForce(FVector(GravityForce));
11     }
12 }

```

È importante sottolineare un limite di sicurezza fondamentale imposto in questa funzione: l'uso del comando nativo `AddForce` è consentito solo se il tempo scorre a velocità quasi reale (`TimeScale <= 1.1f`). I motori fisici, come *Chaos Physics* per Unreal Engine, calcolano il movimento per ogni fotogramma. Se l'utente accelerasse la simulazione in modo estremo, lo spazio percorso dalla sonda tra un calcolo e l'altro diventerebbe troppo ampio: invece di descrivere una curva orbitale fluida, il veicolo procederebbe su lunghe rette, finendo per 'allargare' inesorabilmente la traiettoria e fuggire dall'orbita. Per evitare questo grave errore matematico, la fisica di Unreal viene sfruttata solo quando la densità dei calcoli garantisce una simulazione stabile.

3.4 Il Campo di Pressione Solare (SRP)

Accanto all'attrazione gravitazionale, la seconda componente fondamentale per la simulazione è la Pressione di Radiazione Solare (*Solar Radiation Pressure*). Al contrario della gravità, che agisce sul centro di massa del veicolo, la pressione solare è una forza che agisce su tutta la superficie della vela ed è generata dall'impatto dei fotoni contro di essa.

Mantenendo la coerenza strutturale definita al paragrafo precedente, anche il calcolo di questa interazione è stato separato in due parti logiche distinte: il calcolo della pressione di radiazione vera e propria (gestita centralmente dal `SimulationManager`) e il calcolo della forza risultante sull'oggetto (delegata invece a `SolarSail`).

3.4.1 Calcolo della distanza Vela-Sole

L'intensità della radiazione solare non è costante, ma dipende strettamente dalla posizione della vela rispetto alla posizione del Sole. Pertanto, il primo valore necessario all'algoritmo per quantificare la spinta dei fotoni è la distanza tra il veicolo e la fonte luminosa.

All'interno della funzione `GetSolarPressureAt()`, situata all'interno del gestore centrale `SimulationManager`, il codice esegue per prima cosa una verifica sui parametri in ingresso. Il sistema controlla se la distanza radiale dal Sole (`SailDistanceFromSunKm`) è già stata calcolata e fornita dalla vela che la richiama. In caso contrario (valore nullo o negativo), il Manager provvede a calcolarla autonomamente.

L'operazione viene eseguita misurando la distanza vettoriale tra le coordinate della vela (`SailLocationUU`) e la posizione fissa del Sole (`SUN_POSITION`), precedentemente scalata nelle unità di misura del motore grafico. Il risultato viene poi subito riconvertito in chilometri tramite il fattore `UU_TO_KM`, mantenendo lo schema di unità coerente con il resto del sistema.

3.4.2 La Legge del Quadrato Inverso applicata ai fotoni

Una volta ottenuta la distanza su cui si trova la vela rispetto al Sole, l'algoritmo procede al calcolo effettivo della pressione di radiazione. La propagazione dell'energia luminosa nel vuoto è regolata dalla legge dell'inverso del quadrato: man mano che i fotoni si allontanano dalla stella, si distribuiscono su una superficie sferica sempre più ampia, causando un decadimento quadratico dell'intensità.

Il codice implementa questo principio fisico attraverso la formula standard per il calcolo della pressione solare (P):

$$P = \frac{L}{4\pi r^2 c} \quad (3.3)$$

Dove:

- L è la luminosità solare, definita come `SOLAR_LUMINOSITY` ($3.828 \times 10^{26} W$).
- r è la distanza radiale dal Sole, definita come `DistanceFromSunKm` convertita in metri.
- c è la velocità della luce nel vuoto, definita come `SPEED_OF_LIGHT` e convertita in metri al secondo.

Per fare in modo di essere il più coerenti possibile alle costanti fisiche (esprese nel Sistema Internazionale), il codice converte automaticamente la distanza `DistanceFromSunKm` in metri prima di eseguire la divisione. Il risultato di questa operazione è un valore scalare espresso in Pascal (N/m^2), che rappresenta l'esatta pressione esercitata dalla luce in quel preciso punto dello spazio.

Listing 3.14: Calcolo della Pressione Solare nel `SimulationManager.cpp`.

```

1  double ASimulationManager::GetSolarPressureAt(FVector3d SailLocationUU,
2  double SailDistanceFromSunKm) const {
3  // Risoluzione della distanza in Chilometri
4  double DistanceFromSunKm = SailDistanceFromSunKm > 0.0 ?
5  SailDistanceFromSunKm : FVector3d::Distance(SailLocationUU,
6  SUN_POSITION * KM_TO_UU) * UU_TO_KM;
7
8  // Conversione obbligatoria in Metri per l'analisi dimensionale
9  double DistanceFromSunM = DistanceFromSunKm * 1000.0;
10
11 // P = L / (4 * PI * r^2 * c)
12 double SolarPressure = SOLAR_LUMINOSITY / (4.0 * PI_GREEK *
13 DistanceFromSunM * DistanceFromSunM * (SPEED_OF_LIGHT * 1000.0));
14
15 return SolarPressure * ForceMultiplier;
16 }

```

3.4.3 Ottimizzazione del calcolo della pressione solare

Delegare il calcolo di questa formula al `SimulationManager` non è solo una questione di ordine nel codice, ma una scelta logica ben precisa. La pressione solare, infatti, è un dato ambientale universale: in questa fase dipende esclusivamente dalla distanza, a prescindere dall'inclinazione, dal materiale o dalla grandezza della vela che la riceve.

Mantenendo questa separazione logica, il sistema assicura che il Manager si comporti come un vero e proprio "fornitore" per quanto riguarda la pressione solare. Quando la classe `ASolarSail` richiamerà questa funzione per calcolare la propria spinta, riceverà un valore di pressione pronto per i prossimi calcoli di forza.

3.4.4 Alterazione controllata delle forze con *Force Multiplier*

Inoltre, questa struttura centralizzata permette l'inserimento di variabili di controllo globali, come il `ForceMultiplier`. Moltiplicando il risultato finale per questo fattore esposto all'utente, il sistema offre la possibilità di aumentare o diminuire artificialmente l'intensità della luce solare per scopi di testing. Essendo inserito all'interno del Manager, questo moltiplicatore influenzerà coerentemente tutte le sonde presenti nella simulazione, senza richiedere modifiche successive ai calcoli locali delle singole vele.

3.5 Lo schema ottico della Membrana

Una volta calcolata la pressione ambientale esercitata dalla radiazione solare, il simulatore deve determinare come questa energia si traduca in vera e propria spinta meccanica per la vela. L'interazione tra i fotoni e il veicolo solare non è un semplice urto anelastico: il trasferimento di quantità di moto dipende in maniera diretta dalle proprietà ottiche e dai materiali di cui è composta la membrana.

3.5.1 Scomposizione dell'interazione fotone-superficie

Nella realtà astrodinamica, quando un flusso di fotoni colpisce una superficie, la sua energia viene frammentata in tre fenomeni ottici primari:

- **Assorbimento (α):** Il fotone viene catturato dal materiale, cedendo la sua intera quantità di moto ma generando calore.
- **Riflessione speculare (ρ_s):** Il fotone rimbalza sulla superficie con un angolo di uscita identico a quello di ingresso, fornendo una spinta extra grazie al principio di azione e reazione.
- **Riflessione diffusa (ρ_d):** Il fotone rimbalza, ma a causa dell'irregolarità del materiale viene disperso in una direzione casuale, riducendo l'efficienza della spinta.

3.5.2 Parametrizzazione dei Coefficienti Ottici

Per ottimizzare il sistema, il modello matematico condensa queste proprietà fisiche in un unico moltiplicatore, denominato nel codice `SelectedReflectivityFactor`.

Questo coefficiente sintetizza i parametri ottici (ρ_s, ρ_d, α) e rappresenta l'efficienza specifica della membrana nel tramutare la luce in spinta vettoriale, permettendo al simulatore di calcolare la forza risultante in un unico passaggio algebrico.

3.5.3 Analisi dei Preset di Riflettività

Per garantire flessibilità e facilità d'uso, il software offre all'utente una serie di preset predefiniti per il coefficiente di riflettività, ognuno dei quali simula un diverso tipo di materiale:

Listing 3.15: Selezione del coefficiente ottico in `SolarSail.cpp`.

```
1 double SelectedReflectivityFactor = 0.0;
2 switch (Reflectivity_Factor) {
3     case EReflectivityPreset::Absorber: SelectedReflectivityFactor =
4         RFACTOR_ABSORBER; break; // 1.0
5     case EReflectivityPreset::Medium: SelectedReflectivityFactor =
6         RFACTOR_MEDIUM; break; // 1.5
7     case EReflectivityPreset::Realistic: SelectedReflectivityFactor =
8         RFACTOR_REALISTIC; break; // 1.8
9     case EReflectivityPreset::Perfect: SelectedReflectivityFactor =
10         RFACTOR_PERFECT; break; // 2.0
11     case EReflectivityPreset::Custom: SelectedReflectivityFactor =
12         RFACTOR_CUSTOM; break;
13 }
```

Analizzandoli più nello specifico:

- **Absorber** (`SelectedReflectivityFactor` = 1.0): Simula un corpo perfettamente nero. La membrana cattura tutta la luce incidente e ne assorbe solamente la spinta dovuta all'impatto, senza alcun contributo aggiuntivo derivante dalla riflessione.
- **Medium** (`SelectedReflectivityFactor` = 1.5): Rappresenta un materiale con caratteristiche ottiche intermedie. Risulta utile per valutare scenari di simulazione in cui la vela ha subito un grave degrado spaziale o è composta da polimeri semi-riflettenti.
- **Realistic** (`SelectedReflectivityFactor` = 1.8): Offre una simulazione fisicamente accurata dei materiali aerospaziali reali (come il Mylar o il Kapton alluminizzato). Introduce una fisiologica riduzione delle performance dovuta all'irregolarità della superficie e a un assorbimento parziale dell'energia solare.
- **Perfect** (`SelectedReflectivityFactor` = 2.0): Simula uno specchio ideale. Il fotone non solo colpisce la vela, ma rimbalza via elasticamente e intatto; per il principio di azione e reazione, questo doppio movimento raddoppia di fatto il trasferimento netto di quantità di moto.
- **Custom**: Permette all'utente di inserire manualmente un valore personalizzato per il coefficiente di riflettività, offrendo la massima libertà di sperimentazione. Questa opzione è particolarmente utile per testare materiali innovativi o scenari ipotetici che non rientrano nei preset standard.

3.5.4 Geometria dell'illuminazione e angolo di incidenza

L'effettiva spinta generata dalla pressione solare non dipende solo dalle proprietà ottiche della membrana, ma anche dall'orientamento geometrico della vela rispetto alla direzione dei raggi solari. Durante il volo, infatti, la vela solare cambia continuamente inclinazione: affinché la simulazione sia fisicamente coerente, non basta conoscere solo la riflettività della membrana, ma occorre determinare l'esatto angolo con cui i raggi solari la colpiscono.

Il calcolo di questo orientamento è affidato alla funzione `GetActorUpVector()`. Questa estrapola l'asse verticale dell'oggetto 3D e lo fornisce al simulatore sotto forma di vettore normale $\mathbf{n}$, garantendo che la direzione della vela sia sempre coerente con il sistema di riferimento.

Una volta ottenuta la normale della vela e calcolata la direzione da cui proviene la luce (`SunDirection`), l'algoritmo analizza l'angolo di incidenza dei fotoni attraverso un prodotto scalare (`FVector3d::DotProduct`):

$$\text{Alignment} = \mathbf{u} \cdot \mathbf{n} = |\mathbf{u}||\mathbf{n}| \cos(\alpha) \quad (3.4)$$

Listing 3.16: Estrazione della normale e calcolo dell'angolo di incidenza in `SolarSail.cpp`.

```

1 SailNormal = FVector3d(GetActorUpVector());
2 FVector3d SunPosUU = Manager->SUN_POSITION * Manager->KM_TO_UU;
3 SunDirection = (SunPosUU - SailPosition).GetSafeNormal();
4
5 // Calcolo dell'angolo tra la faccia della vela e i raggi solari
6 double Alignment = FVector3d::DotProduct(SunDirection, SailNormal);

```

Questa operazione matematica restituisce un moltiplicatore di efficienza: se la variabile `Alignment` è pari a 1, la vela è orientata in modo perfettamente perpendicolare ai raggi solari, massimizzando la cattura dei fotoni. Qualora il valore si avvicini allo 0, la membrana risulta posizionata parallelamente alla luce: i fotoni scorrono lungo la sua superficie senza impattare, annullando di fatto la pressione propulsiva. Se invece il valore è negativo, e il parametro `DoubleSidedSail` è impostato a `false`, significa che la vela è rivolta con la faccia opposta al Sole, e quindi non riceve alcuna spinta. In questo caso, l'algoritmo imposta la forza solare a zero, evitando di generare risultati fisicamente incoerenti.

3.6 Il Calcolo della spinta a raggi (Ray Casting)

Per calcolare in modo realistico l'impatto della luce sulla vela solare, specialmente in presenza di ostacoli o auto-ombreggiamento, non è sufficiente considerare la vela come un singolo punto. È necessario analizzarne l'intera vela come una superficie tridimensionale. Per fare questo, il simulatore utilizza un algoritmo di campionamento spaziale che mappa la superficie della sonda attraverso una fitta griglia di sensori luminosi virtuali.

3.6.1 Logica di campionamento della superficie

Il processo inizia dividendo l'area totale della membrana in una matrice bidimensionale. L'utente, tramite l'editor di Unreal Engine, definisce il parametro `GridResolution`, che determina il numero di suddivisioni per lato della vela. Per prevenire errori critici, come divisioni per zero, il codice controlla che il valore inserito sia almeno pari a 1 `SafeResolution`, garantendo così la presenza di almeno un punto di campionamento.

Elevando al quadrato questo valore, il sistema calcola il numero complessivo di "fotoni" simulati (`TotalPhotons`), ovvero quanti raggi paralleli verranno lanciati verso il Sole.

Successivamente, l'algoritmo calcola lo spazio esatto (`Step`) che deve esserci tra un punto e l'altro per distribuire i raggi in modo uniforme su tutta la vela. Fatto questo, esegue l'operazione più importante: divide l'area totale della membrana (`SAIL_AREA`) per il numero complessivo di fotoni, ottenendo l'`AreaPerRay`. Questo valore rappresenta la porzione di superficie associata a ogni singolo raggio: più è alto, più ogni raggio rappresenta una grande percentuale della vela, aumentando la forza che ogni fotone trasferisce alla sonda. Al contrario, un valore più basso implica una maggiore precisione, ma anche un aumento esponenziale delle operazioni di calcolo richieste, e quindi un maggior utilizzo della CPU.

Listing 3.17: Impostazione della griglia di campionamento in `SolarSail.cpp`.

```
1 int32 SafeResolution = FMath::Max(1, GridResolution);
2 TotalPhotons = SafeResolution * SafeResolution;
3
4 double GridSizeUU = 100.0 * SAIL_SCALE;
5 double Step = GridSizeUU / SafeResolution;
6
7 // Porzione di area (in metri quadrati) associata a ogni raggio
8 double AreaPerRay = SAIL_AREA / TotalPhotons;
```

3.6.2 Generazione e orientamento dei raggi di controllo

Una volta definita la struttura della matrice, l'algoritmo esegue un doppio ciclo *for* anidato, che itera su ogni punto di campionamento della superficie. Per ogni iterazione, il sistema calcola le coordinate del punto di origine del raggio (`SamplePos`) sulla vela, partendo dal centro della membrana e spostandosi in base alla posizione del punto nella griglia.

Per calcolare da dove far partire ogni raggio (`SamplePos`), l'algoritmo prende come riferimento il centro della vela e si sposta lungo i suoi assi direzionali (`Forward` e `Right`) per trovare la cella corretta. Una volta individuato il punto esatto sulla griglia, il codice lo imposta direttamente come origine del laser (`TraceStart`).

Ora che il punto di partenza è stato definito, è necessario orientare il raggio verso la fonte luminosa. Per chiudere l'operazione infatti, la destinazione del laser (`TraceEnd`) viene assegnata alla posizione globale del Sole (`SunPosUU`). In questo modo, a ogni giro del ciclo *for*, il simulatore traccia una linea retta perfetta tra un "pixel" specifico della sonda e il centro del Sole. Questo segmento invisibile è esattamente ciò che il motore fisico userà nel passaggio successivo per calcolare le collisioni.

Listing 3.18: Generazione delle coordinate per il singolo raggio.

```

1  for (int32 i = 0; i < SafeResolution; i++) {
2      for (int32 j = 0; j < SafeResolution; j++) {
3
4          // Calcolo degli offset per centrare la griglia
5          double OffX = (i - SafeResolution / 2.0 + 0.5) * Step;
6          double OffY = (j - SafeResolution / 2.0 + 0.5) * Step;
7
8          // Posizione esatta del "pixel" sulla vela
9          FVector3d SamplePos = SailPosition + (Forward * OffX) + (Right *
              OffY);
10
11         // Definizione di inizio e fine del raggio
12         FVector3d TraceStart = SamplePos;
13         FVector3d TraceEnd = SunPosUU;
14
15         // [Inizio routine fisica di LineTrace]
16     }
17 }

```

3.6.3 Ottimizzazioni prestazionali con `RaycastTimer`

Chiedere al motore grafico di calcolare centinaia o migliaia di collisioni (*Line Trace*) a ogni singolo fotogramma (mediamente 60 volte al secondo) è un'operazione pesantissima per la CPU. Senza un'effettiva gestione intelligente, questo approccio porterebbe a un drastico calo delle prestazioni, con frequenti blocchi o rallentamenti della simulazione.

Per garantire prestazioni fluide costanti, la funzione `UpdateSolarForce()` utilizza una misura di sicurezza nota: un *timer* di esecuzione (`RaycastTimer`). In questo modo il sistema esegue il ciclo di calcolo dei raggi solo a intervalli regolari, definiti dal parametro `RaycastInterval` (ad esempio, ogni 0.5 secondi). Durante i fotogrammi in cui il timer non è scattato, la funzione si limita ad applicare la forza solare calcolata nell'ultimo ciclo completo, senza eseguire nuovi calcoli di collisione.

Listing 3.19: Sistema di ritenzione della forza per l'ottimizzazione computazionale.

```

1  RaycastTimer += DeltaTime;
2
3  // Se il timer non e' ancora scattato, usa la forza calcolata nel ciclo
   precedente
4  if (RaycastTimer < RaycastInterval) {
5      if (Manager->TimeScale <= 1.1f && !SolarForce.ContainsNaN()) {
6          if(SAIL_MESH->IsSimulatingPhysics()) {
7              SAIL_MESH->AddForce(FVector(SolarForce));
8          }
9      }
10     return; // Interrompe la funzione saltando il calcolo della griglia
11 }
12 RaycastTimer = 0.0f; // Resetta il timer e procede con il calcolo pesante

```

Terminato il calcolo, la spinta solare complessiva viene salvata nella memoria della sonda (`SolarForce`). Nei *Tick* successivi, la funzione non esegue più alcun calcolo geometrico di collisione, ma si limita ad applicare questa forza pre-calcolata al motore fisico,

garantendo così una simulazione fluida e stabile anche con griglie di campionamento ad alta risoluzione.

3.7 Gestione delle ombre e degli ostacoli

Costruita la griglia geometrica e ricavati i punti di partenza per ogni singolo "pixel" della vela, il simulatore deve stabilire quali di queste porzioni siano esposte alla luce e quali, invece, si trovino in zone d'ombra causate da corpi che si trovano lungo la traiettoria verso il Sole, come la Terra o un asteroide. Per risolvere questo problema, il codice si affida al motore di collisione nativo di Unreal Engine.

3.7.1 Preparazione del raggio e auto-collisione

Prima di lanciare fisicamente il raggio vettoriale, il sistema deve configurare i parametri necessari affinché il motore grafico possa registrare e gestire correttamente l'eventuale collisione. Vengono innanzitutto inizializzate due variabili:

La prima è una struttura dati di tipo `FHitResult` (chiamata `Hit`). Si tratta di un parametro di output che il motore grafico compila automaticamente qualora il laser colpisca un ostacolo, memorizzando informazioni come il nome dell'oggetto colpito e la distanza esatta dell'impatto.

La seconda è una variabile di tipo `FCollisionQueryParams` (chiamata `P`), utilizzata per definire le regole di collisione. Attraverso questa variabile è possibile evitare un errore molto comune, ovvero l'auto-collisione. Questo avviene tramite l'esecuzione dell'istruzione `P.AddIgnoredActor(this)`, che impone al motore fisico di escludere la vela dal calcolo delle intersezioni. Senza questo sistema di sicurezza, i laser colpirebbero i poligoni della membrana nell'istante stesso in cui vengono generati, azzerando la spinta propulsiva e invalidando completamente la simulazione.

3.7.2 L'esecuzione del raggio (Line Trace)

Dichiarate le variabili, il codice procede ad eseguire il lancio del raggio tramite la funzione nativa `LineTraceSingleByChannel`.

Listing 3.20: Esecuzione del raggio di occlusione solare.

```
1 FHitResult Hit;
2 FCollisionQueryParams P;
3 P.AddIgnoredActor(this);
4
5 bool bHitSomething = GetWorld()->LineTraceSingleByChannel(
6     Hit,
7     FVector(TraceStart),
8     FVector(TraceEnd),
9     ECC_Visibility,
10    P
11 );
```

La funzione `LineTraceSingleByChannel` prende in ingresso diversi parametri fondamentali:

- **Hit** (**FHitResult&**): La struttura dati, che il motore grafico compilerà in caso di collisione, altrimenti rimarrà vuota.
- **TraceStart** (**FVector**): Il punto di partenza del raggio, calcolato in precedenza come la posizione del "pixel" sulla vela.
- **TraceEnd** (**FVector**): La destinazione del raggio, impostata alla posizione del Sole.
- **ECC_Visibility** (**ECollisionChannel**): Il canale di collisione da verificare. In questo caso, il sistema controlla solo gli oggetti che sono stati marcati come "visibili" per i raggi di controllo.
- **P** (**FCollisionQueryParams**): Le regole di collisione, che includono l'istruzione per ignorare la vela stessa.

Il risultato della funzione è un booleano (**bHitSomething**) che indica se il raggio ha colpito un ostacolo lungo la traiettoria verso il Sole. Se il valore è **true**, significa che la luce è stata bloccata da un corpo (ad esempio, la Terra), e quindi quel punto specifico della vela si trova in una zona d'ombra. Se invece è **false**, il raggio ha raggiunto il Sole senza ostacoli, confermando che quella porzione della membrana è illuminata e contribuisce alla spinta propulsiva.

3.7.3 Conversione spaziale e debug visivo

Subito dopo aver lanciato il raggio, il codice deve memorizzare la coordinata del punto appena analizzato per poterlo in seguito disegnare a schermo per scopi di debug (i raggi verdi e rossi). Tuttavia, sorge un problema tecnico: la vela viaggia a migliaia di chilometri orari, quindi se il simulatore memorizzasse la coordinata globale dell'impatto (**SamplePos**), nel *Tick* successivo quel punto rimarrebbe fisso nella sua posizione assoluta, mentre il modello 3D si sarebbe già spostato, disallineando i laser.

Per risolvere il problema, l'algoritmo sfrutta la funzione **InverseTransformPosition**. Questa istruzione converte istantaneamente le coordinate assolute in coordinate locali, ancorando il punto al centro della vela in modo che si muova solidalmente con essa.

Listing 3.21: Conversione del punto in coordinate locali.

```
1 FVector LocalP = ActorTrans.InverseTransformPosition(FVector(SamplePos));
```

In questo modo, il vettore risultante (**LocalP**) diventa a tutti gli effetti solidale al sistema di riferimento della *mesh* 3D. Questo garantisce che i laser di debug vengano disegnati sempre nella posizione esatta, rimanendo perfettamente solidali con il modello 3D in movimento senza creare sfasamenti visivi.

3.7.4 Il filtro della luce e il conteggio dei fotoni

L'ultimo passaggio del ciclo *for* serve a decidere se la cella analizzata genera o meno spinta. Il codice usa l'istruzione **if (!bHitSomething)** per ribaltare il risultato della collisione. In parole povere: se il raggio non ha colpito alcun ostacolo, significa che il percorso verso il Sole è completamente libero, e l'algoritmo può quindi calcolare la forza solare per quella sezione.

Il codice esegue quindi due operazioni:

1. Incrementa il contatore `ActivePhotons`, parametro fondamentale per la telemetria che indica in *real time* quante porzioni della vela stanno ricevendo luce.
2. Aggiunge la coordinata locale `LocalP` all'array `LocalActiveRays`. Questo array verrà letto dal sistema per generare i laser diagnostici verdi, a indicare una corretta illuminazione.

Al contrario, se l'istruzione rileva che il raggio è stato bloccato (ad esempio dalla Terra), il tassello si trova in una zona d'ombra. La spinta viene scartata per quel segmento, e la coordinata `LocalP` viene invece inserita nell'array `LocalBlockedRays`. Quest'ultimo verrà sfruttato per dipingere indicatori rossi.

Listing 3.22: Filtro logico e smistamento dei raggi per il debug e la fisica.

```
1  if (!bHitSomething) {
2      // Il raggio ha raggiunto il Sole: il tassello e' illuminato
3      ActivePhotons++;
4
5      // [Qui avviene il calcolo della spinta solare per questo tassello]
6
7      if (Manager->ShowPhotonDebug) {
8          LocalActiveRays.Add(LocalP); // Registrazione laser verde
9      }
10 }
11 else if (Manager->ShowPhotonDebug) {
12     // Il raggio e' stato bloccato: zona d'ombra
13     LocalBlockedRays.Add(LocalP);    // Registrazione laser rosso
14 }
```

3.8 Integrazione Vettoriale e Forza Risultante

Concluso il tracciamento dei raggi ed esclusi quelli bloccati da ostacoli, l'algoritmo dispone dell'elenco delle celle fisicamente illuminate. Per ciascuna di queste, il codice calcola l'intensità della spinta meccanica generata dalla luce, converte il valore scalare in un vettore tridimensionale e lo somma all'accumulatore per ricavare la forza totale. Questa sequenza di operazioni costituisce la base matematica per simulare la Pressione di Radiazione Solare (SRP).

3.8.1 Calcolo della forza propulsiva per singola cella

Superata la verifica geometrica di occlusione tramite l'istruzione `if (!bHitSomething)`, l'algoritmo ha la certezza che la traiettoria verso il Sole è completamente libera. Questo conferma che il frammento di vela analizzato è esposto alla luce diretta e può generare propulsione, autorizzando il sistema a passare dalla fase di tracciamento spaziale, tramite il *Ray Casting* al calcolo fisico della forza. L'obiettivo è determinare l'effettiva spinta generata dall'impatto dei fotoni su quella specifica cella. Il valore scalare di questa forza locale viene ricavata assemblando i parametri ottici e spaziali in un'unica formula algebrica:

Listing 3.23: Calcolo dell'incidenza geometrica e della magnitudo della spinta.

```

1 // 1. Calcolo dell'angolo di incidenza tramite prodotto scalare
2 double CosTheta = FVector::DotProduct(SailNormal, SunDirection);
3
4 // 2. Recupero della pressione solare ambientale dal Manager
5 double LocalPressure = Manager->GetSolarPressureAt(SamplePos,
6     SailDistanceFromSun);
7
8 // 3. Calcolo finale della magnitudo (Newton) per la singola cella
9 double ForceMag = SelectedReflectivityFactor * LocalPressure * AreaPerRay
10     * (CosTheta * CosTheta);

```

Questa singola riga di codice implementa in C++ la vera e propria formula della pressione solare su una superficie riflettente. Scomponendo l'istruzione, ogni variabile moltiplicata ha un compito fisico ben preciso:

- **SelectedReflectivityFactor**: Rappresenta l'efficienza ottica del materiale e implementa fisicamente il principio di conservazione della quantità di moto. Interrogando l'enum `EReflectivityPreset`, il codice assegna un moltiplicatore specifico: 1.0 per un assorbitore perfetto (il fotone cede la sua energia arrestandosi) e 2.0 per uno specchio perfetto (il fotone rimbalza, trasferendo il doppio della quantità di moto). Questa variabile scala direttamente la forza in base alle proprietà del materiale.
- **LocalPressure**: Determina l'energia fornita dal Sole alla distanza in cui si trova la specifica cella. Viene calcolata dinamicamente dal `SimulationManager` tramite la funzione `GetSolarPressureAt()`. Il metodo elabora le costanti astrofisiche (luminosità solare e velocità della luce) restituendo un valore in Pascal che decade rigorosamente secondo la legge dell'inverso del quadrato della distanza.
- **AreaPerRay**: Rappresenta la porzione di superficie assegnata a ciascun raggio. Per convertire correttamente la pressione ambientale (Pascal) in forza propulsiva (Newton), il calcolo deve agire su un'area reale. L'algoritmo ricava questo valore dividendo semplicemente i metri quadrati totali della membrana per il numero di raggi campionati (`SAIL_AREA / TotalPhotons`).
- **(CosTheta * CosTheta)**: Gestisce l'incidenza geometrica dei fotoni rispetto all'orientamento della *mesh*. L'elevazione al quadrato del coseno ($\cos^2 \theta$) deriva dalla necessità di risolvere due distinti fenomeni fisici che avvengono simultaneamente:
 - **Riduzione dell'area esposta**: Il primo `CosTheta` tiene conto della variazione dell'area della cella che è effettivamente esposta. Quando la vela è inclinata rispetto al Sole, la sua area esposta alla luce diminuisce. Questo riduce il numero di fotoni che colpiscono effettivamente la superficie.
 - **Estrazione della spinta normale**: Il secondo `CosTheta` serve a isolare unicamente la forza che agisce perpendicolarmente alla vela. Nel caso di riflessione speculare, le componenti della forza parallele alla superficie si annullano a vicenda; resta utile ai fini della spinta solo il vettore normale. Moltiplicando nuovamente per il coseno, il codice scarta le componenti tangenziali e calcola la spinta effettiva lungo l'asse di movimento della vela.

L'integrazione di questi due contributi impone nel codice la legge del quadrato del coseno ($\cos^2 \theta$), fondamentale per la navigazione spaziale basata sulla pressione di radiazione. Questa implementazione evita che il sistema calcoli spinte artificialmente elevate quando la sonda non è perfettamente allineata al Sole.

3.8.2 Integrazione numerica della spinta totale

La variabile `ForceMag` costituisce unicamente il modulo della spinta espresso in Newton. Per essere integrata nel calcolo dinamico, questa grandezza scalare deve essere trasformata in un vettore applicando l'orientamento spaziale calcolato dal sistema, altrimenti non può essere sommata alle altre forze che agiscono sulla vela.

Il sistema trasforma il modulo della forza in un vettore tridimensionale moltiplicando il valore di `ForceMag` per il versore `PushDirection`. Quest'ultimo viene calcolato all'inizio della procedura analizzando l'allineamento (`Alignment`) tra la normale della vela e la direzione dei raggi solari tramite prodotto scalare. Questa operazione garantisce la corretta direzionalità della spinta, assicurando che il vettore risultante agisca sempre in opposizione alla sorgente luminosa e colpisca esclusivamente la faccia esposta della membrana.

Listing 3.24: Logica di calcolo: dal verso pre-determinato all'accumulo scalare.

```

1 // --- PRIMA DEL DOPPIO CICLO ---
2 // Logica "Double-Sided": si determina il verso in base alla faccia
  illuminata
3 if (Alignment < 0) {
4     PushDirection = SailNormal;
5     CosTheta = FMath::Abs(Alignment);
6 } else {
7     PushDirection = -SailNormal;
8     if (DoubleSidedSail) {
9         CosTheta = Alignment;
10    } else {
11        CosTheta = 0.0;
12    }
13 }
14 FVector3d AccumulatedForce = FVector3d::Zero(); // Accumulatore scalare
15
16 // --- DENTRO IL DOPPIO CICLO (per ogni fotone) ---
17 // Accumulo della sola intensità (Newton) per ogni raggio non occluso
18 AccumulatedForce += ForceMag * PushDirection;

```

Questa architettura ottimizza il calcolo dividendo la valutazione logica dell'assetto dal ciclo intensivo di campionamento dei fotoni. Pre-determinando il verso della spinta e il coefficiente angolare globale prima di iniziare l'iterazione, il sistema elimina migliaia di ricalcoli ridondanti, garantendo al contempo la coerenza fisica della membrana:

- **PushDirection (Verso della spinta):** Definisce l'orientamento del vettore forza in base alla faccia esposta al Sole. Sfruttando il segno di `Alignment`, il codice decide se la spinta seguirà la normale (`SailNormal`) o il suo inverso (`-SailNormal`). Questa inversione automatica è fondamentale per la navigazione, poiché permette alla sonda di generare propulsione sempre in allontanamento dalla sorgente luminosa, indipendentemente dalla rotazione del corpo rigido.

- **Gestione `DoubleSidedSail`:** Il sistema utilizza un controllo condizionale preventivo per abilitare o disabilitare la propulsione. Se la luce colpisce il retro di una vela configurata come *single-sided*, il valore di `CosTheta` viene forzato a 0.0. Questo "interruttore logico" annulla matematicamente il valore della forza all'interno del ciclo, simulando una superficie trasparente o non attiva sul lato posteriore senza appesantire il loop con ulteriori controlli *if*.
- **`AccumulatedForce` (Accumulatore Vettoriale):** Funge da buffer per l'integrazione della spinta totale. Poiché `PushDirection` e `CosTheta` sono già stati validati all'esterno, ad ogni iterazione il codice si limita a sommare il contributo locale (`ForceMag * PushDirection`) al totale. Al termine della scansione, la variabile restituisce la sommatoria di tutti i contributi di pressione attivi, escludendo i contributi delle parti della vela oscurate.

Terminato il doppio ciclo iterativo, il buffer `AccumulatedForce` contiene il vettore della spinta solare risultante. A questo punto, il codice archivia i dati nelle variabili globali per la telemetria e il sistema di log CSV:

Listing 3.25: Estrazione dei parametri telemetrici.

```
1 SolarForce = AccumulatedForce;
2 SolarForceModule = SolarForce.Size();
3 SolarForceVersor = SolarForce.GetSafeNormal();
```

Oltre al vettore puro (`SolarForce`), vengono calcolati e salvati il modulo (`Size()`) e il versore (`GetSafeNormal()`) della forza, parametri fondamentali per la telemetria e l'analisi della simulazione.

3.8.3 Applicazione al motore fisico e controlli di sicurezza

L'ultima fase della funzione `UpdateSolarForce` consiste nel tradurre la matematica in un'accelerazione reale all'interno dello spazio tridimensionale di Unreal Engine. Prima di passare il vettore `SolarForce` al solutore dinamico nativo di Unreal Engine (PhysX/Chaos), il codice si assicura che esso sia privo di errori numerici e che la simulazione fisica sia attiva, altrimenti la forza non verrebbe applicata correttamente, o peggio, potrebbe causare crash o comportamenti imprevedibili.

Listing 3.26: Verifiche di sicurezza e applicazione della forza al Rigid Body.

```
1 if (Manager->TimeScale <= 1.1f && !SolarForce.ContainsNaN()) {
2     if (SAIL_MESH->IsSimulatingPhysics()) {
3         SAIL_MESH->AddForce(FVector(SolarForce));
4     }
5 }
```

L'istruzione `if` principale richiede che vengano soddisfatte due condizioni fondamentali prima di applicare la forza:

1. **Assenza di NaN:** La funzione `ContainsNaN()` verifica che il vettore `SolarForce` non contenga valori "Not a Number". Questi errori numerici possono emergere da operazioni matematiche non definite (ad esempio, divisioni per zero o radici quadrate di numeri negativi) e causano comportamenti imprevedibili nel motore

fisico, come accelerazioni infinite o crash. Questo controllo garantisce che solo forze valide vengano applicate alla vela.

2. **Manager->TimeScale <= 1.1f (Switch Cinematico):** Questo controllo impedisce che la forza venga applicata nel motore fisico nativo durante le fasi di accelerazione temporale (`TimeScale > 1.1f`). Infatti durante il "fast forward", la simulazione fisica viene disabilitata per evitare instabilità e garantire che i calcoli di spinta non interferiscano con la velocità di esecuzione. In questo modo, quando l'utente accelera il tempo, il sistema non applica fisicamente la forza nel motore grafico, ma utilizza i vettori appena calcolati per aggiornare la traiettoria tramite integrazione numerica, mantenendo la coerenza orbitale della vela anche a velocità di esecuzione estreme.

Superati i controlli di sicurezza, l'algoritmo verifica che il componente grafico sia effettivamente governato dal solutore dinamico (`IsSimulatingPhysics()`). Se l'esito è positivo, la spinta viene applicata tramite l'istruzione nativa `SAIL_MESH->AddForce()`. Durante la chiamata avviene un *cast* esplicito da `FVector3d` a `FVector`: questa operazione converte il dato dalla doppia precisione (indispensabile a monte per elaborare le distanze interplanetarie senza incorrere in arrotondamenti indesiderati) alla singola precisione a 32-bit, ovvero il formato richiesto nativamente dal motore di collisione di Unreal Engine. L'esecuzione di questo comando chiude il ciclo di aggiornamento: l'analisi ottica e geometrica della luce è stata formalmente tradotta in un'accelerazione spaziale sul corpo rigido.

Capitolo 4

Dinamica Discreta e Time Warp

Tutte le operazioni descritte nel capitolo precedente, dal calcolo della gravità tramite `UpdateGravityForce()` all'aggiornamento della spinta solare con `UpdateSolarForce()`, devono essere effettuate continuamente per descrivere una traiettoria fluida nello spazio. All'interno dell'architettura di Unreal Engine, questo ciclo continuo è governato dalla funzione nativa `Tick()`, che il motore grafico richiama automaticamente a ogni singolo fotogramma generato.

Simulare un viaggio spaziale solleva però una sfida tecnica legata alla gestione del tempo. Un veicolo spaziale reale impiega molto tempo per compiere frazioni della propria orbita; eseguire una simulazione rigorosamente in "tempo reale" (dove un secondo nel simulatore equivale a un secondo reale) renderebbe l'analisi visiva della missione impossibile per l'utente. È quindi strettamente necessario introdurre un sistema di accelerazione temporale, comunemente definito *Time Warp*.

4.1 Il ciclo di simulazione e il limite del calcolo in tempo reale

Il cuore della gestione temporale risiede nella variabile `DeltaTime`. Questo parametro, fornito nativamente dal motore grafico come argomento della funzione `Tick()`, rappresenta il tempo fisico (in millisecondi) trascorso tra il completamento del fotogramma precedente e l'inizio di quello attuale. Per permettere all'utente di velocizzare la simulazione senza alterare il *framerate* del programma, l'algoritmo moltiplica il `DeltaTime` per una variabile globale denominata `TimeScale`.

Attraverso questa operazione matematica, il sistema scala il passaggio del tempo. Se la simulazione opera a 60 FPS, il `DeltaTime` reale è di circa 0.016 secondi. Se l'utente imposta un `TimeScale` di 10.000, il sistema elaborerà le forze comportandosi numericamente come se tra un frame e l'altro fossero trascorsi 160 secondi.

4.1.1 Criticità del componente Rigid Body in fase di Time Warp

La dilatazione temporale risolve il problema della durata del viaggio, ma ne introduce un secondo: il motore fisico nativo di Unreal Engine, progettato per simulare dina-

niche terrestri a velocità umane, non è adeguato per la gestione forze e spostamenti che avvengono su scale temporali così estreme. Se una forza costante venisse applicata nativamente a un *Rigid Body* utilizzando un `DeltaTime` scalato troppo ampio, il motore fisico non sarebbe in grado di calcolare correttamente la nuova posizione e velocità del veicolo, portando a risultati imprecisi o addirittura a crash del programma. Tra un calcolo e il successivo, lo spazio percorso dal corpo risulterebbe enorme: invece di descrivere una curva orbitale fluida, la sonda procederebbe su lunghi segmenti retti, uscendo inesorabilmente dalla traiettoria teorica e violando il principio di conservazione dell'energia. In pochissimi secondi di simulazione, questa instabilità numerica porterebbe l'orbita ad "allargarsi" fino a sfuggire completamente alla gravità terrestre, con conseguente perdita di controllo totale sulla simulazione.

4.2 Lo Switch Architetture

Per superare i limiti strutturali del motore fisico senza rinunciare alla precisione microscopica quando necessaria, l'algoritmo implementa uno "switch" dinamico all'inizio della funzione `Tick()`. Questo meccanismo di controllo valuta in tempo reale se la simulazione sta operando a una velocità accettabile per il motore fisico o se è necessario passare a una modalità di calcolo completamente manuale, disabilitando temporaneamente il componente di simulazione fisica del veicolo.

Listing 4.1: Logica di switch tra simulazione fisica e cinematica manuale.

```

1  bool bShouldBePhysics = (Manager->TimeScale <= 1.1f);
2  bool bCurrentlyPhysics = SAIL_MESH->IsSimulatingPhysics();
3
4  if (bShouldBePhysics && !bCurrentlyPhysics) {
5      // 1. Ritorno alla normalita': si riattiva il motore fisico
6      SAIL_MESH->SetSimulatePhysics(true);
7      SAIL_MESH->SetPhysicsLinearVelocity(FVector(SailVelocity * Manager->
8          KM_TO_UU));
9      this->CustomTimeDilation = Manager->TimeScale;
10 }
11 else if (!bShouldBePhysics && bCurrentlyPhysics) {
12     // 2. Ingresso in Time Warp: si disattiva il motore fisico
13     SailVelocity = FVector3d(SAIL_MESH->GetPhysicsLinearVelocity()) *
14         Manager->UU_TO_KM;
15     SAIL_MESH->SetSimulatePhysics(false);
16     this->CustomTimeDilation = 1.0f;
17 }

```

La soglia di transizione è impostata al valore di 1.1, che rappresenta il limite di sicurezza prima che la fisica di base inizi a degradare. Finché il moltiplicatore temporale rimane entro questo margine, il veicolo delega il calcolo del moto al solutore nativo tramite l'istruzione `SetSimulatePhysics(true)`. In questa fase, la variabile `CustomTimeDilation` dell'attore viene mantenuta sincronizzata con il `TimeScale`. In questo modo il motore fisico scala correttamente tutte le forze e i movimenti, mantenendo la coerenza della simulazione anche quando l'utente decide di rallentare o accelerare il tempo in modo moderato.

Nel momento in cui l'utente supera la soglia di sicurezza, il codice esegue il passaggio inverso. L'algoritmo salva l'ultima velocità registrata, disattiva il motore di collisione tramite `SetSimulatePhysics(false)` e imposta il tempo locale della vela a 1.0. Questa operazione "congela" l'attore dal punto di vista del motore di gioco, consentendo al codice di assumere il pieno controllo sulla posizione e sull'orientamento della sonda senza interferenze esterne. Da questo momento in poi, ogni calcolo di forza, accelerazione e spostamento viene eseguito manualmente all'interno del ciclo `Tick()`, garantendo una precisione assoluta anche a moltiplicatori di tempo estremi.

4.3 Integrazione numerica della traiettoria tramite il Metodo di Eulero

Durante la fase di *Time Warp*, la posizione tridimensionale della vela deve essere calcolata, fotogramma dopo fotogramma, a partire dalle forze totali che agiscono su di essa. Avendo calcolato la forza totale (somma vettoriale della gravità terrestre e della spinta solare) nei passaggi precedenti, il `Tick()` applica un solutore cinematico basato sul metodo di Eulero, chiamato **Integrazione di Eulero semi-implicita**. Tale metodo rappresenta una delle tecniche più semplici e dirette per integrare numericamente le equazioni del moto.

Questo algoritmo traduce la forza applicata in uno spostamento tridimensionale, elaborando i dati in tre fasi consecutive direttamente all'interno dell'istruzione `else` del blocco di switch:

Listing 4.2: Implementazione del solutore cinematico basato sul metodo di Eulero.

```

1 double SimulationDeltaTime = DeltaTime * Manager->TimeScale;
2
3 // 1. Calcolo dell'accelerazione (a = F/m)
4 FVector3d AccelMetersS2 = TotalForce / SAIL_MASS;
5 FVector3d AccelKmS2 = AccelMetersS2 / 1000.0;
6
7 // 2. Integrazione della Velocita' (V = V0 + a*dt)
8 SailVelocity += AccelKmS2 * SimulationDeltaTime;
9
10 // 3. Integrazione della Posizione (P = P0 + V*dt) e spostamento
11 FVector3d DisplacementKm = SailVelocity * SimulationDeltaTime;
12 FVector3d DisplacementUU = DisplacementKm * Manager->KM_TO_UU;
13
14 FVector3d NewPosition = SailPosition + DisplacementUU;
15 SetActorLocation(FVector(NewPosition));

```

La logica matematica implementata rispetta la rigida gerarchia della meccanica newtoniana, procedendo per tre "gradini" logici che trasformano una forza invisibile in uno spostamento concreto sulla griglia spaziale:

- **Accelerazione:** Sfruttando il secondo principio della dinamica ($F = m \cdot a$), il vettore della forza totale espressa in Newton viene diviso per la massa della vela espressa in chilogrammi (`SAIL_MASS`). Il risultato, che rappresenta un'accelerazione in m/s^2 , viene immediatamente diviso per 1000 in modo da ottenere un valore coerente con il sistema di riferimento spaziale (Km/s^2).

$$\vec{a} = \frac{\vec{F}_{tot}}{m}$$

- **Velocità:** Viene applicata la prima iterazione numerica. La nuova velocità orbitale si ottiene sommando la velocità posseduta nel fotogramma precedente con il prodotto tra l'accelerazione ricavata al passaggio precedente e il tempo scalato (`SimulationDeltaTime`).

$$\vec{v}_{new} = \vec{v}_{old} + (\vec{a} \cdot \Delta t)$$

- **Spostamento:** Eseguita la seconda iterazione, il vettore velocità viene moltiplicato per il tempo, restituendo la quantità effettiva di chilometri percorsi dalla sonda durante quel singolo fotogramma (`DisplacementKm`).

$$\vec{x}_{new} = \vec{x}_{old} + (\vec{v}_{new} \cdot \Delta t)$$

Ottenute le nuove coordinate nello spazio tridimensionale, il dato viene convertito in *Unreal Units* e la funzione nativa `SetActorLocation()` viene utilizzata per spostare forzatamente, e in modo istantaneo, il modello 3D della vela. Sostituendo il simulatore fisico continuo con questo approccio matematico discreto, il software è in grado di gestire moltiplicatori di tempo estremi mantenendo la precisione e la coerenza della traiettoria, senza incorrere in errori di calcolo o instabilità numeriche che avrebbero inevitabilmente portato a risultati errati o a crash del programma.

4.4 Aggiornamento indipendente dell'assetto

Il comportamento spaziale della vela non si limita alla semplice traslazione lungo l'orbita. Durante il volo, il veicolo deve ruotare costantemente su sé stesso affinché la sua traiettoria risulti visivamente realistica, simulando la conservazione del momento angolare di un oggetto lanciato nel vuoto.

Essendo il motore fisico disabilitato durante l'accelerazione temporale, la rotazione non può essere calcolata tramite l'applicazione di forze di torsione. Il codice gestisce quindi l'orientamento della sonda attraverso la funzione `UpdateSailRotation()`, che viene richiamata indipendentemente dalla modalità fisica o cinematica attualmente in uso.

L'obiettivo della funzione è quello di vincolare e mantenere allineato l'asse verticale della sonda (l'asse Z, ovvero il vettore "Up") alla direzione del vettore velocità (`SailVelocity`). In questo modo, la sonda espone sempre la sua faccia frontale rispetto alla direzione del moto.

Listing 4.3: Allineamento dell'assetto tramite Quaternioni in `SolarSail.cpp`.

```

1 FVector CurrentZ = GetActorUpVector();
2 FVector TargetVel = (FVector)VelocityDir;
3
4 // Calcolo della rotazione differenziale necessaria
5 FQuat DeltaRot = FQuat::FindBetweenNormals(CurrentZ, TargetOrientation);
6 FQuat TargetQuat = DeltaRot * GetActorQuat();

```

```
7 // Interpolazione sferica (Slerp) per una rotazione fluida
8 FQuat NewQuat = FQuat::Slerp(GetActorQuat(), TargetQuat, 2.0f * DeltaTime
9 );
10 SetActorRotation(NewQuat);
```

Per prevenire rotazioni "a scatti" o vibrazioni visive (che risulterebbero molto evidenti durante i rapidi cambi di direzione orbitale), il sistema evita di imporre la rotazione finale in un unico passaggio. Si affida invece all'interpolazione sferica lineare (**Slerp**) applicata ai Quaternioni.

Dal punto di vista pratico, lo **Slerp** agisce come un ammortizzatore matematico: invece di "teletrasportare" la vela istantaneamente verso il nuovo angolo, il sistema calcola il percorso più breve tra l'orientamento attuale e quello desiderato, muovendo la sonda solo di una piccola frazione ad ogni frame.

L'inserimento del **DeltaTime** in questo calcolo è il dettaglio tecnico che garantisce la stabilità del software: assicura che la velocità di rotazione rimanga costante e fluida indipendentemente dal framerate del computer o dal **TimeScale** impostato. In questo modo, anche durante le fasi di accelerazione temporale più estreme, la sonda mantiene un comportamento visivo coerente e realistico, evitando movimenti bruschi o imprevisti che comprometterebbero l'esperienza utente e la credibilità della simulazione.

Capitolo 5

Telemetria e Strumenti di Diagnostica

Un simulatore spaziale perde la sua importanza scientifica se non è in grado di fornire dati quantitativi e qualitativi sull'evoluzione della missione. La telemetria, intesa come l'insieme di strumenti e tecniche per la raccolta, visualizzazione e archiviazione dei dati di volo, è quindi un componente cruciale per qualsiasi piattaforma di simulazione orbitale.

In questo capitolo, verranno illustrate le soluzioni implementate per garantire una telemetria completa, efficiente e chiara all'interno del simulatore di vele solari. Verranno analizzati i sistemi di debug visivo in tempo reale, il meccanismo di logging persistente su file CSV e l'approccio alla post-elaborazione dei dati tramite script Python dedicati.

5.1 Telemetria Live e Debug Visivo

Durante la fase di test e simulazione, è fondamentale che l'utente comprenda istantaneamente come le forze stiano agendo sul veicolo senza dover attendere l'analisi dei dati a fine missione. A tale scopo, la funzione `DebugVisuals()` si occupa di sovrapporre alla simulazione tridimensionale una serie di indicatori vettoriali e testuali, aggiornati a ogni singolo fotogramma.

5.1.1 Rappresentazione Vettoriale e Scia Orbitale

Il primo strumento riguarda l'analisi visiva delle forze e della traiettoria. Se abilitati dal pannello del `SimulationManager`, il sistema disegna frecce direzionali tridimensionali (`DrawDebugDirectionalArrow`) ancorate al centro di massa della vela. Queste frecce rappresentano visivamente i versori della Gravità (blu), della Pressione Solare (rosso), della Forza Totale risultante (magenta) e della Normale della vela (verde).

Listing 5.1: Generazione delle frecce vettoriali per il debug delle forze in `SolarSail.cpp`.

```
1 const float ArrowLen = 1000.0f;
2 const float ArrowSize = 150.f;
3
4 // Disegno dei vettori di forza (scalati per visibilità)
5 if (Manager->ShowSailNormalDebug)
6     DrawDebugDirectionalArrow(GetWorld(), SailLoc, SailLoc + (FVector(
7         SailNormal) * ArrowLen), ArrowSize, FColor::Green, false, -1, 0,
8         5.0f);
```

```

7
8 if (Manager->ShowGravityForceDebug)
9     DrawDebugDirectionalArrow(GetWorld(), SailLoc, SailLoc + (FVector(
        GravityForceVersor) * ArrowLen), ArrowSize, FColor::Blue, false,
        -1, 0, 5.0f);
10
11 if (Manager->ShowSolarForceDebug)
12     DrawDebugDirectionalArrow(GetWorld(), SailLoc, SailLoc + (FVector(
        SolarForceVersor) * ArrowLen), ArrowSize, FColor::Red, false, -1,
        0, 5.0f);
13
14 if (Manager->ShowTotalForceDebug)
15     DrawDebugDirectionalArrow(GetWorld(), SailLoc, SailLoc + (FVector(
        TotalForce.GetSafeNormal()) * ArrowLen), 200.f, FColor::Magenta,
        false, -1, 0, 7.0f);

```

Oltre a questo, per rendere chiara e marcata la forma dell'orbita, il codice genera una scia luminosa che traccia il percorso della vela. Per evitare di saturare la memoria della RAM con migliaia di coordinate inutili, il sistema implementa un filtro spaziale e una logica a coda (FIFO):

Listing 5.2: Gestione ottimizzata della scia orbitale in `SolarSail.cpp`.

```

1 // Aggiunta di un nuovo punto solo se la distanza supera la soglia
2 float DistSq = FVector::DistSquared(OrbitHistory.Last(), SailLoc);
3 float ThresholdUU = TrailPointMinDistance * Manager->KM_TO_UU;
4
5 if (DistSq > (ThresholdUU * ThresholdUU)) {
6     OrbitHistory.Add(SailLoc);
7
8     // Rimozione del punto piu' vecchio se si supera il limite di memoria
      (FIFO)
9     if (OrbitHistory.Num() > MaxTrailPoints) {
10         OrbitHistory.RemoveAt(0);
11     }
12 }

```

La scia non memorizza la posizione a ogni singolo *Tick*, ma registra un nuovo vertice solo quando la vela ha percorso una distanza minima prestabilita, indicata dalla variabile `TrailPointMinDistance`. Inoltre, l'array `OrbitHistory` è limitato a un numero massimo di punti (`MaxTrailPoints`); raggiunto il limite, il vertice più vecchio viene eliminato tramite `RemoveAt(0)`. Una volta accumulati nell'array, i punti devono essere uniti per formare la linea continua dell'orbita. Per evitare che lo schermo si riempia di linee opache e visivamente piatte, il sistema calcola "l'età" di ogni singolo segmento basandosi sulla sua posizione all'interno della coda (tramite la variabile `Alpha`). Sfruttando questo valore percentuale, la funzione `FMath::Lerp` esegue una modifica cromatica: i segmenti più vecchi (in testa all'array) vengono disegnati con un colore scuro e semi-trasparente, mentre quelli appena generati (vicino alla sonda) vengono colorati di un ciano brillante. Questo approccio matematico crea un perfetto "effetto cometa", simulando la dissipazione della scia nel vuoto e rendendo chiara la direzione del moto a colpo d'occhio.

Listing 5.3: Rendering e interpolazione cromatica della scia orbitale in `SolarSail.cpp`.

```

1 // Disegno dei segmenti della scia
2 if (OrbitHistory.Num() > 1) {
3   for (int32 i = 0; i < OrbitHistory.Num() - 1; i++) {
4
5     // Calcolo dell'eta' del segmento (da 0.0 a 1.0)
6     float Alpha = (float)i / (float)OrbitHistory.Num();
7
8     // Sfumatura dal blu scuro/trasparente al ciano brillante
9     FLinearColor ColorLinear = FMath::Lerp(FLinearColor(0.1f, 0.0f, 1.0f,
10       0.5f), FColor::Cyan, Alpha);
11
12     DrawDebugLine(
13       GetWorld(),
14       OrbitHistory[i],
15       OrbitHistory[i + 1],
16       ColorLinear.ToFColor(true),
17       false, -1, 0,
18       150.0f // Spessore visivo della linea
19     );
20 }
21 // Collegamento finale tra l'ultimo punto salvato e la posizione attuale
22 // della sonda
23 DrawDebugLine(GetWorld(), OrbitHistory.Last(), SailLoc, FColor::Cyan,
24   false, -1, 0, 150.0f);
25 }

```

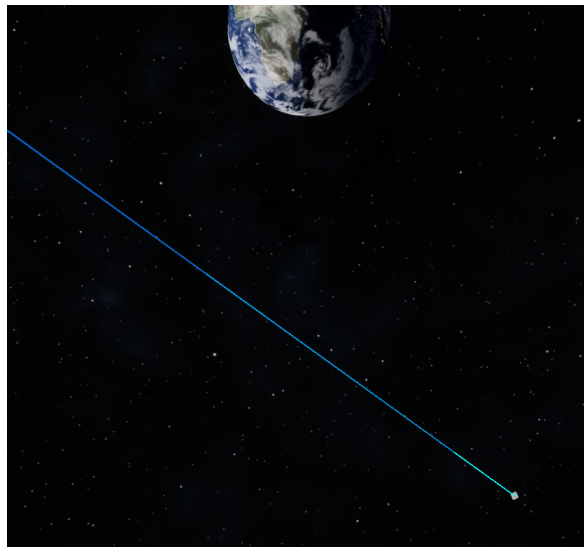


Figura 5.1: Rappresentazione visiva della scia orbitale generata da una vela solare.

5.1.2 Debug Ottico del Ray Casting

Come analizzato nel Capitolo 4, l'interazione della luce con la membrana viene calcolata tramite il lancio di raggi virtuali. La funzione `DebugVisuals()` estrae gli array generati durante quel calcolo (`LocalActiveRays` e `LocalBlockedRays`) e li converte nuovamente in coordinate globali del mondo tramite la matrice di trasformazione dell'attore

(`CurrentTrans.TransformPosition`). Il motore grafico disegna quindi un fascio di laser verdi per i fotoni che impattano correttamente sulla vela e rossi per quelli che si trovano nella zona d'ombra generata da eventuali ostacoli, fornendo così un feedback visivo immediato sulla distribuzione dei raggi e sull'efficacia dell'orientamento della vela rispetto alla posizione del Sole.

Listing 5.4: Rendering dei laser diagnostici per il Ray Casting in `SolarSail.cpp`.

```
1  if (Manager->ShowPhotonDebug) {
2      FTransform CurrentTrans = GetActorTransform();
3      FVector SunDir = (Manager->SUN_POSITION * Manager->KM_TO_UU - SailLoc
4          ).GetSafeNormal();
5      float RayLen = 200.0f; // Lunghezza fissa visiva del raggio
6
7      // Rendering dei Raggi Verdi (Tasselli illuminati)
8      for (const FVector& LP : LocalActiveRays) {
9          FVector WorldStart = CurrentTrans.TransformPosition(LP);
10         FVector WorldEnd = WorldStart + (SunDir * RayLen);
11         DrawDebugLine(GetWorld(), WorldStart, WorldEnd, FColor::Green,
12             false, -1, 0, 1.0f);
13     }
14
15     // Rendering dei Raggi Rossi (Tasselli in ombra)
16     for (const FVector& LP : LocalBlockedRays) {
17         FVector WorldStart = CurrentTrans.TransformPosition(LP);
18         FVector WorldEnd = WorldStart + (SunDir * RayLen);
19         DrawDebugLine(GetWorld(), WorldStart, WorldEnd, FColor::Red,
20             false, -1, 0, 1.0f);
21     }
22 }
```

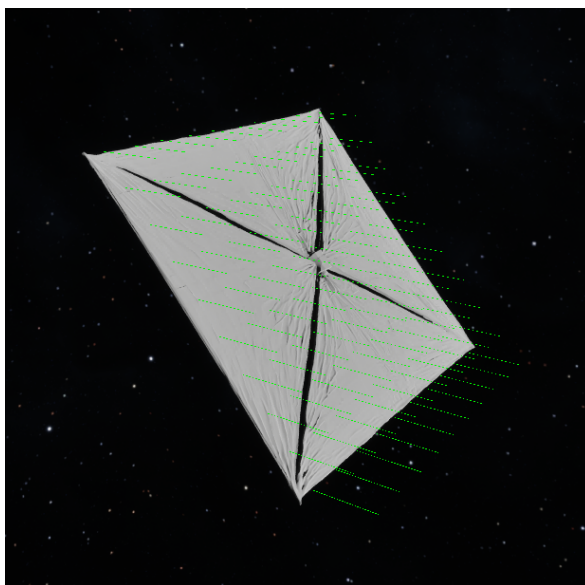


Figura 5.2: Rappresentazione visiva dei raggi di luce che colpiscono la vela (verde) e quelli bloccati da ostacoli (rosso).

5.1.3 L'Interfaccia Telemetrica a Schermo (HUD)

Per fornire dati numerici in tempo reale senza dover aprire alcun menu, il codice inietta direttamente sullo schermo dell'utente una *Heads-Up Display* (HUD) basata su testo. Poiché la simulazione può ospitare decine di vele contemporaneamente, il sistema esegue innanzitutto una raccolta dinamica di tutti gli attori di classe `ASolarSail` presenti nel mondo virtuale. Prima di stampare i dati, l'array viene ordinato in modo deterministico tramite una funzione Lambda basata sul `SailIndex`:

Listing 5.5: Ordinamento dinamico delle vele per la stampa a schermo.

```
1 ValidSails.Sort([](const ASolarSail& A, const ASolarSail& B) {
2     if (A.SailIndex == B.SailIndex) {
3         return A.GetActorLabel() < B.GetActorLabel();
4     }
5     return A.SailIndex < B.SailIndex;
6 });
```

Per ogni vela individuata nella scena, l'algoritmo assembla un blocco di stringhe che funziona come vero e proprio pannello di controllo in cui i dati spaziali (Velocità, Distanza, Forza Solare e Area) vengono formattati e impaginati a colonna. Per massimizzare la leggibilità, l'efficienza propulsiva viene tradotta in un indicatore grafico (`AsciiBar`) che calcola il rapporto percentuale tra fotoni attivi e totali, riempiendo proporzionalmente una barra da 10 segmenti visivi. Il blocco di testo viene infine inserito nel ciclo di rendering del motore tramite `AddOnScreenDebugMessage`, assegnando colori differenti per evidenziare eventuali cali di efficienza o l'ingresso in una zona d'ombra planetaria.



Figura 5.3: Esempio di HUD testuale a schermo che mostra i dati telemetrici in tempo reale per una vela solare.

Listing 5.6: Logica di ordinamento e generazione dell'HUD telemetrico in `SolarSail.cpp`.

```
1 // Ordinamento deterministico delle vele tramite funzione Lambda
2 ValidSails.Sort([](const ASolarSail& A, const ASolarSail& B) {
3     if (A.SailIndex == B.SailIndex) {
4         return A.GetActorLabel() < B.GetActorLabel();
5     }
6     return A.SailIndex < B.SailIndex;
```

```

7 });
8
9 // Rendering dell'HUD testuale per ogni vela validata
10 for (int32 i = 0; i < ValidSails.Num(); ++i) {
11     ASolarSail* S = ValidSails[i];
12     int32 BaseKey = RootKey - (i * LinesPerSail);
13
14     // Calcolo efficienza propulsiva e generazione barra grafica ASCII
15     float EfficiencyVal = (S->TotalPhotons > 0) ? ((float)S->
        ActivePhotons / (float)S->TotalPhotons) : 0.0f;
16     int32 FilledSegments = (int32)(EfficiencyVal * 10);
17     FString AsciiBar = TEXT("");
18
19     for (int32 b = 0; b < 10; b++) {
20         AsciiBar += (b < FilledSegments) ? TEXT("#") : TEXT("-");
21     }
22
23     // Impostazione cromatica condizionale
24     FColor C_Warn = (EfficiencyVal > 0) ? FColor::Green : FColor::Red;
25     FString StatusStr = (S->ActivePhotons > 0) ? TEXT("ACTIVE") : TEXT("
        SHADOW");
26
27     // Impaginazione dati (esempio intestazione e riga efficienza)
28     FString R00 = FString::Printf(TEXT("+-[ID %d]--[%s]-----"), S
        ->SailIndex, *S->GetActorLabel());
29     FString R10 = FString::Printf(TEXT("| Efficiency: [%s] %s"), *
        AsciiBar, *StatusStr);
30     // [...] (omissione delle restanti righe spaziali e vettoriali)
31
32     // Stampa a schermo tramite il motore grafico
33     GEngine->AddOnScreenDebugMessage(BaseKey - 0, 1.0f, FColor::Cyan, R00
        );
34     // [...]
35     GEngine->AddOnScreenDebugMessage(BaseKey - 10, 1.0f, C_Warn, R10);
36 }

```

5.2 Il Sistema di Logging Persistente (CSV)

Mentre la telemetria live è utile all'utente durante l'esecuzione, l'analisi scientifica rigorosa richiede che i dati vengano salvati in modo persistente su file per poter essere esaminati a posteriori. Per questo motivo, è stata implementata una funzione dedicata alla scrittura di un file per ogni singola vela solare, garantendo così un'archiviazione ordinata e facilmente accessibile dei dati di volo.

Il codice accede alle directory del sistema operativo interrogando la classe nativa `FPaths::ProjectDir()` e genera una cartella dedicata denominata "Csvs". Per prevenire collisioni nei salvataggi, il sistema costruisce dinamicamente il nome del file concatenando la dicitura *SolarSailData_* con l'identificativo univoco dell'attore (*SailName*), garantendo l'isolamento dei dati per ogni specifica vela.

Listing 5.7: Inizializzazione del file system e creazione dinamica dei file CSV in `SolarSail.cpp`.

```

1 void ASolarSail::InitializeCSVReporting() {
2     // 1. Costruzione sicura e multiplatforma dei percorsi
3     FString ProjectDir = FPaths::ProjectDir();
4     FString CsvsDir = FPaths::Combine(ProjectDir, TEXT("Csvs"));
5
6     // 2. Creazione della directory con controllo di sicurezza
7     if (!FFileManager::Get().MakeDirectory(*CsvsDir, true)) {
8         UE_LOG(LogTemp, Error, TEXT("[%s] Impossibile creare la cartella
9             Csvs!"), *SailName);
10        return;
11    }
12
13    // 3. Generazione dinamica del nome file per prevenire sovrascritture
14    FString FileName = FString::Printf(TEXT("SolarSailData_%s.csv"), *
15        SailName);
16    CsvFilePath = FPaths::Combine(CsvsDir, FileName);
17
18    bCsvHeaderWritten = false;
19    UE_LOG(LogTemp, Warning, TEXT("[%s] Log pronto in -> %s"), *SailName,
20        *CsvFilePath);
21 }

```

Il formato scelto per l'esportazione è il CSV, uno standard leggero, scalabile e universalmente compatibile con qualsiasi ambiente di calcolo.

5.2.1 Ottimizzazione I/O e gestione del Buffer temporale

Salvare fisicamente file su un disco rigido è un'operazione notoriamente lenta. Se il motore grafico tentasse di eseguire una scrittura sul disco 60 volte al secondo (per ogni singolo *Tick* e per ogni singola vela), il sistema andrebbe incontro a un gravissimo rallentamento, con un impatto devastante sulle prestazioni e sulla fluidità della simulazione, ma soprattutto compromettendo la precisione del solutore cinematico.

Per ovviare a questo grave problema di ottimizzazione del software, la funzione `AppendDataToCSV()` implementa una tecnica di *buffering* temporale:

Listing 5.8: Sistema di buffering per l'ottimizzazione delle scritture su disco in `SolarSail.cpp`.

```

1 // Creazione della stringa dati per il fotogramma corrente
2 FString DataRow = FString::Printf(TEXT("%s,%.3f,%.2f,%.4f,%.4f,%.4e,%.4e,
3     ,%.2f,%d\n"),
4     *SailName, GetWorld()->GetTimeSeconds(), SailDistanceFromEarth,
5     SailDistanceFromSun / 149597870.7, SailVelocity.Size(), TotalForce.
6     Size(),
7     SolarPressure, IncidenceAngle, ActivePhotons);
8
9 // Accumulo dei dati nella memoria RAM temporanea (Buffer)
10 CsvBuffer += DataRow;
11 CsvWriteTimer += DeltaTime;
12
13 // Scrittura fisica su disco solo ogni 1.0 secondi di tempo reale
14 if (CsvWriteTimer > 1.0f) {
15     FFileHelper::SaveStringToFile(CsvBuffer, *CsvFilePath,
16         FFileHelper::EEncodingOptions::
17             AutoDetect,

```

63.3. POST-PROCESSING E VISUALIZZAZIONE (SCRIPT PYTHON)

```
15         &IFileManager::Get(), FILEWRITE_Append)
16         ;
17     CsvBuffer.Empty(); // Svuota il buffer dopo la scrittura
18     CsvWriteTimer = 0.0f;
19 }
```

Con questa strategia, le righe contenenti i dati del singolo fotogramma non vengono scritte su file istantaneamente. Vengono invece concatenate all'interno di una singola e lunga variabile testuale immagazzinata nella memoria RAM (`CsvBuffer`), che è molto più veloce rispetto a qualsiasi operazione di I/O su disco.

Solo quando il timer `CsvWriteTimer` certifica che è trascorso 1.0 secondo esatto di tempo reale, il sistema "scarica" in blocco tutto il testo accumulato scrivendolo in coda al file fisico (tramite la *flag* `FILEWRITE_Append`). Subito dopo, il buffer in memoria viene svuotato per ricominciare il ciclo.

Questa tecnica di buffering riduce drasticamente il numero di scritture su disco, mantenendo al contempo un flusso costante e affidabile di dati. Inoltre, per garantire che nessun dato venga perso in caso di chiusura improvvisa della simulazione, la funzione `EndPlay()` esegue un'ultima scrittura forzata del buffer residuo, assicurando così che anche i frammenti di dati più recenti vengano salvati correttamente prima di terminare definitivamente la simulazione.

5.3 Post-Processing e Visualizzazione (Script Python)

Una volta salvati i dati nei rispettivi file CSV, è necessario poterli visualizzare in modo grafico. In questo modo, è possibile studiare l'andamento della traiettoria nel tempo e verificare la correttezza matematica del simulatore rispetto alle leggi fisiche.

A tale scopo, invece di appesantire l'architettura di Unreal Engine integrando complesse librerie utente (*UI*) per la generazione di grafici bidimensionali a schermo, si è scelto di mantenere diviso il codice di simulazione da quello di visualizzazione.

Il simulatore in C++ agisce esclusivamente come solutore numerico e fornitore di dati. La fase diagnostica e visiva vera e propria è invece stata interamente delegata a uno script esterno indipendente, programmato in Python.

5.3.1 Automazione e Librerie Scientifiche

Lo script, denominato `SolarSailPlot.py`, è stato strutturato per automatizzare il ciclo di lettura e stampa visiva senza richiedere all'utente di selezionare manualmente i file generati dal simulatore. All'avvio, il programma naviga autonomamente nella directory nativa di Unreal Engine, esegue un *parsing* della cartella *Csvs* condivisa e importa tutti i documenti disponibili. Sfruttando la potente libreria matematica `pandas`, il codice compila il *dataset*, correggendo eventuali errori di formattazione.

Listing 5.9: Automazione della ricerca file e importazione dati tramite Pandas in `SolarSailPlot.py`.

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import glob
4 import os
```

```

5
6 # 1. Ricerca automatica dei file CSV nella cartella di Unreal Engine
7 SIMULATION_CSV_DIR = os.path.abspath(os.path.join(SCRIPT_DIR, '..', '
      Simulation', 'Csvs'))
8 sim_files = glob.glob(os.path.join(SIMULATION_CSV_DIR, "*.csv"))
9
10 # [...] (logica di copia dei file nella directory locale)
11
12 # 2. Creazione della figura con 2 grafici impilati (Distanza e Velocità)
13 fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)
14
15 # 3. Lettura robusta del CSV tramite pandas
16 data = pd.read_csv(csv_path, skipinitialspace=True)

```

Per il grafico risultante, lo script impiega `matplotlib` per generare due grafici impilati verticalmente: il primo dedicato alla distanza radiale e il secondo alla velocità orbitale, entrambi in funzione del tempo:

1. **Grafico della Distanza Radiale:** Traccia la distanza in chilometri tra la vela e il centro della Terra. Questa curva rappresenta lo strumento primario per misurare l'efficacia propulsiva della vela: permette di visualizzare a colpo d'occhio il progressivo innalzamento dell'orbita generato dalla spinta continua e cumulativa dei fotoni solari sulla membrana.
2. **Grafico della Velocità Orbitale:** Mappa le oscillazioni cinetiche del veicolo. Oltre a confermare il rispetto delle leggi kepleriane (la sonda decelera allontanandosi dal pianeta e accelera riavvicinandosi), l'analisi di questa linea rivela come la pressione di radiazione applichi costantemente nuova energia nel sistema, alterando nel tempo il bilancio complessivo della vela rispetto a quello di un normale satellite.

Il vero vantaggio di questo approccio a script risiede nella sua nativa rappresentazione di grafici multi-vela. In una simulazione in cui si stanno per esempio eseguendo tre vele solari caratterizzate da tre coefficienti di riflettività differenti, lo script è in grado di sovrapporre automaticamente le tre curve di distanza e le tre curve di velocità, assegnando a ciascuna un colore diverso (`['b', 'g', 'r', 'c', 'm', 'y', 'k']`) e compilando automaticamente la legenda a schermo per facilitare il confronto visivo.

Listing 5.10: Ciclo di sovrapposizione multi-veicolo e plotting vettoriale.

```

1 # Colori ciclici per distinguere visivamente le diverse vele
2 colors = ['b', 'g', 'r', 'c', 'm', 'y', 'k']
3
4 for idx, csv_path in enumerate(csv_files):
5
6     # Generazione automatica dell'etichetta legenda ripulendo il nome del
       file
7     label = os.path.basename(csv_path).replace("SolarSailData_", "").
       replace(".csv", "")
8     color = colors[idx % len(colors)]
9
10    # Estrazione delle colonne di interesse dai dati caricati
11    time = data['Timestamp(s)']

```

79.3. POST-PROCESSING E VISUALIZZAZIONE (SCRIPT PYTHON)

```
12 dist_earth = data['DistEarth(Km)']
13 velocity = data['Velocity(Km/s)']
14
15 # Plotting sovrapposto sui due assi
16 ax1.plot(time, dist_earth, label=label, color=color, linewidth=2)
17 ax2.plot(time, velocity, label=label, color=color, linestyle='--',
18         linewidth=1.5)
19 # [...] (formattazione griglia, etichette e salvataggio dell'immagine PNG
20 )
```

In questo modo, l'utente ottiene un quadro completo immediato, pulito ed esportabile per la stesura di *paper* accademici o analisi prestazionali sui materiali della vela.

Bibliografia

- [1] M. Rodrigues, “Solar sails: Academic report and technology review,” tech. rep., University Research Report, 2023.
- [2] C. R. McInnes, *Solar Sailing: Technology, Dynamics and Mission Applications*. Springer Praxis Books, 1999.
- [3] L. McConnell *et al.*, “Solar cruiser: Enabling high-inclination solar science,” in *AIAA SCITECH 2023 Forum*, 2023.
- [4] L. Johnson, “Solar sail propulsion: Status report,” tech. rep., NASA Marshall Space Flight Center, 2012.
- [5] D. A. Spencer, B. Betts, *et al.*, “The lightsail 2 mission: Flight results and lessons learned,” *Acta Astronautica*, 2023.
- [6] C. Ericson, *Real-Time Collision Detection*. CRC Press, 2004. Reference for Ray Casting algorithms.
- [7] J. Simo and C. R. McInnes, “Solar sail trajectory design with a realistic optical model,” *Journal of Guidance, Control, and Dynamics*, 2016.
- [8] Epic Games, “Unreal engine 5 documentation,” 2024. Consultato per l’architettura dei componenti, il sistema fisico Chaos e le funzioni di Ray Casting.